

Software Language Engineering (SLE): Lectures

Erwan Bousse

Version 2026-2027, 2026-10-22

Table of Contents

1. 📚 Ch1. Introduction and motivation . . . 1
   1.1. Abstractions: the heart and soul of software engineering . . . 1
   1.2. Software languages as powerful (but costly) abstractions . . . 2
   1.3. Software Language Engineering (SLE) . . . 3
2. 📚 Ch2. Principles of SLE . . . 7
   2.1. Design of a software language . . . 7
   2.2. Implementation of a software Language . . . 9
   2.3. Engineering a DSL . . . 12
3. 📚 Ch3. The Langium Language Workbench . . . 19
   3.1. Langium overview . . . 19
   3.2. Installing a Langium development environment . . . 21
   3.3. Preparing a Langium project . . . 22
   3.4. Working on a Langium project . . . 24
   3.5. Packaging and distributing a Langium DSL . . . 26
4. 📚 Ch4. Abstract syntax . . . 29
   4.1. Designing an abstract syntax . . . 29
   4.2. Implementating an abstract syntax in Langium . . . 41
   4.3. Manually constructing an AST in Langium . . . 46
5. 📚 Ch5. Textual concrete syntax . . . 55
   5.1. Designing a textual concrete syntax . . . 55
   5.2. Implementing a textual concrete syntax in Langium . . . 73
   5.3. Testing a generated editor of a Langium language . . . 89
6. 📚 Ch5. Semantics with code generation . . . 91
   6.1. Types of semantics . . . 91
   6.2. Designing a code generation semantics . . . 95
   6.3. Implementation of a code generation semantics . . . 96
   6.4. References . . . 110

Chapter 1. Ch1. Introduction and motivation

1.1. Abstractions: the heart and soul of software engineering

While initially software was written in the form of *binary code* or *assembly code* that computers could directly understood, it became rapidly necessary to write software using reusable **abstractions** (such as *loops*, *conditionals*, *functions*) that *generalized* common redundant patterns found in code. Not only these abstractions were a way to avoid redundancies, but they also started providing higher-level *concepts* and *ideas* adapted to the mind of software engineers, and to the problems they needed to solve. This eventually led to the birth of *general-purpose programming languages* (GPLs) in their modern form, such as Pascal [https://en.wikipedia.org/wiki/Pascal_(programming_language)], Fortran [https://en.wikipedia.org/wiki/Fortran], C [https://en.wikipedia.org/wiki/C_(programming_language)], Java [https://en.wikipedia.org/wiki/Java_(programming_language)], and many others [https://en.wikipedia.org/wiki/Third-generation_programming_language].

In this chapter, we understand the word *abstraction* as the act of **identifying common characteristics of different entities, and defining these characteristics as proper self-reliant concepts**. Or, as written on the Wiktionary [https://en.wiktionary.org/wiki/abstraction]:

> The act of comparing commonality between distinct objects and organizing using those similarities; the act of generalizing characteristics; the product of said generalization.
>
> — 6th definition of the word "abstraction", Wiktionary (accessed on 16/11/2024)

While GPLs are key abstractions used to build software, the whole field of software engineering nowadays continuously relies on **the definition of suitable and efficient new abstractions to solve always more complex problems**. This abstraction-centered engineering process can be summarized as follows:

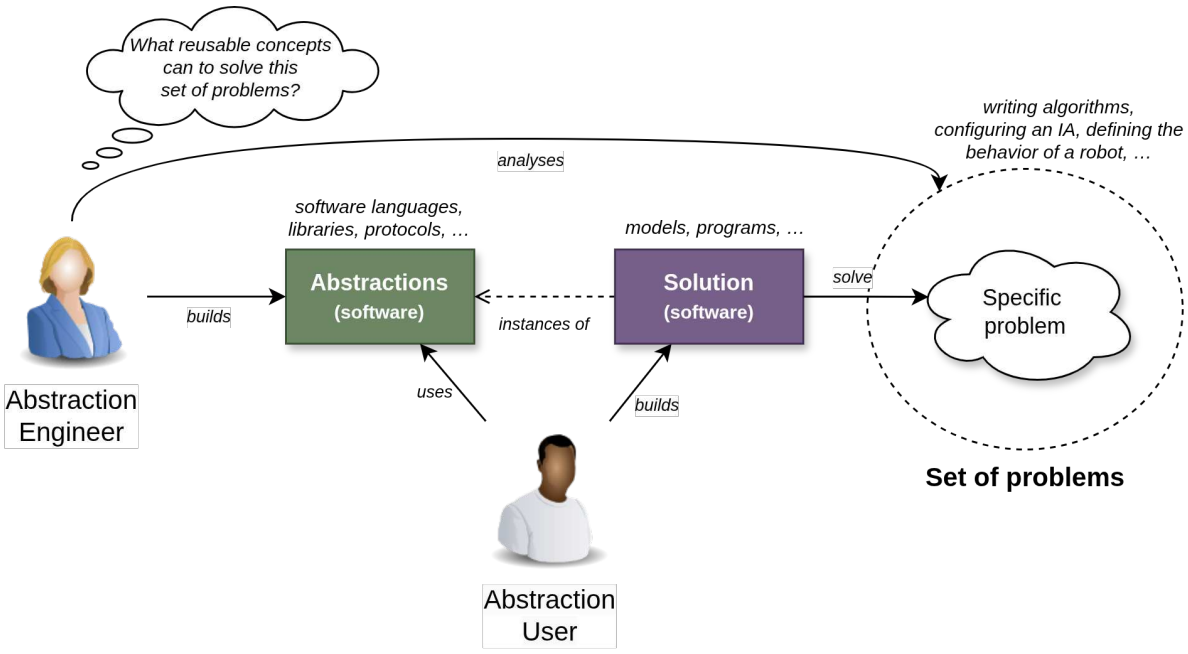


Figure 1. Engineering abstractions and using abstractions to solve problems

- On the right, a given **set of problems** is depicted. The size of a set of problems can greatly vary, ranging from "*writing data structures and algorithms to be executed on computers*" (the very large set of problems addressed by GPLs), to "*writing Java test cases*" (the smaller set of problems addressed by the JUnit framework [https://junit.org]), or to something even more domain-specific

such as "*exchanging instant messages over the network*" (problems addressed by the Matrix communication protocol [https://matrix.org/]).

- On the left, the **abstraction engineer** is a software engineer that analyzes a given set of problems, and produces a set of reusable *abstractions* as a result (such as a software language, library, framework, or protocol). These abstractions are made of meticulously identified *concepts* fitting to solve a given set of problems. Consequently, engineering abstractions requires a very good understanding of the *domain of expertise* related to the set of problems.
- At the bottom, the **abstraction user** is a software engineer that take advantage of existing abstractions in order to build a *solution* (typically a software model or a program) to an specific problem. For the abstractions to be useful, the problem at stake must be part of the set of problems targeted by the abstractions.



In 99% of cases, *defining new abstractions requires pre-existing abstractions*! For example, implementing a new software library is achieved using an existing software language and using other existing libraries (called *dependencies*). Consequently, the *abstraction engineer* is always also an *abstraction user*.

Abstractions can take *many forms* in the great wild of software engineering, such as:

- **Operating systems (OSs) [https://en.wikipedia.org/wiki/Operating_system] and device drivers [https://en.wikipedia.org/wiki/Device_driver]** provide abstractions to manage *hardware resources*. For instance, a software application can use the *file system* abstraction provided by an OS to interact with a storage device.
- **Communication protocols [https://en.wikipedia.org/wiki/Communication_protocol]** provide abstractions allowing two or more entities to exchange information. For instance, the HTTP [https://en.wikipedia.org/wiki/HTTP] protocol provide abstractions (`GET`, `POST`, `PUT`, etc.). to interact with a web server.
- An **Application programming interface (API) [https://en.wikipedia.org/wiki/API]** provides a set of abstractions to programmatically interact with an existing piece of software (a server, a library, an application, etc.). This allows such piece of software to be used by other code without exposing any of its internals.
- A **software library [https://en.wikipedia.org/wiki/Library_(computing)]** is a collection of abstractions in the form of reusable *data structures* and *functions* targeting a particular computing problem, and that can serve to create new software. A software framework [https://en.wikipedia.org/wiki/Software_framework] is a specific sort of library where parts of the library are *extensible* and can be redefined for a specific use.
- Last but not least, **software languages** (which include both programming languages [https://en.wikipedia.org/wiki/Programming_language] and modeling languages [https://en.wikipedia.org/wiki/Modeling_language]) are fundamental abstractions used to design and build software. We focus on this unique category of abstractions in the following section.

1.2. Software languages as powerful (but costly) abstractions

Software languages can come in many different shapes and forms:

- **General-purpose programming languages (GPLs)** (Python, Java, C++, etc.) are the best known software languages. They provide all commonly expected abstractions to write algorithms and data structures. They are used to write programs for the vast majority of software today. Being general-purpose, these languages are often quite large and complex, and therefore require a lot of engineering effort for their creation and maintenance. Their lifespan is usually very long, and their user base very large.

- A **modeling language** (UML, SysML, AADL, etc.) is a software language commonly used to *design* software and systems. They often provide high-level abstractions specialized for specific concerns or processes (for instance AADL focuses on *architecture design*). While some modeling languages have precise execution semantics (for example Petri nets and Statecharts) and can be executed as programming languages, other modeling languages do not have semantics precise enough to be executable (for example UML). Their notation is often based on graphical shapes, arrows, and diagrams, making them easy to use on drawing boards.
- A **domain-specific languages (DSLs)** is a programming or modeling language of smaller size designed for a much smaller set of problems. The problems that are covered by a given DSL – also called the *domain* covered by the DSL – can be related both to *business concerns* (salary calculation, robotics, hospital management, etc.) and more cross-cutting *technical concerns* (security policies, concurrent behaviors, user interfaces). A DSL is typically created for a limited set of users of for a specific engineering needs (for example the needs of a specific branch of a specific company), with a shorter lifespan and maintenance effort than a GPL.



The line between a software language of "normal size" and a DSL is not strict at all, and it can be sometimes difficult to decide whether a given software language qualifies as a DSL or not. For example, some considers *HTML* to be a DSL because it focuses on a very specific and limited technical concern (which is defining the contents of a graphical user interface), while some consider it is too broadly used and has too many applications to qualify as a DSL.



Low-code development relies on simplified development environments called Low-code development platforms (LCDP) [https://en.wikipedia.org/wiki/Low-code_development_platform] that can be used by non-software experts to write software components. An LCDP is usually built around a set of well-defined DSLs easy to use and understand.

Compared with other forms of abstractions (such as software librairies), software languages have a characteristic making them *unique*: **a software language defines a fully dedicated *syntax* for all the abstractions it provides**. This syntax can be either textual (such as Java) or graphical (such as UML), and can be extremely powerful in how it allows an engineer to *directly* manipulate the provided abstractions.



On the opposite, using the abstractions provided by a *software library* requires being skilled with the programming language the library was made for.

We presented in the previous section the always growing need to define *new abstractions* to tackle the always growing complexity of problems to solve. While it remains more common today to create new abstractions in the form of software libraries, **there is a rising trend to rely on the engineering of new software languages – especially DSLs – to answer the need for abstractions**. This is especially true when *non-software experts* are involved in the engineering process, and can therefore remarkably benefit from a dedicated DSL syntax both tailored for their domain of expertise, and simpler to learn than GPLs. For example, a DSL to configure the behavior of specific physical machines can be designed to be used directly by the experts of said machines, without the contant help of a software engineer.

1.3. Software Language Engineering (SLE)

It can be observed with little surprise that implementing a new software language, with its own interpreter/compiler and tools, can be a costly endeavor. The field of *Software Language Engineering* (*SLE*) emerged for this purpose: providing **dedicated methods, principles, approaches and tools to**

efficient engineer new software languages with limited cost.

1.3.1. SLE as an engineering problem and discipline

SLE is a specific case of *abstraction engineering* that can be summarized as follows:

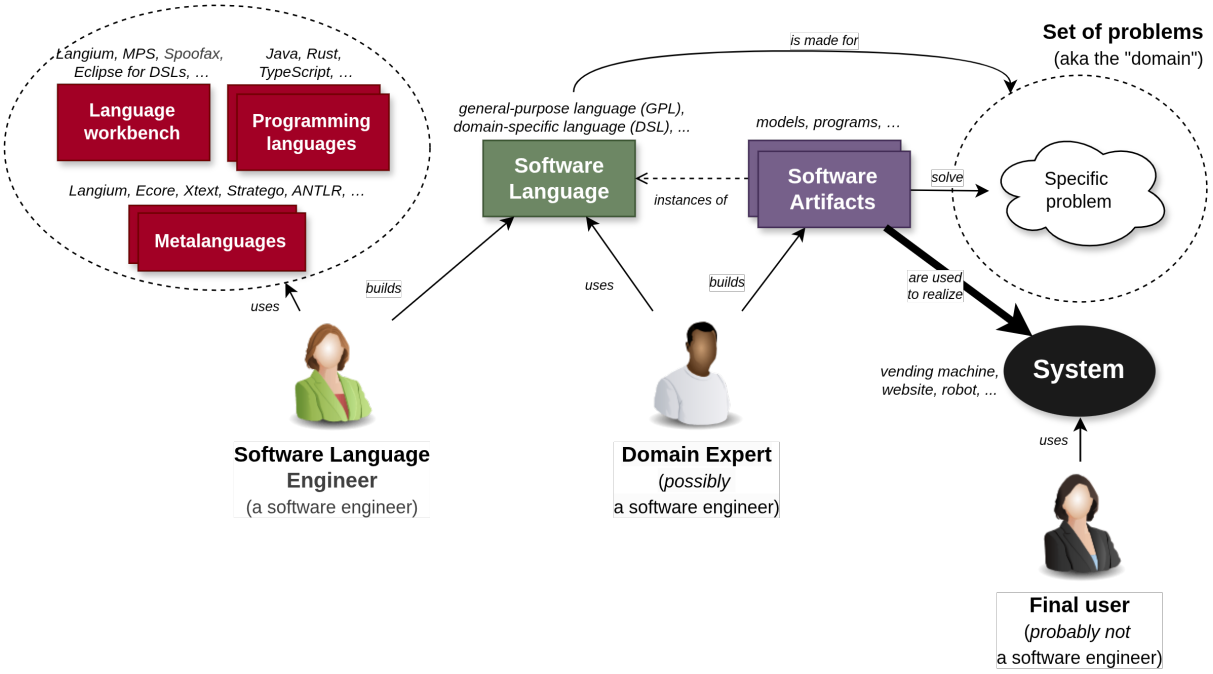


Figure 2. Software Language Engineering (SLE)

- On the left, the **software language engineer** is the role responsible for designing, building and maintaining a **software language**. This person is a specific kind of software engineer that is specialized in the making of languages. Their goal is to provide a software language that is made to solve a specific set of problems called the *domain*. To achieve this goal, the software language engineer relies on a diverse toolbox that includes:
  - *Metalanguages*, which are specific kinds of software languages dedicated to the engineering of new software languages. This includes languages to implement grammars (eg. ANTLR, Langium), languages to implement graphical syntaxes (eg. Sirius, Langium), languages to implement abstract syntaxes (eg. Langium, Ecore), or languages to traverse and transform abstract-syntax trees (eg. Stratego, Kermeta, ATL).
  - *Programming languages*, which are well-known traditionnal software languages (eg. Java, Rust, TypeScript). Since a sofware language is made of software, programming languages can be used to implement all parts of a software language (eg. OpenJDK is implemented in Java, C++, C), either in place of metalanguages or in addition to metalanguages.
  - A *language workbench* (eg. Langium, MPS), which is a set of tools dedicated to the engineering of software languages. A language workbench may be provided in the form of an Integrated Development Environment (IDE), and typically includes a set of *metalanguages* to implement the different parts of a language. Most language workbenches are dedicated to the engineering of domain-specific languages (DSLs), instead of bigger general-purpose programming languages (GPLs).
- In the middle, the **domain expert** is the user of the software language. This person is a specialist of the considered *domain*, and hence should easily use the language engineered specifically for this domain. Using the provided software language, the domain expert produces a set of *software artifacts* (programs, or models) which are used in the concrete realization of a *system* (eg. a vending machine, a website, a robot…).



Depending on the context and domain, **the domain expert *may or may not* be a software engineer!** In particular, a DSL can be made for experts of a domain that is not directly related to computer science, such as a DSL for electrical engineers, or for hospital management, or railway design...

- Finally, on the right, the **final user** is the person that uses the resulting system.

In this process, each role provides a service to the next: the software language engineer provides a *language*, used by the domain expert to provide a *system* to the final user. This means that, from the software language engineer perspective, the "user" is the domain expert, while from the domain expert perspective it is the "final user" shown in the figure.

1.3.2. SLE for engineering complex systems

We have seen the importance of *abstractions* to be able to build systems, and we have seen that *software languages* are a specific and powerful kind of abstractions to achieve this goal. We have also seen that many sorts of software languages exist, such as general-purpose programming languages (GPLs), modeling languages, and domain-specific languages (DSLs).

However, if we focus on the engineering of **complex systems** (eg. cyberphysical systems such as cars or airplanes), it is often impossible to design and implement a single whole system using a single software language. Each facet of the system (security, structure, behavior, ...) requires specific abstractions, which are each best provided by a corresponding software language. We can therefore end-up with a situation where **heterogeneous software languages** are used together to build the system:

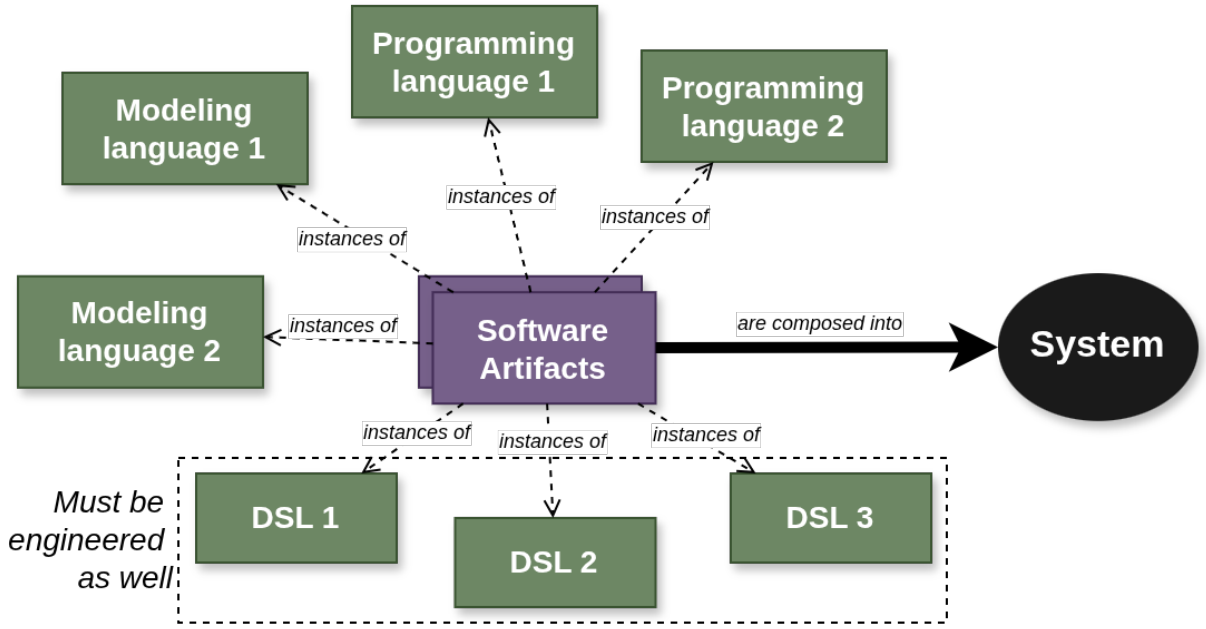


Figure 3. Engineering a system with heterogenerous languages

As shown above, it is possible to rely on specialized DSLs to engineer specific facets of a complex system. While some DSLs are available *off-the-shelf*, **it is often required to engineer custom DSLs adapted to the modeling/programming needs of the target system**. In this case, SLE become a central part of the system engineering: we build the languages the best adapted to the problems we aim to solve, ie. the languages that best match the *domains* covered by the system.

SLE as a foundation for MDE and LOP

Different methods and paradigm rely heavily on DSLs and therefore on SLE, in particular:

- **Model-driven engineering (MDE)** is a method that heavily relies on *models* (and hence modeling languages) to design the different facets of an system, and then to process models as first-class artifacts to automatically produce simulations or generate code. While MDE can rely on well-established modeling languages, such as UML or Simulink, a great part of MDE relies on the engineering of DSLs suitable for the different facets of the system.
- **Language-oriented programming (LOP)** is a programming paradigm where a software system is not built using traditionnal programming languages, but is entirely built using DSLs, which are themselves made using SLE. This vision implies that a software engineer *must also* be a language engineer in order to build a system.

Chapter 2. Ch2. Principles of SLE

This chapter presents the key principles of *Software Language Engineering* (SLE), a set of of software engineering methods and techniques to efficiently **create and maintain new software languages**.

2.1. Design of a software language

A **software language** can generally be *designed* with the following ingredients:

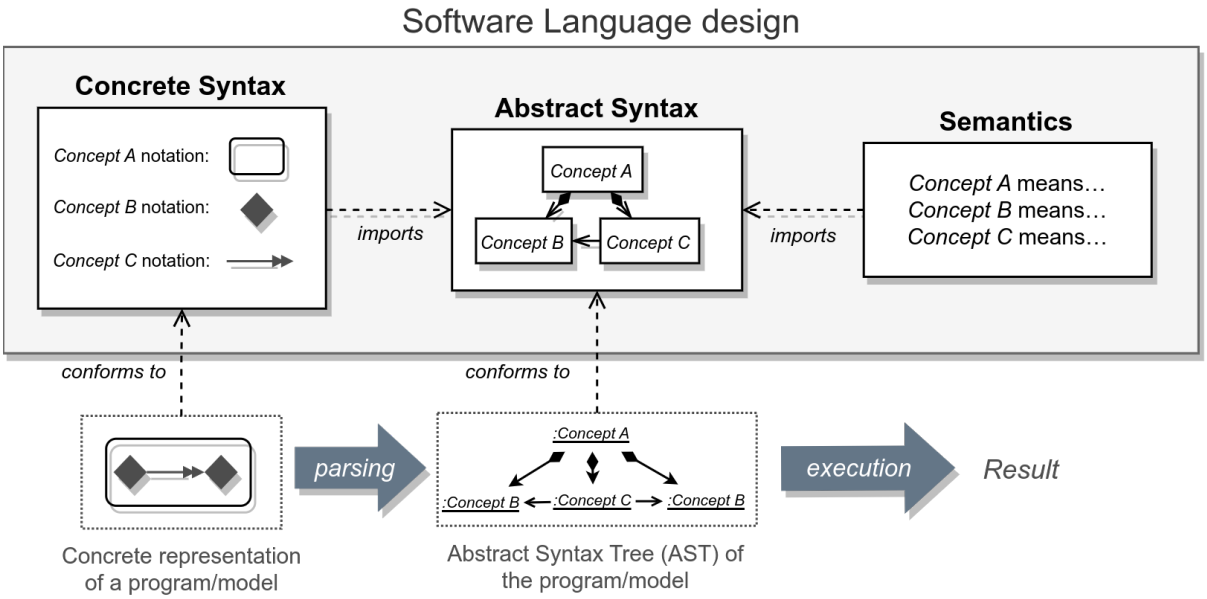
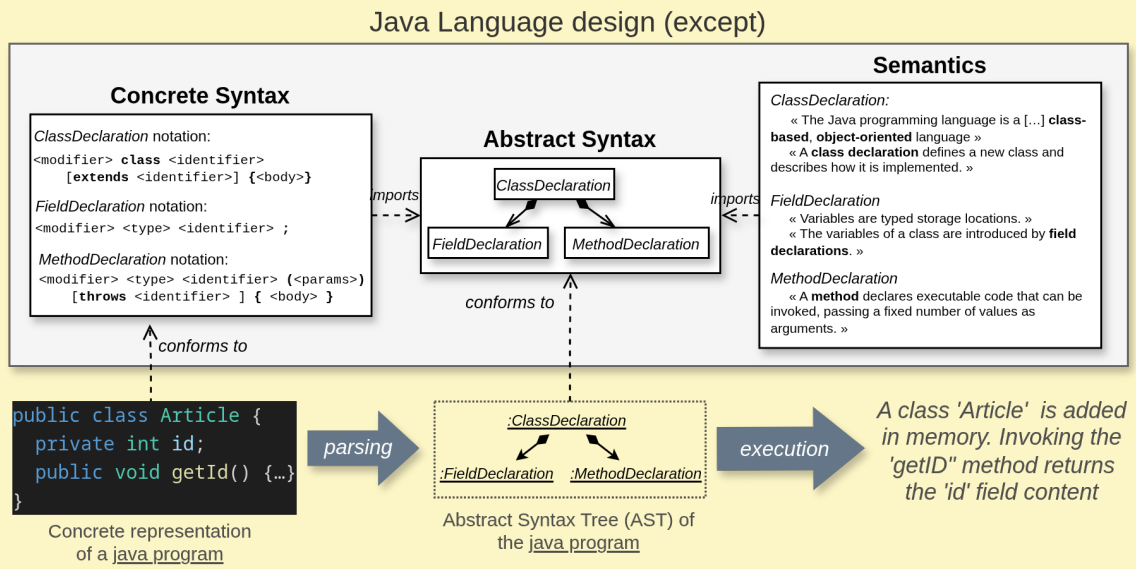


Figure 4. General representation of a software language and its use

- At the center, the **abstract syntax** of a software language defines the *concepts* of the language and the relations between said concepts. These concepts are the *abstractions* provided by the software language. A common way to design an abstract syntax is to rely on *object-oriented modeling*, for example using a simplified UML class diagram (also called a *metamodel* in a language engineering context) without operations.
    - Below the abstract syntax, an example of *abstract syntax tree* (AST) is depicted. It is composed of *instances* of the concepts of the abstract syntax. An AST is the "pure" representation of a program or model which can be programmatically processed by the semantics.
- On the left, the **concrete syntax** of a software language defines the *notation* of concepts defined in the abstract syntax. Notations can be either textual (as most programming languages) or graphical (as most modeling languages). Notation is commonly expressed as *rules* telling with words or shapes correspond to each concept.
    - Below the concrete syntax, an example of *concrete representation* of a program/model is depicted. Here since the concrete syntax is graphical, it is composed of shapes and arrows allowed by the graphical syntax. Using the concrete syntax rules, a concrete representation can be *parsed* to obtain the corresponding AST.
- On the right, the **semantics** of the software language defines the *meaning* of each concept defined in the abstract syntax. The meaning of a concept can be expressed in many ways, informally (with natural language and examples) or formally (with mathematical notations), and should somehow explain what happens when a concept is encountered during an *execution* of the AST.
    - Below the semantics, the result of the execution of AST is depicted. This result is generally a sequence of *execution steps* with a computational effect.

Example 1. Java language design (excerpt)

The official Java Language specification [https://docs.oracle.com/javase/specs/jls/se23/html/index.html] architectures the Java language in the following way:

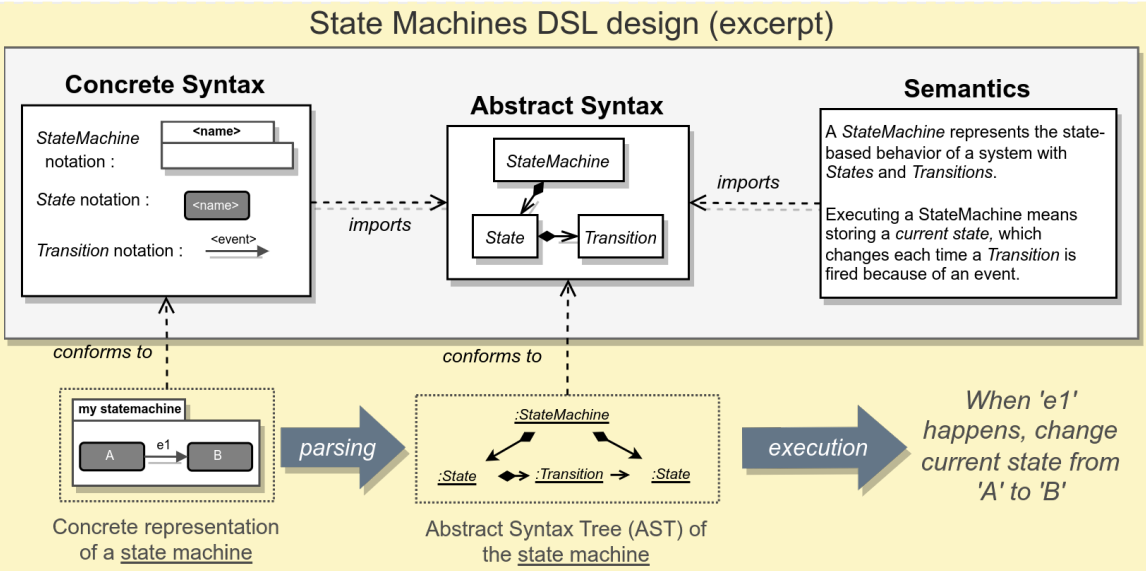


- In the center, the **abstract syntax** is partially shown with three concepts: *ClassDeclaration* represents the declaration of a class, *FieldDeclaration* the declaration of a typed field (also called attribute) within a class, and *MethodDeclaration* the declaration of a method (along with type and parameters) within a class.
- On the left, the **concrete syntax** must describe how each concept should be represented textually. A *ClassDeclaration* is written using the `class` keyword, preceded with a *modifier* (`public`, `private`, etc), and followed with a name and, optionally, a superclass with the `extends` keyword.
- On the right, the **semantics** must explain what each of the concepts mean. Here we show direct quotes from the official Java specification.

Example 2. Designing a state machines DSL

We would like a DSL to represent a specific sort of **state machines**.

We architecture our DSL in the following way (with only excerpts of each of the three constituents):



- At the center, a small excerpt of the **abstract syntax** defines that the DSL includes concepts such as *StateMachine*, *State* and *Transition*.
- At the left, a small excerpt of the **concrete syntax** defines that a *StateMachine* is represented by a drawing area, in which *States* are represented as rounded boxes with a name, and *Transitions* as solid arrows with an event name as label.
- At the right, a small excerpt of **semantics** defines that executing a state machine means changing a *current State* from one state to another based on which *Transitions* are triggered by events.



Throughout the course, we will design and implement a more detailed version of this DSL, and with an alternate *textual* concrete syntax instead of a graphical concrete syntax.

2.2. Implementation of a software Language

Once a software language has been designed with the three ingredients we have seen in the previous section (abstract syntax, concrete syntax, semantics), the next step is to **implement the software language**... as software! We will distinguish:

- the implementation of the *core components* of a software language, which are the minimal parts required to use the language,
- the implementation of *additional tools* and of *IDE integrations* for a software language, to make the language convenient and productive to use for language users.

2.2.1. Core components of a software language implementation

Each of the three parts of a language design must be implemented as a language component, as depicted in the following figure:

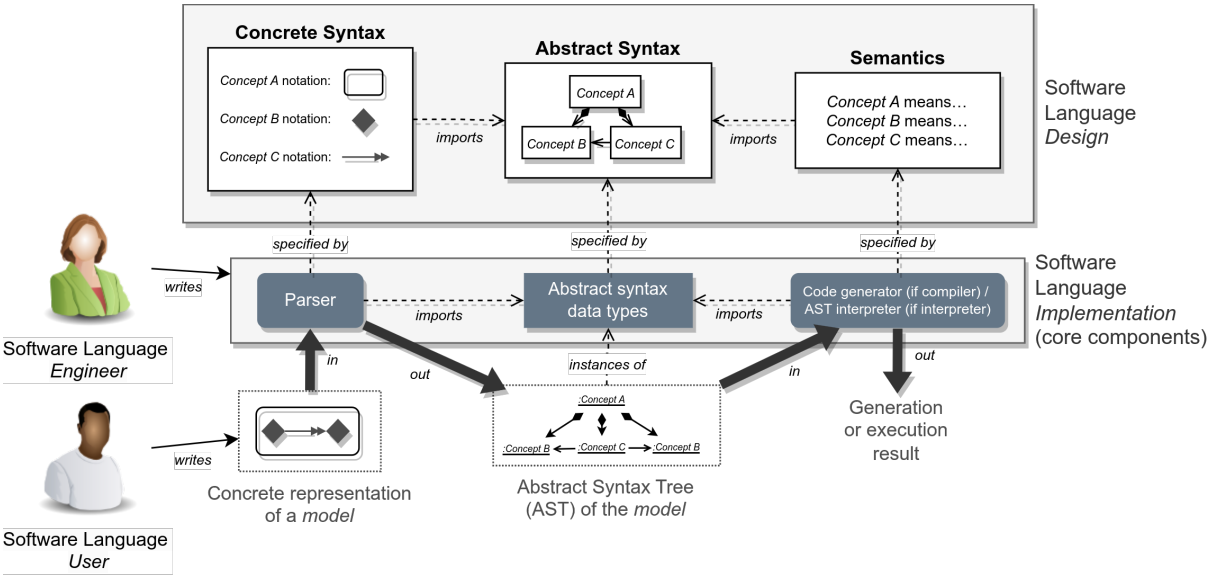


Figure 5. Core components of a software language implementation

- The *concrete syntax* must be implemented in the form of a **parser**, which must be able to take as input a concrete representation of a program of the language, and produce at output an AST of the same program. This requires being able to identify (textual or graphical) *patterns* in the concrete representation.



While the word parser is mostly used for *textual* languages, here we also use this word for *graphical* languages. A parser for a graphical language is of course very different from a textual language parser because it must somehow recognize *shapes*, *arrows* and *drawings* instead of traditional text.

- The *abstract syntax* must be implemented in the form of a a set of **data types**, typically *classes* in an object-oriented context. The parser must be able to *instantiate* these data types in order to create the nodes of an AST of a program.
- The *semantics* can typically be implemented in two different ways:
  - A **code generator** takes an AST as input and produces executable code as output. This output code is written using a separate *target language* that was already implemented. A target language can be either a bytecode language (such as Java bytecode [https://en.wikipedia.org/wiki/Java_bytecode]), or an assembly language (such as x86 assembly [https://en.wikipedia.org/wiki/X86_assembly_language]), or even another programming language (such as Java, C, Rust).
  - An **AST interpreter** takes an AST as input and directly performs a computation based on the AST contents. This computation is typically a sequence of *execution steps* that changes the current state of the program.

A language implementation whose semantics is implemented with a code generator is commonly called a **compiler**, while a language implementation whose semantics is implemented with an AST interpreter is commonly called an **interpreter**.



- **GPLs** are nowadays always implemented as *compilers* that rely either on machine languages (such as x86 or ARM assembly) or *bytecode languages* (such as *Java Bytecode*) as targets for code generation – even Python, contrary to popular knowledge!
- **Bytecode languages** are always implemented as special kinds of interpreters called *process virtual machines* (for exemple the *Java Virtual Machine* (JVM) is

the interpreter for *Java Bytecode*).

- **DSLs**, on the other hand, can sometimes be implemented as *interpreters* when performance is not a key issue, or as *compilers* when targeting a specific execution environment and/or when performance is a concern.



Just-in-time compilation [https://en.wikipedia.org/wiki/Just-in-time_compilation] is a hybrid form of semantics implementation that combines both interpretation and compilation.

2.2.2. Additional tools and IDE integration

Once the core components of a software language have been implemented (which means a complete *interpreter* or *compiler*) we are now technically able to use the language by writing and executing programs. However, common tools are still missing to make it properly usable by the language user, such as:

- An **editor** for the language, both to help *reading a program* (with syntax highlighting, an outline view, navigable symbols, etc) and *writing a program* (with code completion, smart renaming of elements, refactoring actions, etc.). We call *editing services* the set of services offered by an editor for a given software language.
- A **validator** for the language (also called *linter* or *static analyzer*) for the language, able to analyze an input program and to detect structural problems as early as possible. A validator is typically implemented as a set of *validation rules*.
- A **debugger** for the language, allowing the language user to execute a program step by step and to visualize the changes in the program state.

Moreover, a proper *development environment* is important to be able to use a software language efficiently, including the aforementioned tools. This can be achieved through an **integration into existing Integrated Development Environments (IDEs)** such as Eclipse, IntelliJ or Visual Studio Code.

With all these new concerns in mind, the scope of a software language implementation can be extended in the following way:

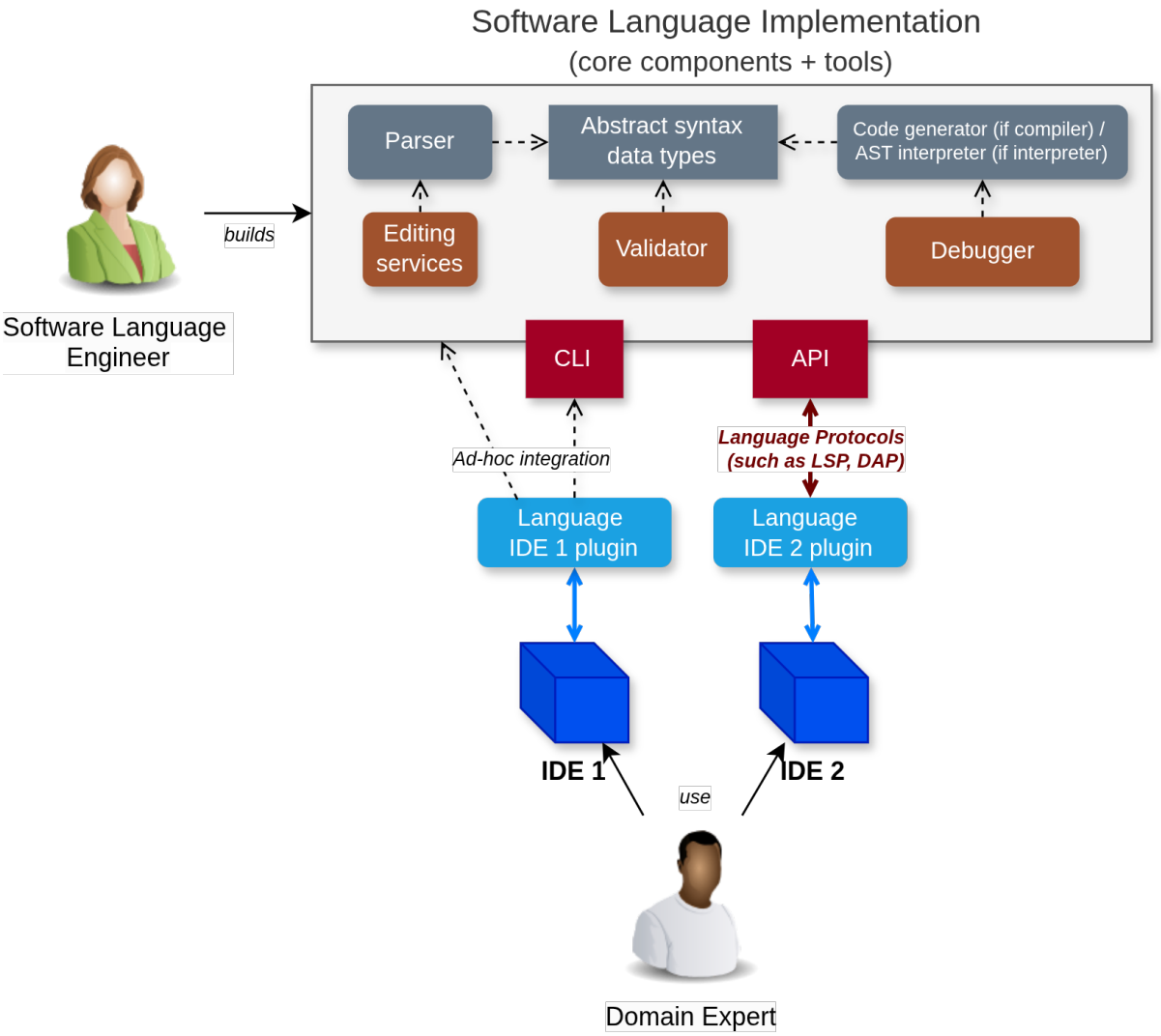


Figure 6. Additional tools and IDE integration for a software language implementation

- We can observe that most tools are dependent on the *abstract syntax data types* of the language implementation. In other words, most tools can be written without any dependency on the concrete syntax of the language.
- At the bottom, **two different ways to integrate the language with an IDE** are depicted:
  - On the left, we see a plugin developed for "IDE 1" which directly uses the language implementation, whether by directly using the language implementation code, or by using the command-line interface (CLI) of the language components and tools.
  - On the right, we see plugin developed for "IDE 2" which relies on *languages protocols* to interact with the language implementation. A *language protocol* is a novel way to communicate with a language implementation in a client-server fashion, therefore decoupling as much as possible a language implementation from an IDE. This approach requires that the language implementation provides a **language API**, typically as a JSON-RPC web server. Two language protocols are commonly in use today: the Language Server Protocol (LSP) [https://microsoft.github.io/language-server-protocol/] for *editing services*, and the Debug Adapter Protocol (DAP) [https://microsoft.github.io/debug-adapter-protocol/] for *debugging services*.

2.3. Engineering a DSL

We now know what SLE is about, and what are the ingredients of a software language. Let's dive into a

more concrete question: **how to engineer a new DSL?**

Since a DSL is a software system, we can proceed similarly:

1. Design the DSL, which includes designing each component of the DSL,
2. Implement the DSL, which means implementing each component of the DSL.

2.3.1. How to design a DSL?

Since a software language is made of software, a software language can be seen as a specific kind of **product** that must be delivered to a **client** (the language users, aka the domain experts). Hence, a great part of designing a DSL is to properly understand why such product is need, what are the needs of the client, and in which context the product will be used.

A particularity of software languages is that their purpose is always to provide the *means to express something*, such as a solution to a given problem, or some data relevant to a system, or the configuration of a system. Hence designing a DSL means focusing greatly on **expressiveness**, through the following question:

- ***What* do we want to express using the DSL, and why?** This question aims to define the exact scope and domain covered by the DSL. It requires good understanding of the context and of the target users, and it results in a design of the *abstract syntax* of the DSL.
- ***How* do we want to express something?** This question aims to define how will target users manipulate the concepts of the language: words, separators, shapes, colors, etc. This also requires a good understanding of the target users, and results in the design of the *concrete syntax* of the DSL.



The concrete syntax can be considered the "user interface" of the DSL. Hence it is a central matter regarding the resulting user experience.

- **What is the *meaning/purpose* of a program written with the DSL?** This question aims to define what should a program become once expressed. Is it executable, and should therefore be either interpreted or compiled? Or does it only serve purpose as documentation, or data definition? This results in a design of the *semantics* of the DSL.

Each following chapter will dive into a specific component of a DSL (abstract syntax, concrete syntax, semantics) and will present an approach to design said component.

2.3.2. How to implement a DSL?

There is a vast ecosystem of tools available to help with the implementation of a DSL.

Metalanguages are specialized software languages directly made for the engineering of other software languages. There are metalanguages to implement an abstract syntax (such as *Ecore* or the *MPS Structure Language*), metalanguages to implement a graphical concrete syntax (such as *Sirius View Model*), metalanguages to implement a textual concrete syntax (such as *ANTLR*, *Xtext*, *Langium*), and many more. Metalanguages provide abstractions directly dedicated to SLE, and can therefore facilitate the design and implementation of many aspects of a DSL.

Languages workbenches are complete environments or frameworks that assist with the *end-to-end* creation and packaging of DSLs. A language workbench usually provides a set of metalanguages, a set of software librairies, and tools such as generators and debuggers. A common (but not systematic) pattern is to rely on metalanguages to implement the abstract and concrete syntaxes, and to rely on

general-purpose programming languages (GPLs) to implement the semantics (eg. a code generator) and to implement the tooling for the DSL (eg. CLI, validator).

There are many industry-grade language workbenches available:

- **MetaEdit+** [https://www.metacase.com/products.html], a proprietary language workbench made by MetaCase [https://www.metacase.com/]. It allows the creation of *graphical* DSLs using dedicated view, forms, and metalanguages. It provides an advanced debugger to help with debugging a semantics implemented as a code generator.

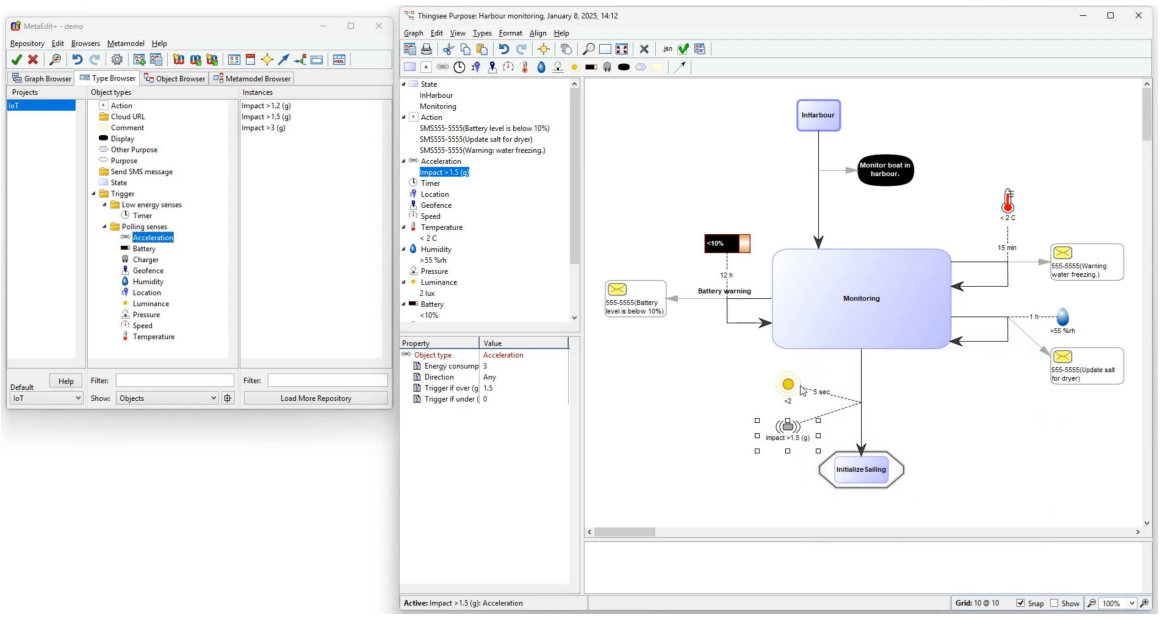


Figure 7. MetaEdit+ language workbench (left) and resulting DSL (right)

- **Xtext** [https://eclipse.dev/Xtext/], developped by Itemis [https://itemis.com] and TypeFox [https://www.typefox.io/], and hosted by the Eclipse Foundation. It allows the creation of *textual* DSLs using the Ecore metalanguage for the abstract syntax, and Xtext Grammar Language for the concrete syntax. Many parts of an Xtext DSL are developped using pure Java, such as the validator, the code generator or the editor. The Eclipse IDE is recommended for implementing an Xtext DSL, since an Xtext DSL is packaged as an Eclipse plugin. An Xtext DSL can then be published as an Eclipse IDE extension, or as an LSP server that can be integrated in different IDEs.

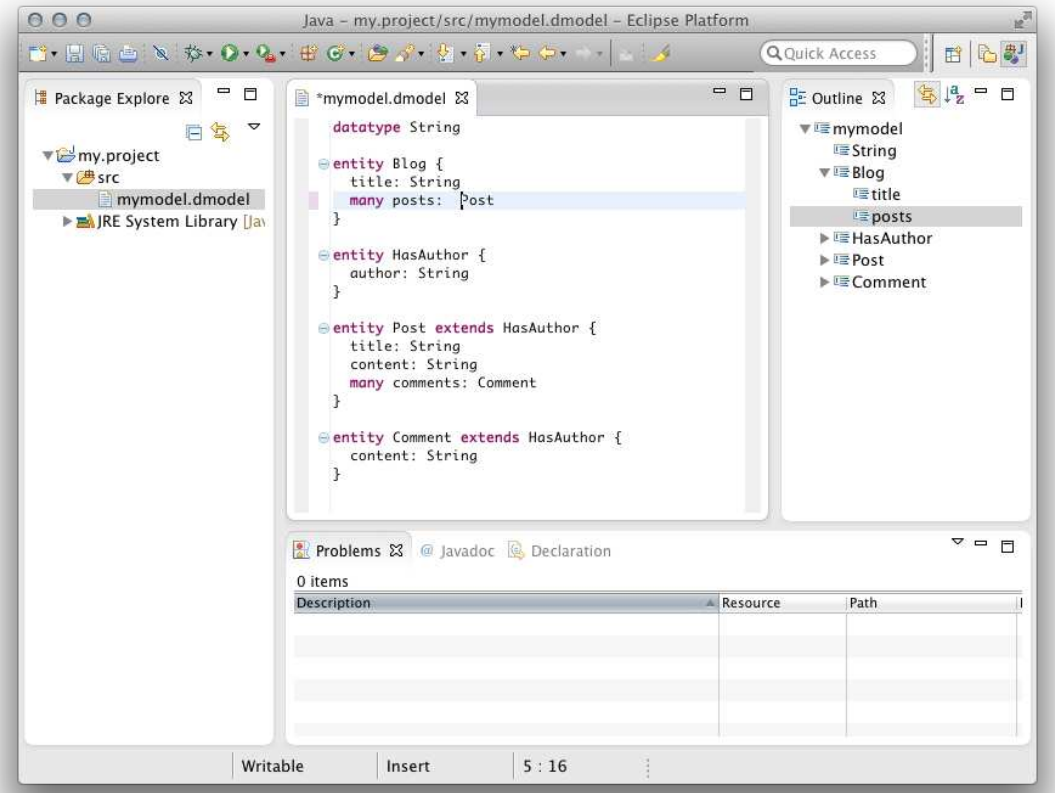


Figure 8. An Xtext DSL running in the Eclipse IDE

- **Sirius Web** [https://eclipse.dev/sirius/sirius-web.html], developped by Obeo [https://www.obeosoft.com/] and hosted by the Eclipse Foundation [https://www.eclipse.org/org/]. It allows the creation of web-based *graphical* DSLs using a fully web-based language workbench. It provides a Domain DSL to implement an abstract syntax, and a View DSL for the concrete syntax.

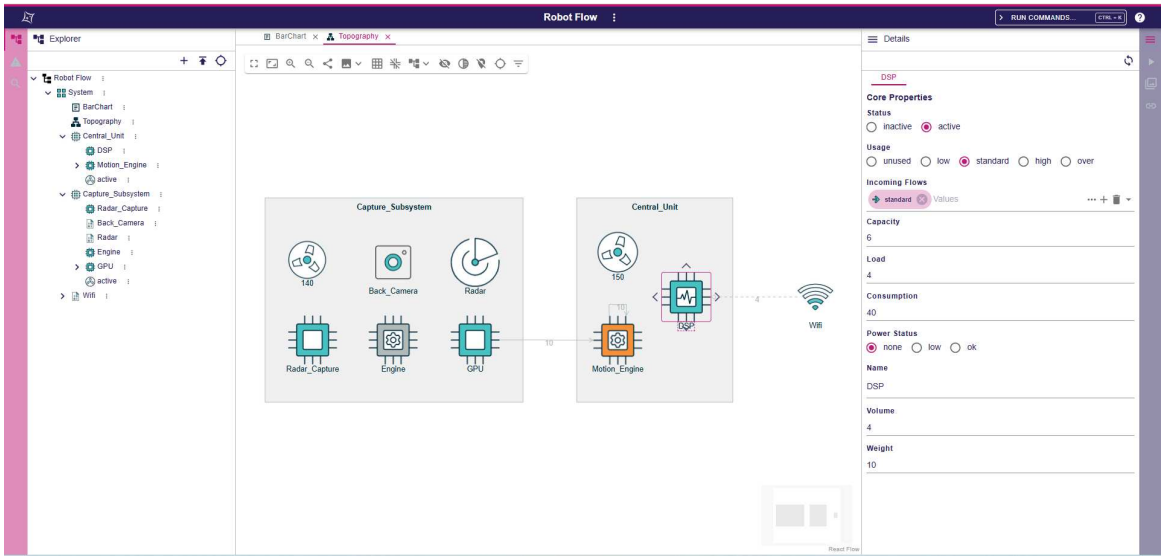


Figure 9. A Sirius Web DSL running in the browser

- **Spoofax** [https://spoofax.dev/], an open-source language workbench by researchers of Delft University of Technology [https://www.tudelft.nl/]. It allows the creation of *textual* DSLs by fully using a wide range of of sophisticated metalanguages (SDF3, Statix, Stratego), and without using

any GPL.

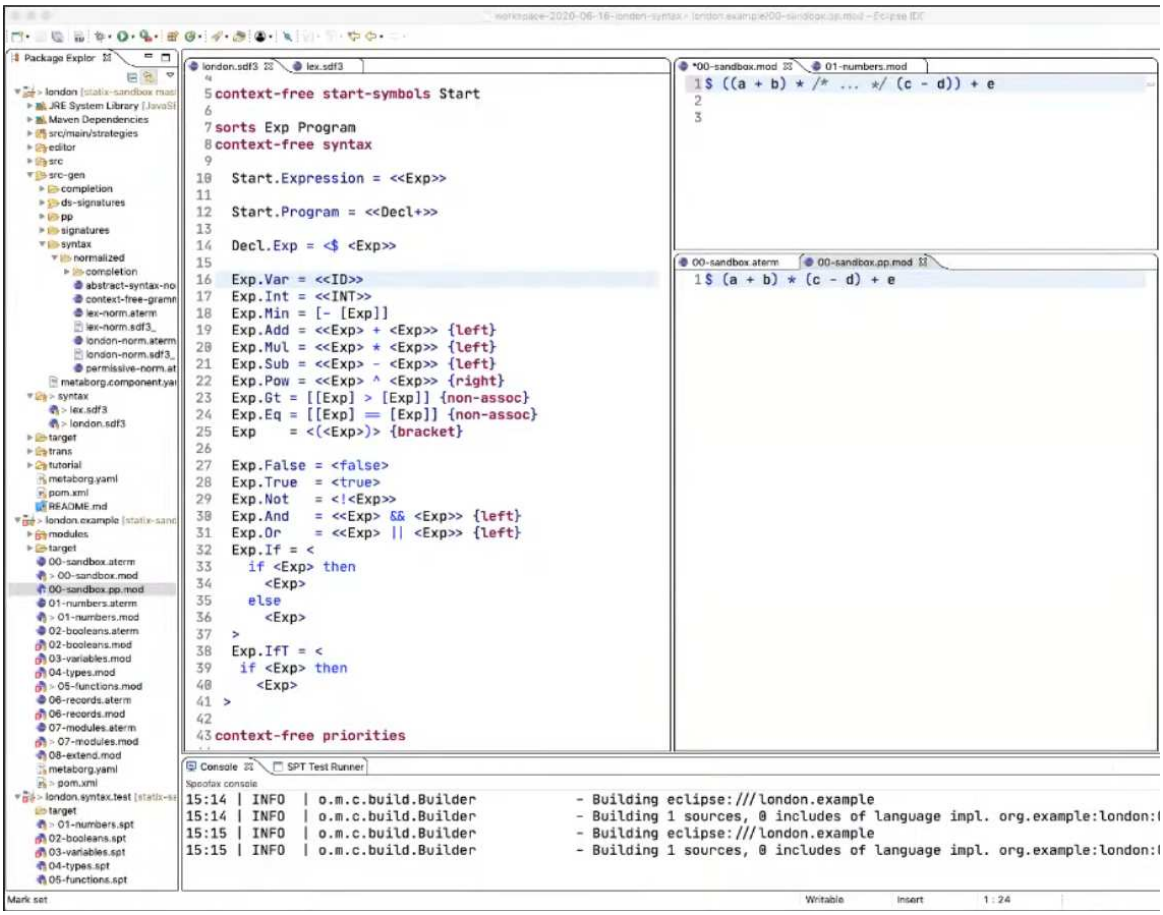


Figure 10. Spoofax language workbench

- **Meta Programming System (MPS)** [http://www.jetbrains.com/mps/], an open-source language workbench by JetBrains [http://www.jetbrains.com/]. It allows the creation of hybrid textual-graphical DSLs using a using form of *projectional* concrete syntax. Other aspects of the DSL are fully implemented using a wide range of of provided metalanguages.

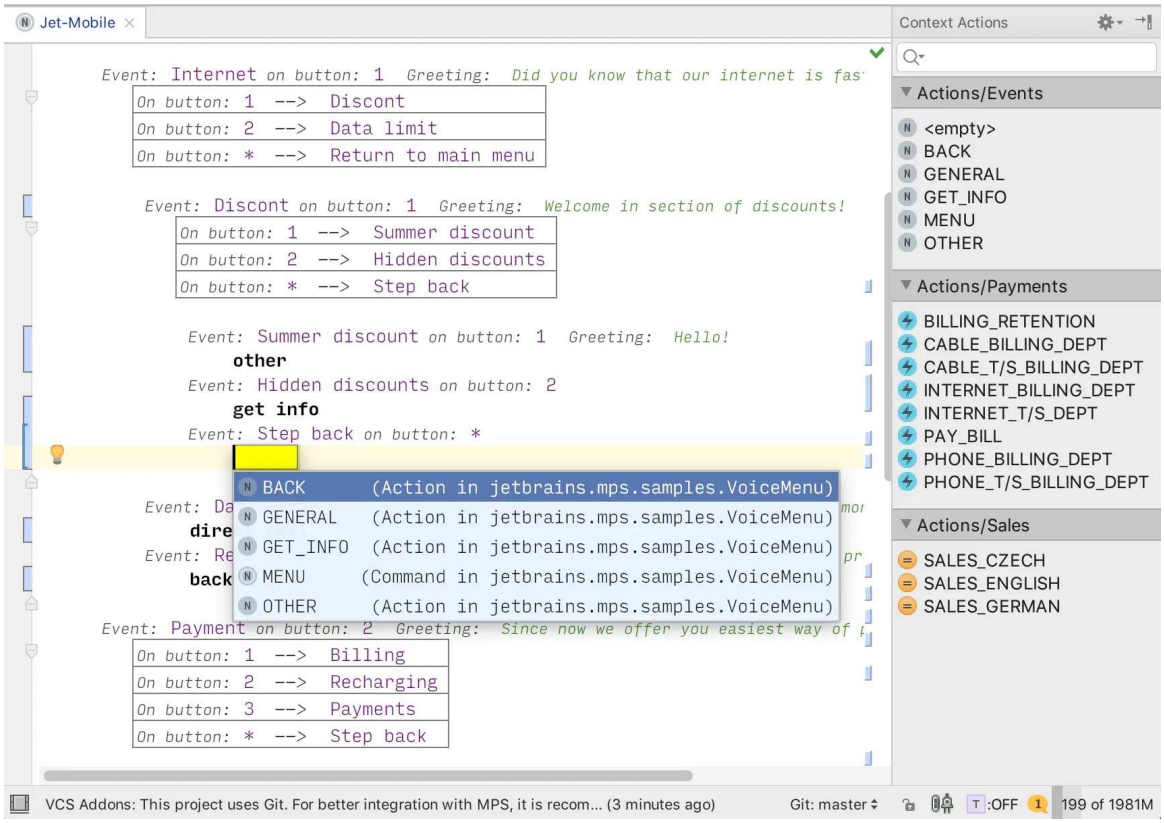


Figure 11. An MPS DSL running in MPS

- **Langium** [https://langium.org/], an open-source language workbench made by TypeFox [https://www.typefox.io/] and hosted by the Eclipse Foundation [https://www.eclipse.org/org/]. It allows the creation of textual DSLs using both a metalanguage called Langium Grammar Language, and Typescript. A Langium DSL can be published as an LSP server embedded into a Visual Studio code extension.

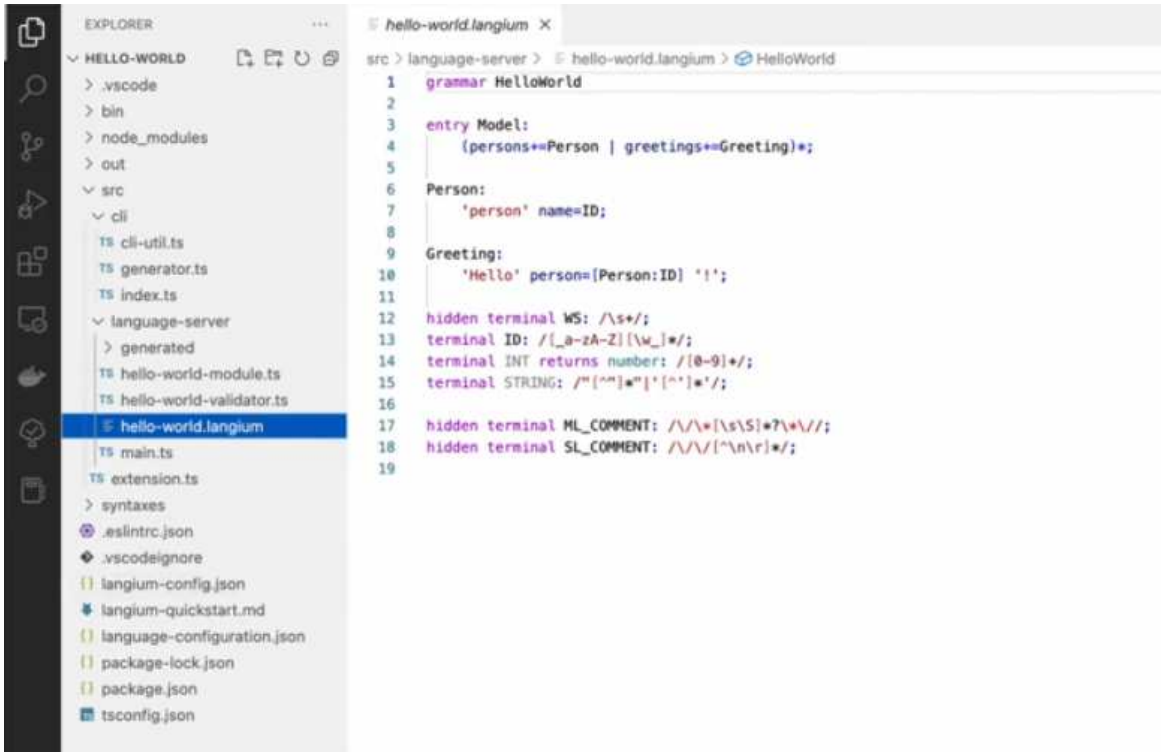


Figure 12. Langium language workbench in Visual Studio Code

Each following chapter will dive into a specific component of a DSL (abstract syntax, concrete syntax, semantics) and will present an approach to implement said component using Langium.

Chapter 3. Ch3. The Langium Language Workbench



Figure 13. Langium logo

Langium [https://langium.org/] is an open-source language engineering framework developed by TypeFox [https://www.typefox.io/language-engineering/] and hosted by the Eclipse Foundation [https://www.eclipse.org/org/]. It can be used to implement a software language in Typescript [https://www.typescriptlang.org/], which can then be deployed in a web environment or as an extension for the Visual Studio Code [https://code.visualstudio.com/] IDE.



Langium is the "spiritual successor" of an older language engineering framework called Xtext [https://eclipse.dev/Xtext/index.html], which can be used to implement a software language in Java and to generate a powerful integration of this language in the Eclipse IDE.



This chapter introduces very general aspects of Langium, but **does not explain how to implement a complete DSL with Langium**. Implement language components with Langium will be progressively presented in the next three chapters.

3.1. Langium overview

The Langium framework is organized around three main components:

- A **meta-language**, called the *Langium Grammar Language*, which can be used to create a *Langium grammar*. A Langium grammar defines both the abstract syntax and the concrete syntax of the software language being developed. The abstract syntax is written as a set of *data types* (with a notation similar to TypeScript), while the concrete syntax is defined in the form of a *grammar rules* (with a notation similar to EBNF).
- A **generator**, which takes a Langium grammar as input and produces a set of Typescript language components as output (detailed below).
- A **Typescript library** used by both the generator and the generated language components.

Using the Langium framework to *create a new software language* can be depicted as follows:

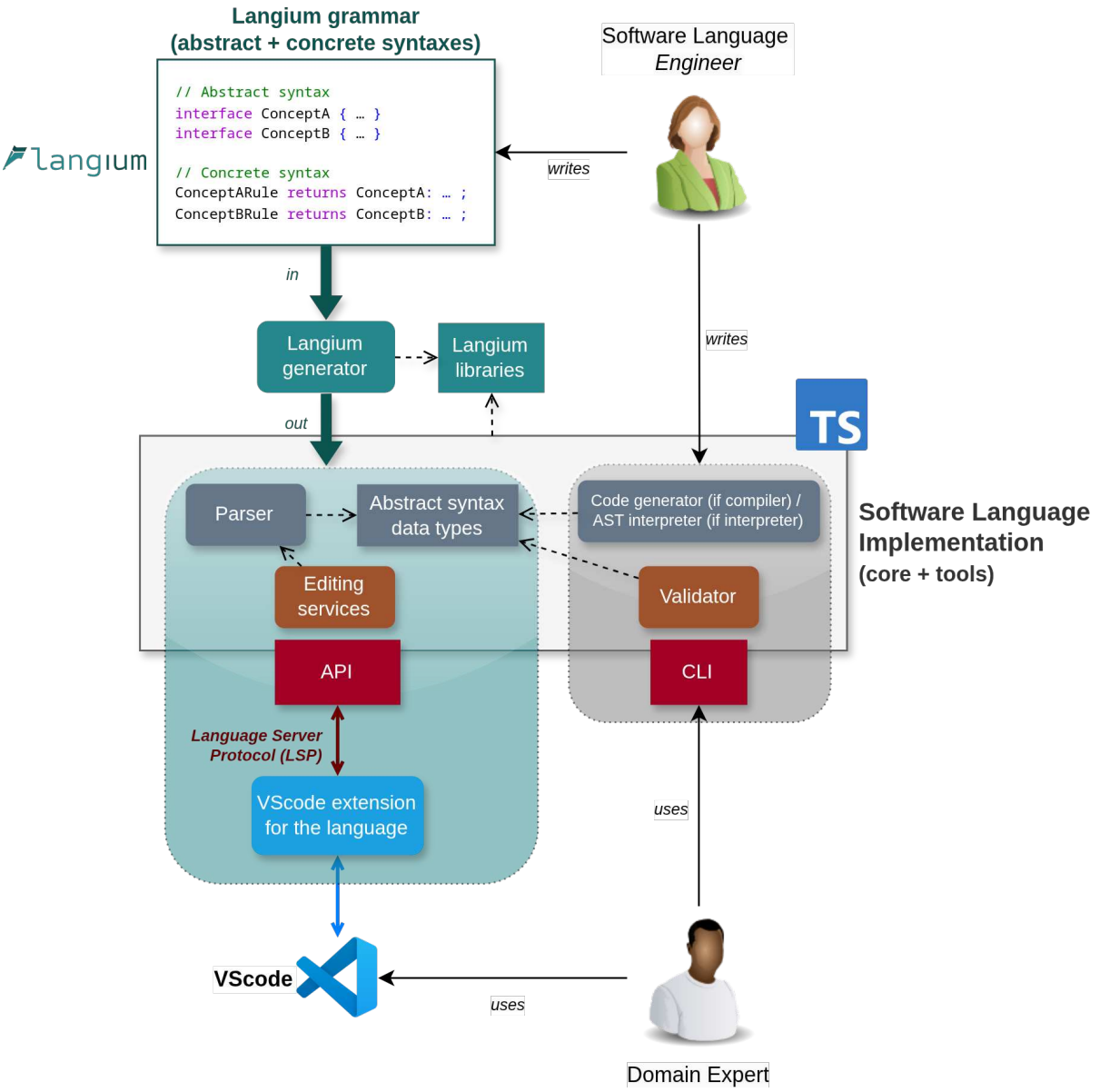


Figure 14. Using the Langium framework to implement a software language

1. At the top, the *software language engineer* uses the **Langium Grammar Language** to define both the abstract and concrete syntaxes of the developed language.
2. The resulting Langium grammar is given to the **Langium generator**, which produces as output a wide array of artifacts:
    - A set of *Abstract syntax data types*, in the form of Typescript types, for all the concepts of the abstract syntax,
    - A *parser*, able to read a text file and to produce an AST as defined by the grammar rules,
    - A set of *editing services* which are essentially all the logic of a code editor for the language being developed (syntax highlighting, error highlighting, autocompletion, outline view, etc.).
    - A *language* API, which can be used as a single HTTP entrypoint to access most languages components. The services of this API follow the standardized Language Server Protocol (LSP) [https://en.wikipedia.org/wiki/Language_Server_Protocol].
    - A *Visual Studio Code extension* which can be used to easily deploy the software language editor in any Visual Studio Code IDE. This extension is mostly an adapter that interacts with the *language* API to provide most services.

3. Finally, some **parts of the language implementation must be developed manually** by the language engineer. This includes:
    - the implementation of the *semantics*, either in the form of a *code generator* (if the DSL is implemented as a compiler) or in the form of an AST *interpreter* (if the DSL is implemented as an interpreter),
    - the implementation of a *validator* (in the form of *validation rules* executed on the AST),
    - the implemention of a *Command Line Interface* (CLI) to call specific language services (such as the code generator).
4. Once the language has been fully implemented, the *domain expert* (at the bottom) can *create and edit programs* using the VSCode extension, and can call specific services (eg. the code generator) using the CLI.

3.2. Installing a Langium development environment



This section assumes you are running a Linux-based environment.

3.2.1. Prerequisites

Developping a Langium DSL requires a Javascript/Typescript development environment:

- Make sure you have `npm` and `node` (*at least* version 20, if possible higher) available in your system:
    - Run `npm --version` to check `npm` is installed,
    - Run `node --version` to check `node` is installed with a compatible version.
- Make sure you have configured npm to allow installation of packages globally [https://docs.npmjs.com/resolving-eacces-permissions-errors-when-installing-packages-globally]:
    - Run this command: `npm config set prefix ~/.local`,
    - Modify your `.bashrc` file and add: `export PATH=~/.local/bin:$PATH`,
    - Restart your terminal.

Installing a custom `node` version (without being root)

If your system has `node` with a version before v20, then you need to install a more recent `node` version to be compatible with Langium.

Here is how to install `node` version 24 on any Linux system, without being root, using the nvm tool [https://github.com/nvm-sh/nvm]:

1. If that's not a problem for you, for a fresh start, delete your complete current npm installation and configuration:

```
rm -rf ~/.npm ~/.npm-global ~/.npmrc
```

2. Install `nvm` :

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

3. Restart your terminal.
4. Install `node` version 24 using `nvm`:

```
nvm install 24
```

5. Check that you have the version you want:

```
node -v
```

3.2.2. Langium Yeoman project generator

Langium provides a Yeoman [https://yeoman.io/] generator which is mandatory to quickstart a Langium project.

Yeoman can be installed on your system with npm [https://www.npmjs.com/] with the following command:

```
npm install -g yo generator-langium
```



The `-g` option requires that you first configure your system to install npm packages in your home directory, as written in this page of the npm documentation [https://docs.npmjs.com/resolving-eacces-permissions-errors-when-installing-packages-globally].

3.2.3. Langium VScode extension

Langium provides an official Visual Studio Code extension [https://marketplace.visualstudio.com/items?itemName=langium.langium-vscode] that includes a complete textual editor for the *Langium Grammar Language* (`.langium` files).

You can install this extension directly using the *Extensions* pane of Visual Studio Code.

3.3. Preparing a Langium project

To generate a new folder that will contain your new Langium project, run this Yeoman command:

```
yo langium
```

This will ask you different questions about your Langium project, such as the name of the software language being developed, the file extension for this language, and which development environments should be supported (CLI, Visual Studio Code, web, etc.). For more information, refer to this page of the Langium documentation [https://langium.org/docs/learn/workflow/scaffold/].



If you are unsure, you can always answer *yes* to all questions.

However note that the question "*Generate example project?*" is different from

> others: it will create an example DSL that will be a pain to replace with your own DSL! In most cases you will prefer to answer No to this question.

After generation of your base Langium project, you obtain a **Langium project structure**, which can be depicted like this:

This structure is centered on the folder `packages`, which contains in the following subfolders (each containing a different npm package):

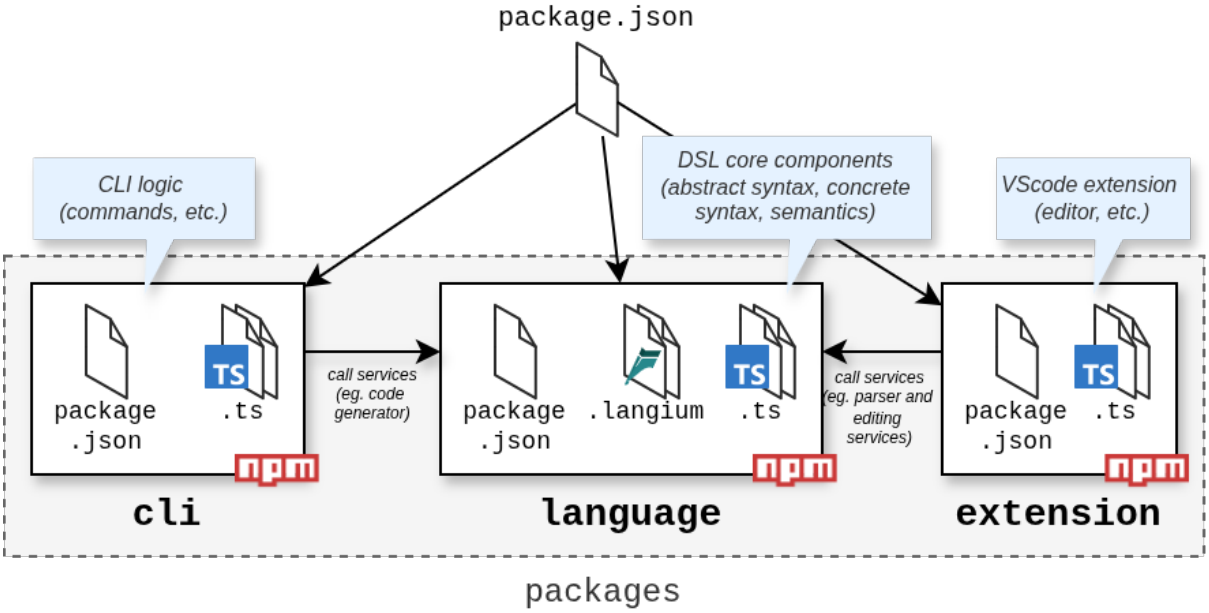


Figure 15. Langium DSL project, organized as three inter-dependent npm packages

- `language` : Contains the main components of the software language, in particular:
  - `langium-config.json` : Contains metadata about your language, including the language name, the file extension, or the path to the main `.langium` file. If multiple `.langium` files are used, they must be imported from the `.langium` file referenced in `langium-config.json`
  - `src`: Contains the `.langium` files defining the *Langium grammar* of the language, which itself defines the abstract syntax and the concrete syntax of the language (*will be presented in the next section*), with one subfolder:
    - `generated`: Contains files generated by the Langium generator from the `.langium` files, including **the `ast.ts` file that contains the abstract syntax data types**.
  - `test`: Contains test cases for the language.
- `cli`: contains a base implementation of a Command Line Interface (CLI) for the language, which by default only includes a placeholder for a **code generator** where the *semantics* of the language can be implemented.
- `extension`: Contains the code of the Visual Studio Code extension of the language, which is essentially a code editor for the DSL.



> Some of the subfolders may be absent, depending on the answers given to the Yeoman generator.

It is important to note that the npm components `cli` and `extension` are both strongly dependent on the `language` component, which is logical given that all the "business logic" of the DSL (syntax, semantics, editing services, etc.) is implemented in the `language` component.

3.4. Working on a Langium project

3.4.1. Building a Langium project

The `package.json` file of a Langium project provides the following actions available using `npm`:

- `npm run langium:generate`: Call the Langium Generator, which will compile the `.langium` files into TypeScript, and into a few other artifacts (*must be called after any change to a `.langium` file*).
- `npm run build`: Compiles all the Typescript code (*must be called after any code change, and after any call to the Langium Generator*).

Hence, to fully build a Langium project, the following command can be used:

```
npm run langium:generate && npm run build
```

3.4.2. Manual testing of a Langium DSL



Langium also provides a way to perform *automated* testing of a DSL, but this course will not cover this possibility. If you would like to try, you can answer *Yes* when Yeoman asks if you would like your project to include *language tests*.

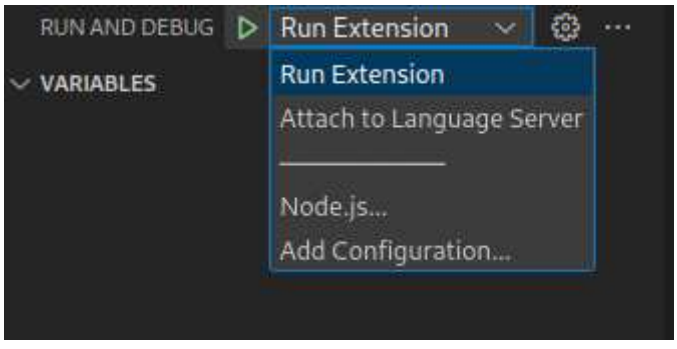
Manual testing of the Visual Studio Code extension

Once a Langium project is built, it is possible to manually test the resulting **Visual Studio Code extension** (and the *editor* for the DSL included within).

Open the "*Run and Debug*" pane:



Observe that two run configurations have been prepared by Yeoman when you initialized your project:



- "*Run Extension*" allows you to test your extension by starting a second Visual Studio Code window in which your language in developpment will be installed as a Visual Studio Code extension.
- "*Attach to Language Server* " allows you to debug your language implementation by attaching a TypeScript debugger to an already running extension of your language (requires doing *Run Extension* first).

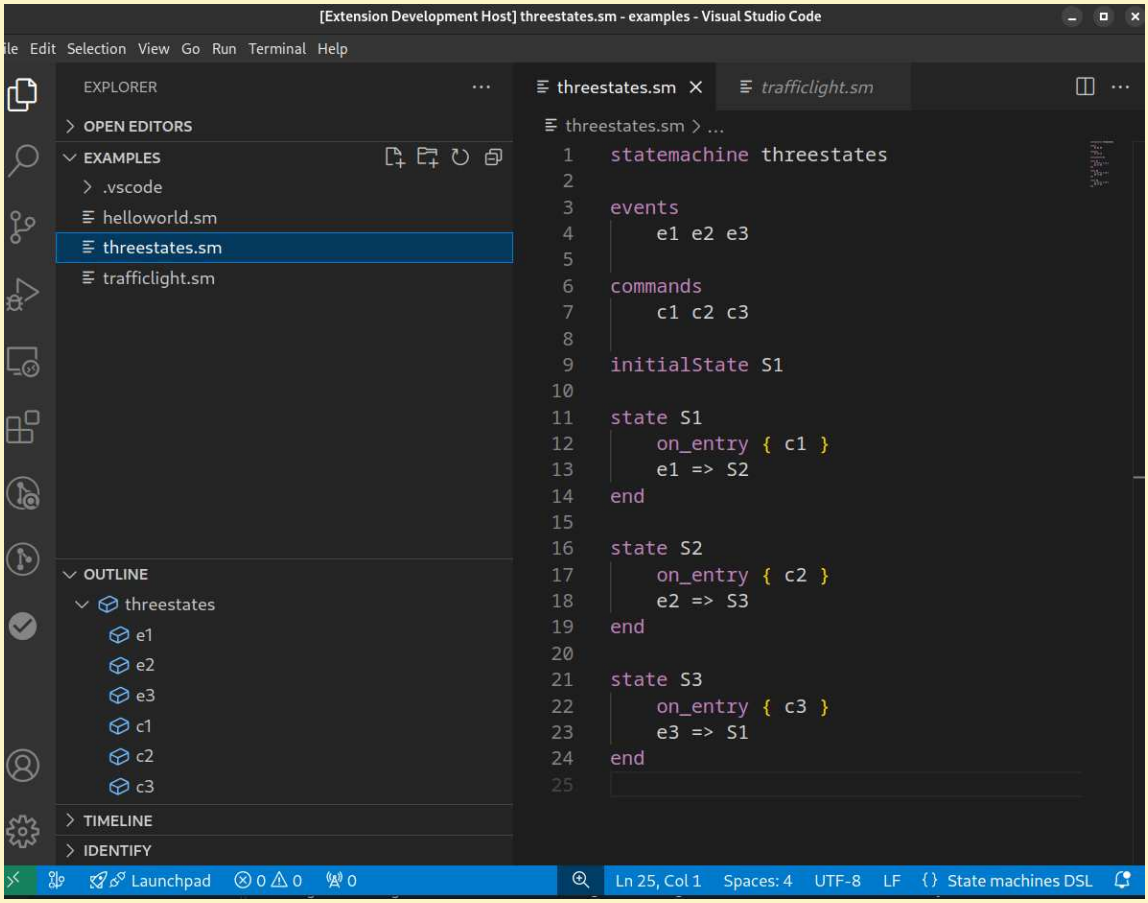
When you use "*Run Extension*", a second "development mode" Visual Studio Code window appears in

an empty workspace. In this empty workspace, create a new file **using the file extension of your Langium language** (important). This will open the customized editor for your language.

Example 3. Manual testing of a State Machines DSL extension

We are implementing a DSL called the *State Machines DSL* using Langium.

We would like to manually test and debug the Visual Studio Code extension, especially the editor of the DSL. We execute the "*Run Extension*" run configuration, which starts a new Visual Studio Code instance where the State Machines DSL editor can be tested:



Manual testing of the CLI

When a Langium project is built, a file named `./packages/cli/bin/cli.js` is produced which allows running the CLI. Hence to manually test the CLI, we can simply run this file (here with the action `generate`):

```
node ./packages/cli/bin/cli.js generate <path to program to compile>
```



You can also make your CLI available globally in your system by running the `npm link` command in your project folder. This should create a command `<language name>-cli` available in your console, which is a symbolic link to the `./packages/cli/bin/cli.js` file of your language.

Example 4. Manual testing of a State Machines DSL CLI

We are implementing a DSL called the *State Machines DSL* using Langium.

We would like to manually test and debug the CLI of the DSL, especially the code generator that can be called using the CLI.

We prepare a State Machines program in a file `examples/helloworld.sm`, and we test the DSL CLI using this command:

```
node ./packages/cli/bin/cli.js generate examples/helloworld.sm
```

3.5. Packaging and distributing a Langium DSL

Once a Langium DSL is ready to use, the last step is to properly package the DSL to install it on end-user environments (for example the environment of the *domain expert*).

3.5.1. Packaging and distributing the Visual Studio Code extension

The Visual Studio Code extension of a Langium DSL can be packaged as **a `.vsix` file** that can be published and copied/downloaded to end-users.

In the DSL project, we first move into the folder where the *extension* code is located:

```
cd packages/extension
```

Then we install the `vsce` dependency, which is required to build and package a Visual Studio Code extension:

```
npm install --save-dev @vscode/vsce
```

Finally we run `vsce` to build and package the extension:

```
npx vsce package
```

This produces a `.vsix` file that can be installed in any Visual Studio Code environment.

Example 5. Packaging of a State Machines DSL Visual Studio Code extension

We implemented a DSL called the *State Machines DSL* using Langium. We would like to package and then distribute the Visual Studio Code extension of this DSL to end users, to allow them to use the editor to create State Machines programs.

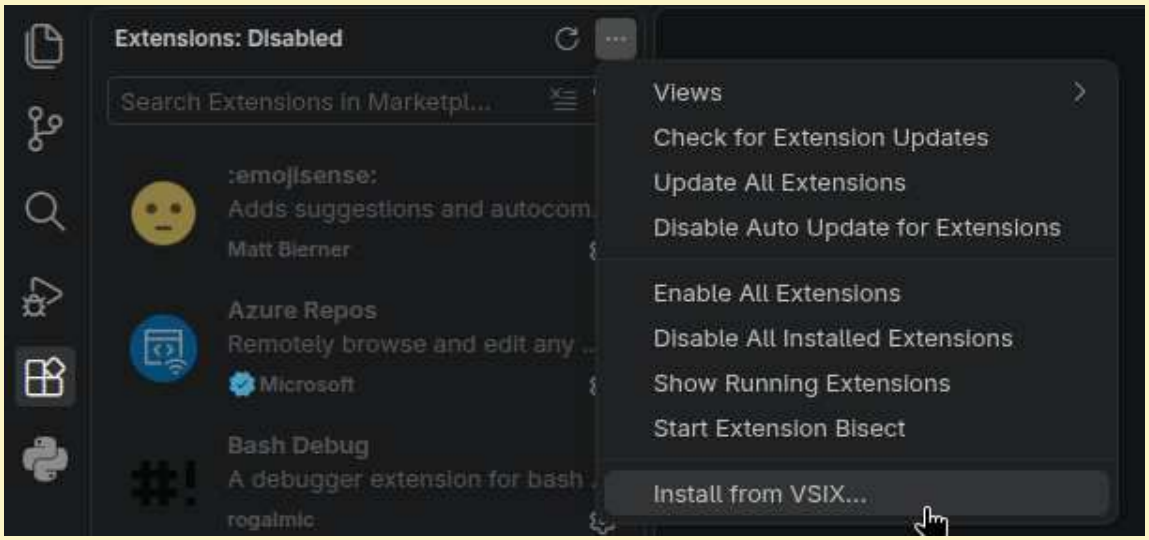
In the project, we run the required command:

```
cd packages/extension
npm install --save-dev @vscode/vsce
```

```
npx vsce package
```

This produces a file `packages/extension/vscode-state-machines-dsl-0.0.1.vsix`.

Later, we take this file and copy it in the environment of an end-user (eg. a domain expert), and we install them in this new environment using this menu of the *Extensions* pane:



We can then see in the *Extensions* pane that the extension is correctly installed:



3.5.2. Packaging and distributing the CLI

The CLI (which usually includes the *semantics*) of a Langium DSL can be packaged and distributed as an **npm package**.

In the DSL project, we run this command:

```
npm pack -w packages/language -w packages/cli
```

which creates two files:

- a `.tgz` for the `language` package, which contains the DSL core components,
- a `.tgz` for the `cli` package, which contains the DSL CLI.

These `.tgz` files can be copied into a user environment, and can then be installed with the following command:

```
npm install -g langium <language tgz> <cli tgz>
```

which installs `langium` libraries, the DSL core components `.tgz`, and the DSL CLI `.tgz`.

This installs a new command in the user environment, which can be directly executed from a terminal. The name of this new command is the one specified in the `bin` part of the `package.json` of the `cli` package (see example below).

Example 6. Packaging of a State Machines DSL CLI

We implemented a DSL called the *State Machines DSL* using Langium. We would like to package and then distribute the CLI of this DSL to end users, to allow them to call the code generator on State Machines DSL programs.

In the root of the *State Machines DSL* project we run:

```
npm pack -w packages/language -w packages/cli
```

This produces the following two files:

- `state-machines-dsl-language-0.0.1.tgz`: an npm package containing the core components of the DSL,
- `state-machines-dsl-cli-0.0.1.tgz`: an npm package containing the CLI of the DSL.

Later, we take these two files and copy them in the environment of an end-user (eg. a domain expert), and we install them in this new environment using this command:

```
npm install -g langium state-machines-dsl-cli-0.0.1.tgz  state-machines-dsl-language-
0.0.1.tgz
```

In this end-user environment, I can now call an executable called `state-machines-dsl-cli` to compile state machine programs, for example:

```
state-machines-dsl-cli generate my_state_machine.sm
```



We can find the name of the resulting executable (`state-machines-dsl-cli`) in the `packages/cli/package.json` of the State Machines DSL at this location:

```
"bin": {
    "state-machines-dsl-cli": "./bin/cli.js"
},
```

Chapter 4. Ch4. Abstract syntax

The **abstract syntax** is the most central part of a software language. It defines the *concepts* of the language and the relationships between these concepts. Moreover, it defines the *valid structure* of an AST of language, along with the *valid types* of the AST nodes.

In this chapter we will first learn how to *design* an abstract syntax using the MOF standard (which is a simplification of UML class diagrams), then we will learn to *implement* an abstract syntax in a Langium DSL.

4.1. Designing an abstract syntax

4.1.1. Object-oriented modeling

A common way to design the abstract syntax of a DSL is to rely on an **object-oriented modeling language** such as MOF [https://en.wikipedia.org/wiki/Meta-Object_Facility], Ecore (from the EMF [https://en.wikipedia.org/wiki/Eclipse_Modeling_Framework] platform), or UML [https://www.omg.org/uml/] class diagrams.

Using an object-oriented modeling language as a metalanguage for designing an abstract syntax has several benefits:

- each *concept* of the DSL can be represented using a *class*,
- each node of an AST can be represented by an instance of once of such class (hence an AST can be represented as an *object diagram*),
- important characteristics of a concept can be represented by *properties* in the corresponding class,
- *relationships* between concepts can be represented using *associations* between classes,
- when multiple concepts share some characteristics, *abstract classes* and *inheritance* can be used,
- to make sure the nodes of an AST are organized into a tree, *composite associations* can be used to enforce parent-child relationships between concepts.

Metamodeling?

An abstract syntax represented by an object-oriented model, is usually called a **metamodel**. The process of creating a metamodel is called *metamodeling*.

A metamodel is a specific kind of software model that defines how to create models (here, the ASTs of the software language we are desining). When all objects of a model are instances of classes of a metamodel, we say that this model *conforms* to this metamodel.

4.1.2. MOF: a convenient subset of UML class diagrams

In this course we will use the **MOF metalanguage** [https://www.omg.org/mof/] to design abstract syntaxes. MOF (*Meta-Object Facility*) is a standardized subset of UML class diagrams tailored specifically for metamodeling.

MOF is a standard that was proposed because we do not need the entire expressivity of UML class diagrams for metamodeling. Hence, MOF removes significant parts of UML class diagrams, such as:

- no interfaces, *since we want to define concrete data structures*,
- no shared aggregations, *since we prefer compositions to enforce a tree structure*,
- no visibilities (public/private/protected), *since we do not need encapsulation.*

In addition, in this course, we will use a metamodel as a pure data structure, hence we will never use *operations*:

- no operations, *since we are only interested in data and structure.*

We can summarize the subset of MOF used in this course with the following figure:

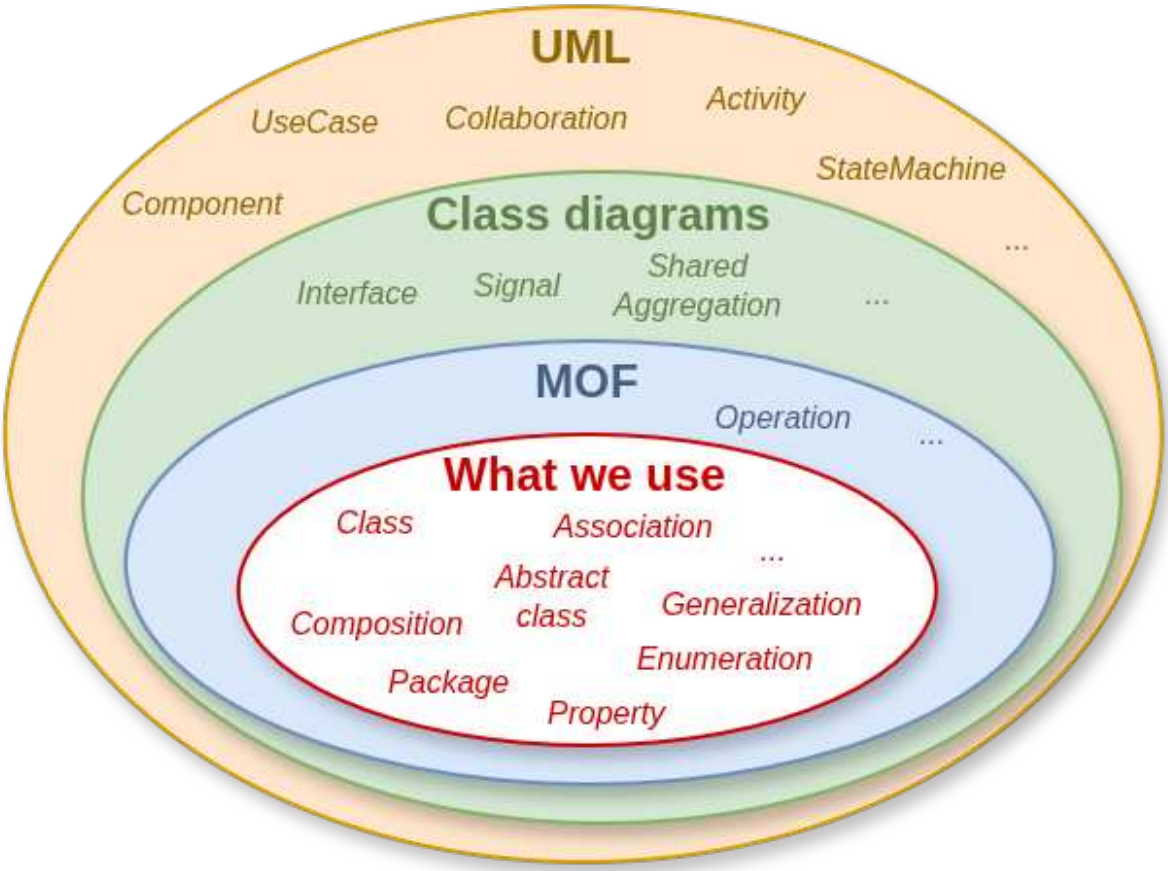
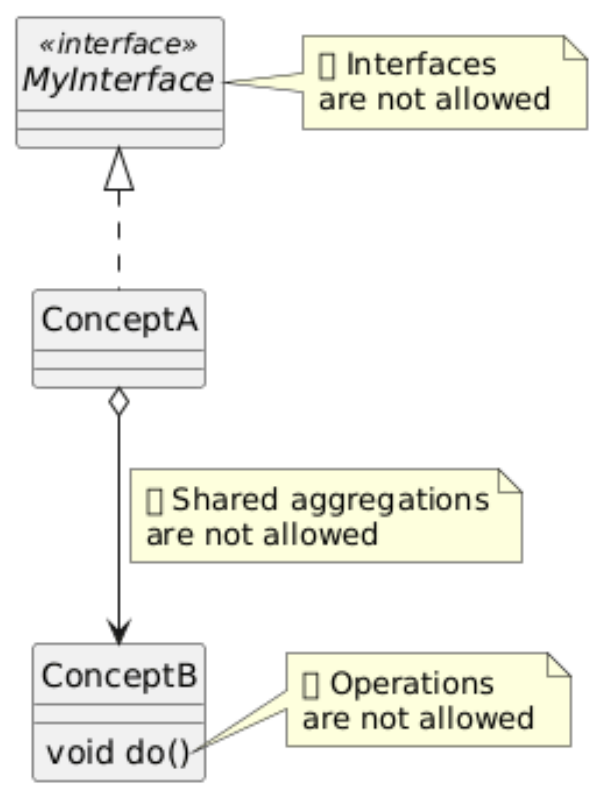
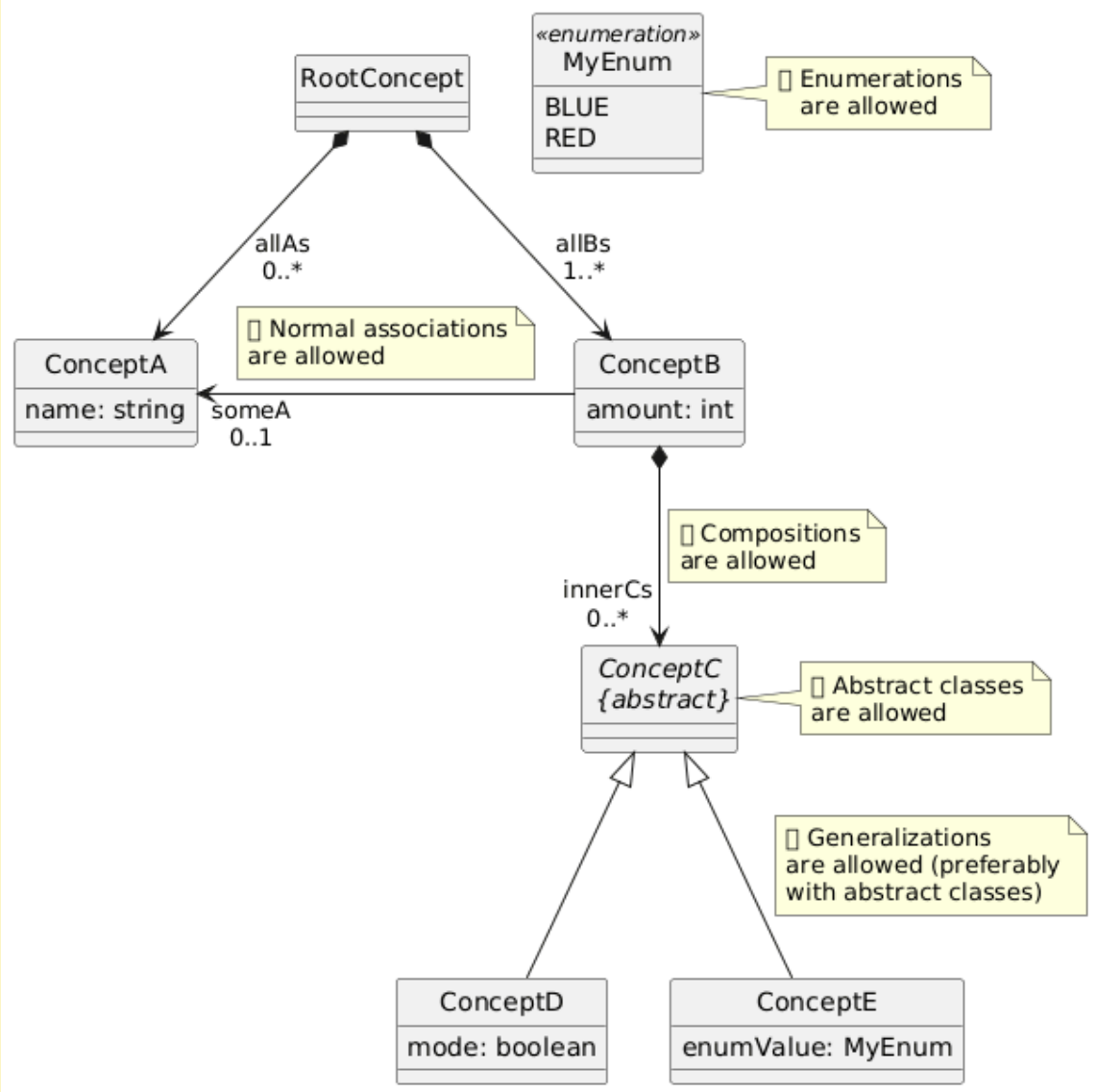


Figure 16. Venn diagram that approximately shows the subset of MOF used in this course

Example 7. Forbidden UML constructs



Example 8. Allowed UML constructs (MOF without operations)



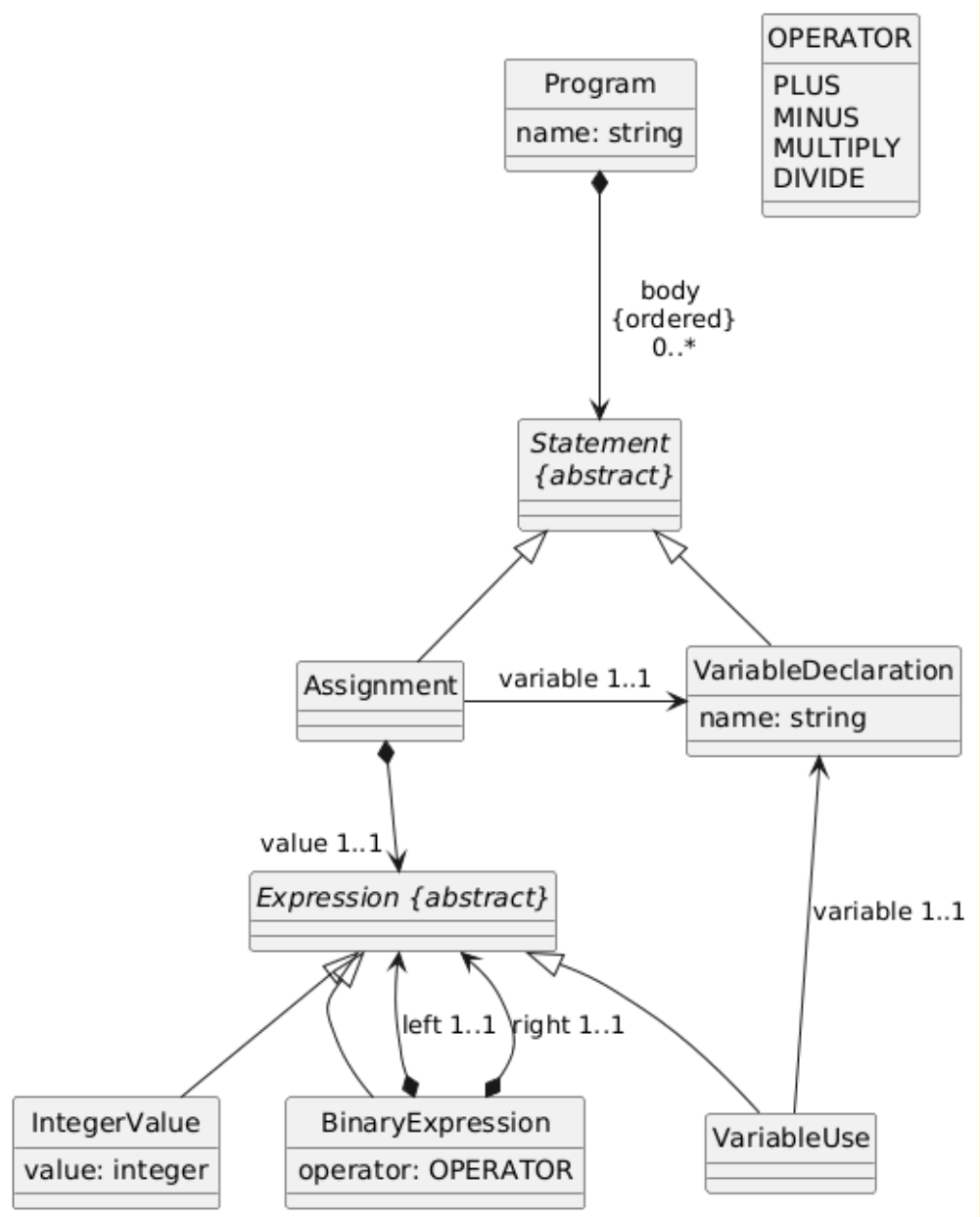
Example 9. MOF abstract syntax of a tiny programming language DSL

We design a tiny programming language DSL, to write simple programs with variables and basic arithmetic expressions. With a mockup concrete syntax, a program should look like this:

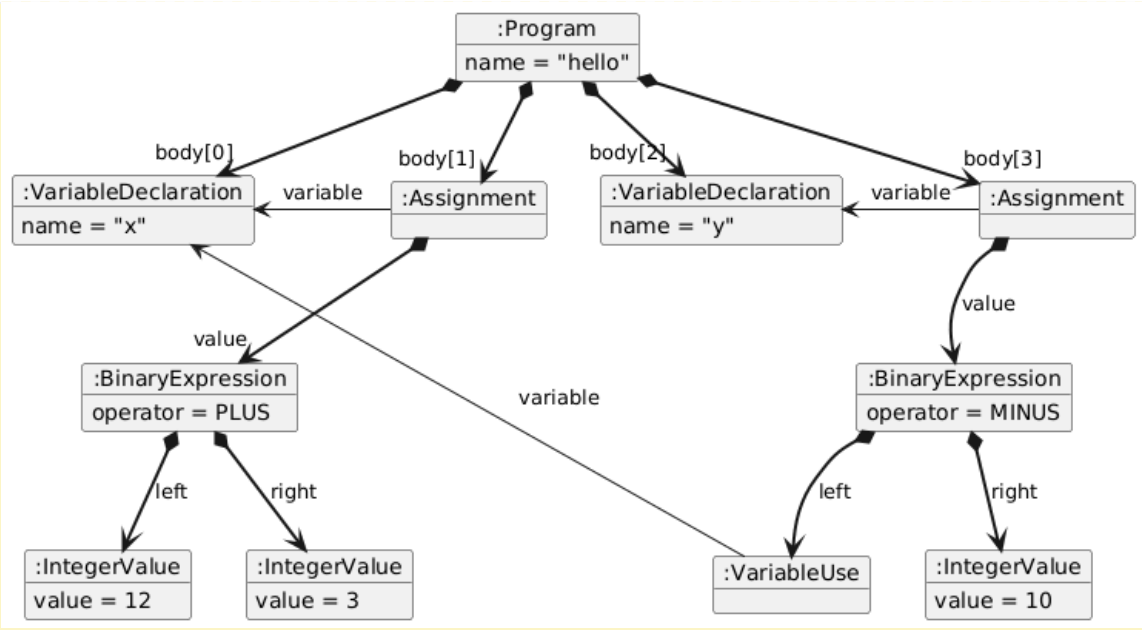
```
program hello

var x;
x = 12 + 3;
var y;
y = x - 10;
```

We design the abstract syntax of this DSL with the following MOF metamodel:



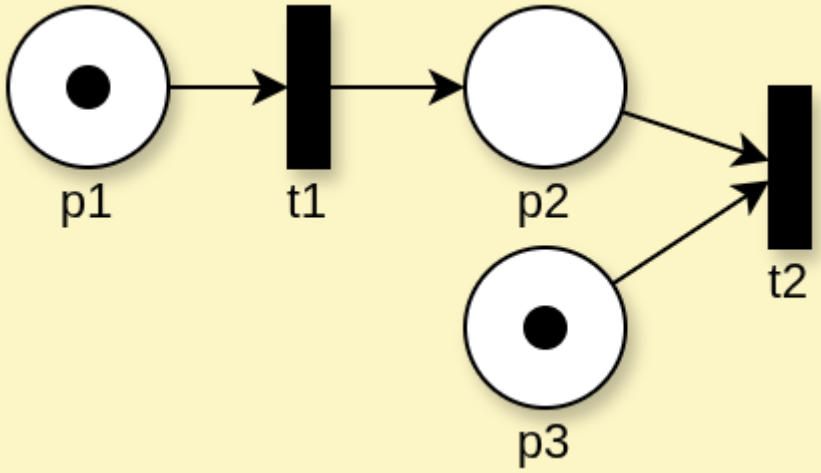
Using the classes of this abstract syntax, we can instantiate an AST representing the example program shown above:



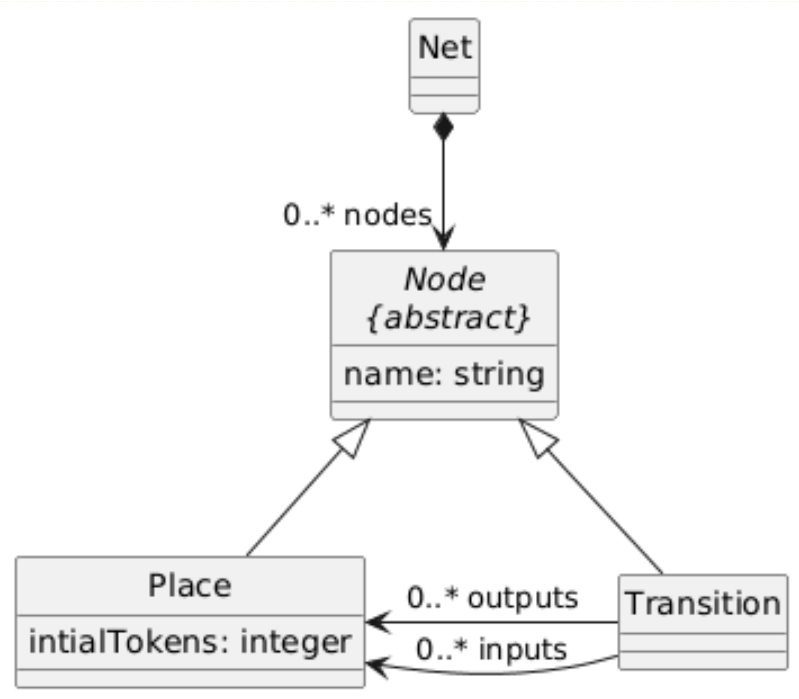
Example 10. MOF abstract syntax of a Petri nets DSL

We would like a DSL that allows the modeling of Petri nets [https://en.wikipedia.org/wiki/Petri_net]. A Petri net is a formal model used to study concurrent systems. It is composed of a set of *places* (represented as circles) and a set of *transitions* (represented as black rectangles). A place contains an amount of *initial tokens*. A transition has a set of *input places*, and a set of *output places*.

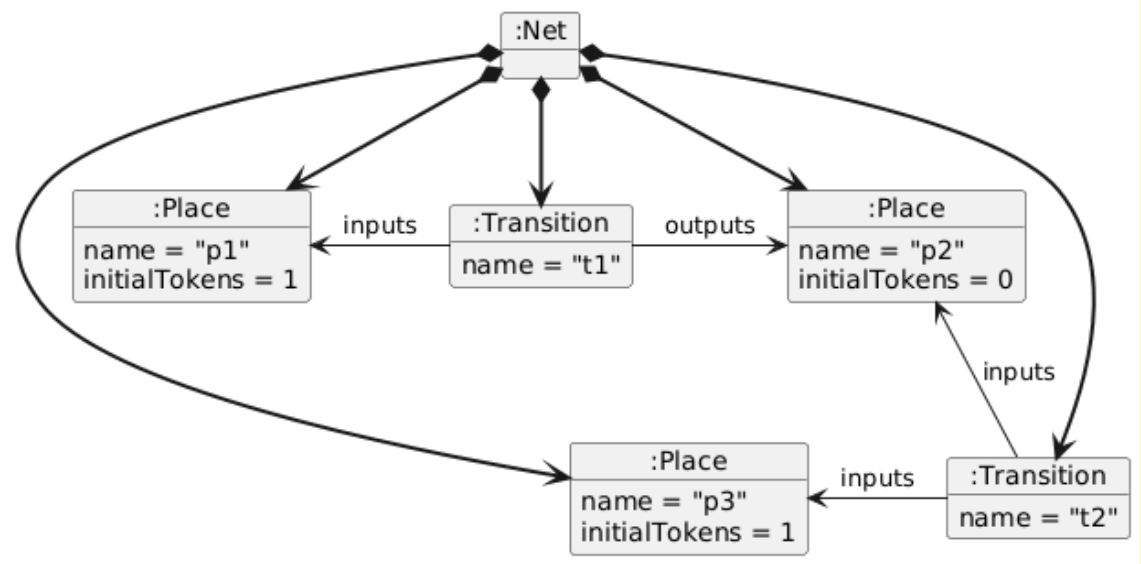
Here is an example of Petri net model represented using the traditional concrete syntax:



We design the abstract syntax of this DSL with the following MOF metamodel:



Using the classes of this abstract syntax, we can instantiate an AST representing the example model shown above:



4.1.3. How to design an abstract syntax?

Domain analysis

Where to start the design of a DSL, and especially of its abstract syntax? Since an abstract syntax is model of a specific *domain* of interest, a common approach is to borrow principes from **domain analysis**:

- **Q1: Purpose**. What is the purpose of the language? What are the use cases?
- **Q2: Stakeholders**. Who are the key stakeholders and the intended users ofthe language?
- **Q3: Concepts**. What are the key domain concepts that users care about?
- **Q4: Relations**. How are domain concepts related, and what are their relevant properties?

- **Q5: Examples**. What examples of language instances are available or can be prototyped?

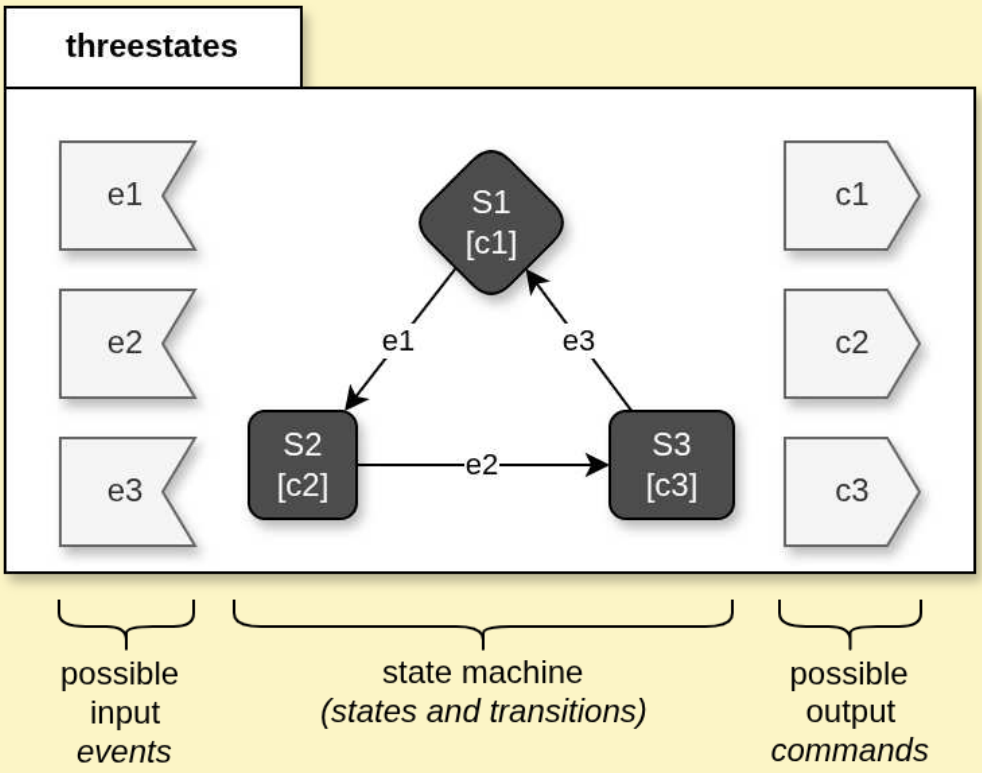


Since in a learning context are not able to discuss with real DSL stakeholders and users, we will consider Q1 and Q2 more sparsely, and focus mostly on Q3-Q5.

Example 11. Domain analysis of a States Machines DSL

We would like to design the abstract syntax of a States Machines DSL, to be able to create state machine models. We perform the following domain analysis:

- **Q1: Purpose**. To model the state-based behavior of simple systems, and to generate event-driven Java code that can be integrated in the broader implementation of said systems. State changes are trigered by input events provided by an external system. Each state has a list of output commands that are executed when the state is reached. Executing an output command eventually results in calling a concrete Java method implemented separately.
- **Q2: Stakeholders**. Systems engineers (who design the system), software engineers (who implement the software part of said system).
- **Q3: Concepts**. *State machine*, *State*, *Transition*, *Event*, *Command*.
- **Q4: Relations**. States have names and are connected through transitions. A transition has a name, a trigering input event, and possibly an trigered output command. Events and commands have names.
- **Q5: Examples**. The engineering team provides the following example, represented using a mockup graphical concrete syntax:



Guidelines for abstract syntax design

When designing an abstract syntax as a metamodel, the main goal is to capture each relevant *concept* as a *class* in a MOF metamodel, and connect these concepts using *associations*.

However there are many ways to organize a metamodel. The following **guidelines** can help you improve the quality and usefulness of your metamodel.

- **Ensure there is a single partonomy.** A partonomy is the decomposition of a class diagram into "part-of" relationships, which are defined by *compositions* between classes. Since the purpose of an abstract syntax is to be able to instantiate ASTs, it is important to make sure instances will always lead to a tree structure. Hence, *a metamodel should have a single partonomy*: each instance of a class should always be contained by another object, through a hierarchy where a single root object stands at the top.
- **Be careful with generalization**. Do not hesitate to use generalization to avoid redundancies, if multiple concepts share some common properties, but avoid genealizing two concepts that are have nothing to do with one another.
- **If relations between concepts carry information, reify them into classes.** You may be tempted to represent each relation betwen two concepts as an *association* in your metamodel; however sometimes this relation itself carry some extra information. In this case, reify this relation as a dedicated *class*.
- **Name your classes, properties and roles appropriately.** Use singular for class names, singular for properties and associations with single elements, and plural for properties and associations with multiple elements.
- **Don't put your tools (interpreter, compiler, validator, etc.) in your metamodels.** Metamodeling is not the same as traditional software modeling! You are not making an architectural diagram of a software application or tool. A class in metamodel is a domain concept, not an implementation concept.
- **Keep it as simple as possible**. Remember that designing a language is about *expressivity*: only put in your language concepts that you will require to express a solution.
- **If your DSL will be used to describe complex systems, design for modularity**. Provide concepts that allow modularization, such as modules, components, packages. Also provide concepts that can can be capitalized and instantiated multiple times, similar to classes and functions.

Example 12. Designing a MOF abstract syntax for a State Machines DSL

As started above with the domain analysis, we would like to design the abstract syntax of a States Machines DSL.

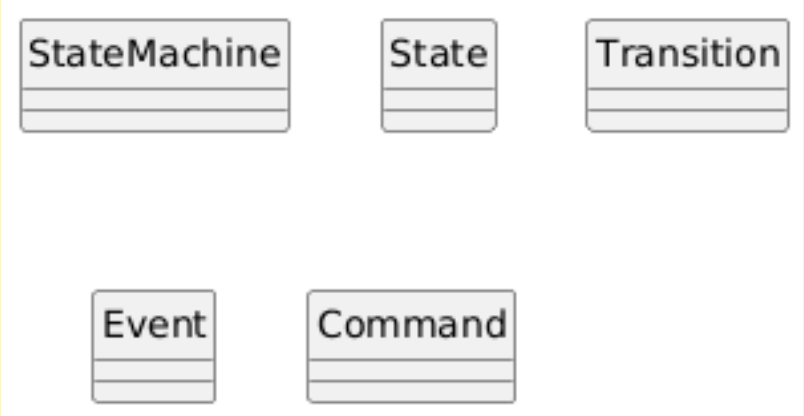
Going further in the design process, we go into more details and identify the following concepts, properties and relations:

- A *state machine* has a *name*, and contains the following:
  - A set of *commands*, each with a *name*, representing external actions that can be invoked by the state machine.
  - A set of *events*, each with a *name*, representing external events that are received by the state machine, and that may trigger transitions.
  - A set of *states*, each with a *name* and containing the following:
    - A set of *commands* that are invoked when we enter the state. These commands should first be declared at the root of the state machine.
  - A set of *transitions*, each with a *name* and containing the following:
    - an *event* that triggers the transition when received while we are in the state
    - a *target state* that is reached when the transition is fired. This state should be

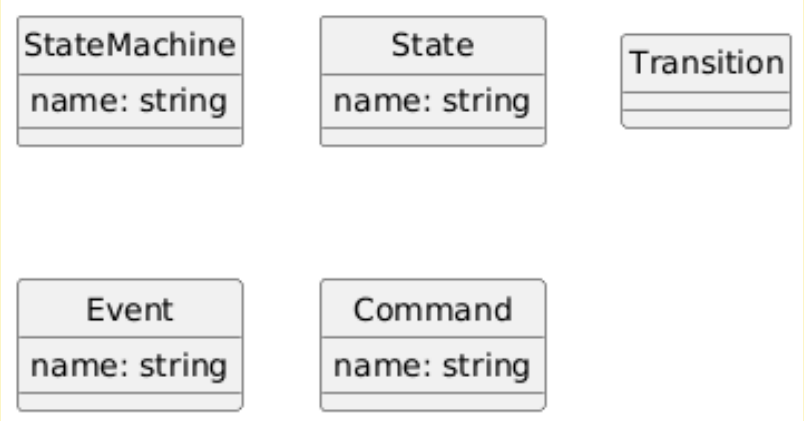
already part of the state machine.

We start the process of making a **MOF metamodel** with all this information.

First, we transform each identified concept into a dedicated *class*:



Next, when a concept contains information, we add dedicated *properties*. Here, many concepts require a `name` property:



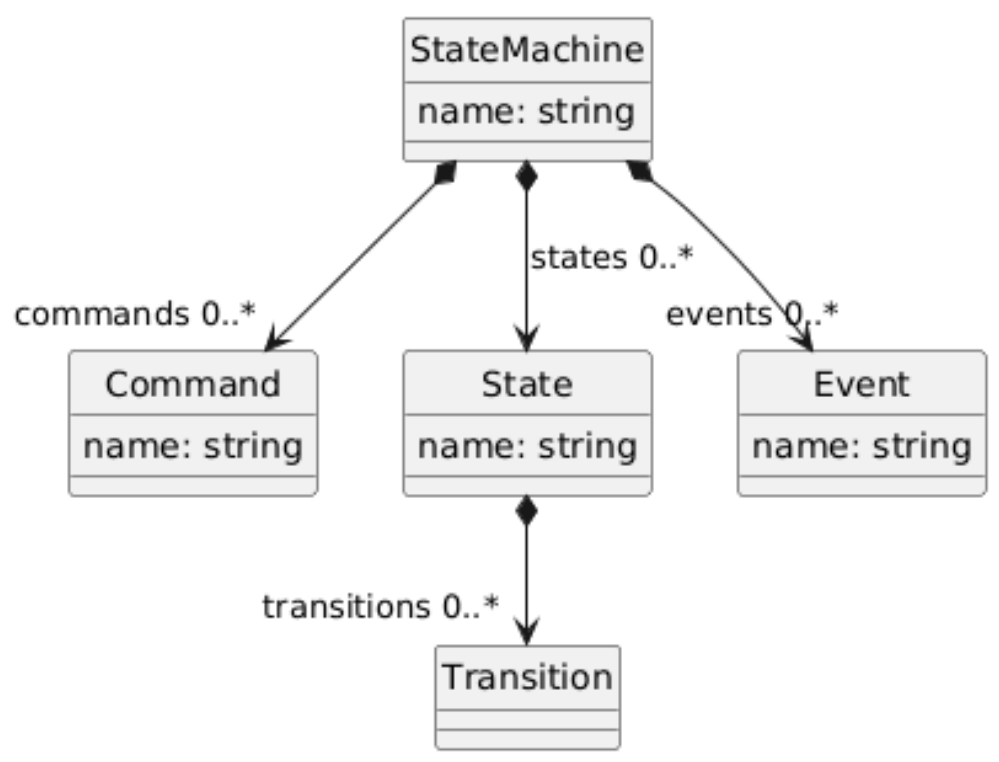
Next, a crucial step is to define the partonomy of our metamodel using *compositions*. The goal is to make sure that each object of an AST created using our metaodel is *contained* within anotion object.

- We identify `StateMachine` as a logical *root class* for our partonomy, which means each AST will contains a single root `StateMachine` object at the top, which will transitively contain all objects of the AST.
- Since a command or an event can be reused in different states and transitions, we choose to put them at the top-level of each AST, which means putting the classes `Command` and `Event` right below `StateMachine`.
- Since nothing in particular can contain a state, we also put the `State` class at the top, right below `StateMachine`.
- Regarding transitions, we observe that it must be connected both to an *input* state, and to an *output* state. To mark the difference between input and output relations, we decide that a state will *own* all of its output transitions. Therefore, we put a composition between `State` and `Transition`.



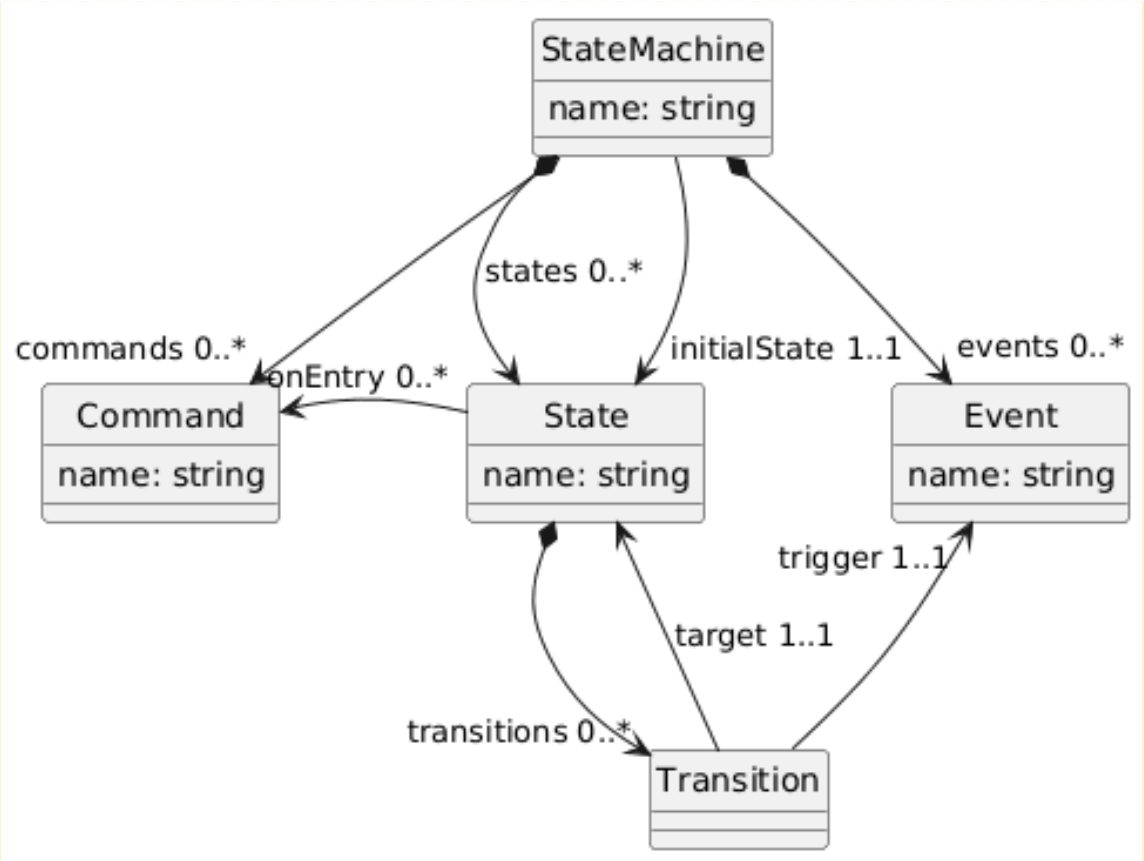
Another valid solution would be to put transitions at the same level as

states, right below `StateMachine`, but it would require one extra association in the next (and last) step.

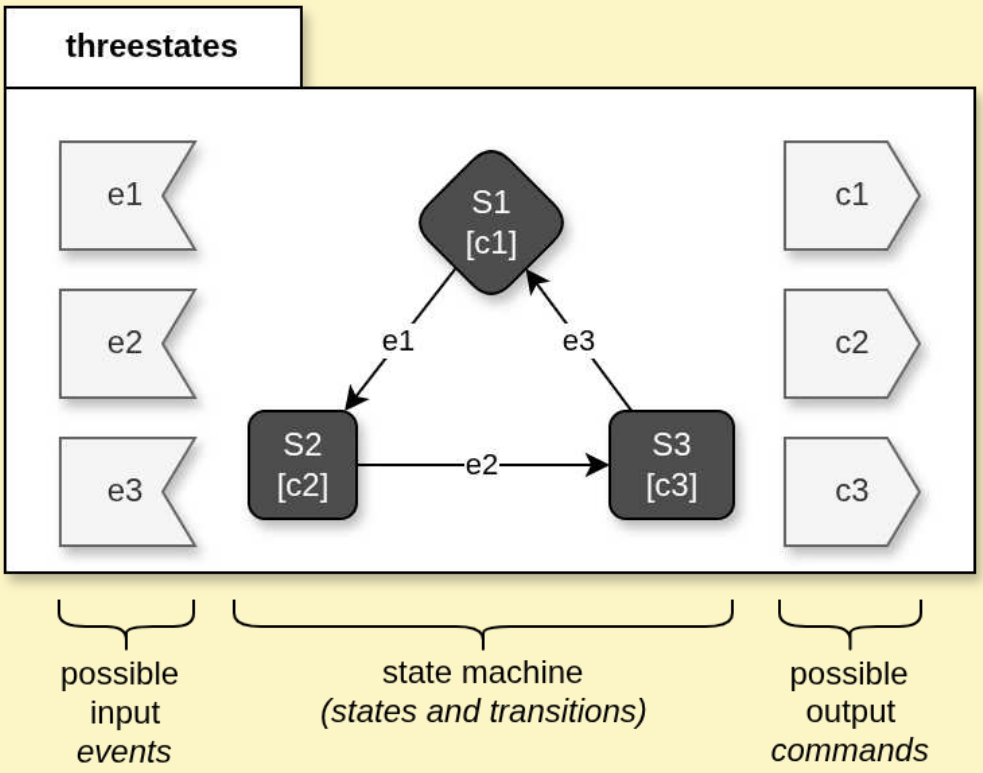


Finally, we add *associations* between classes when some objects are required to have a relationship with another existing object of the model.

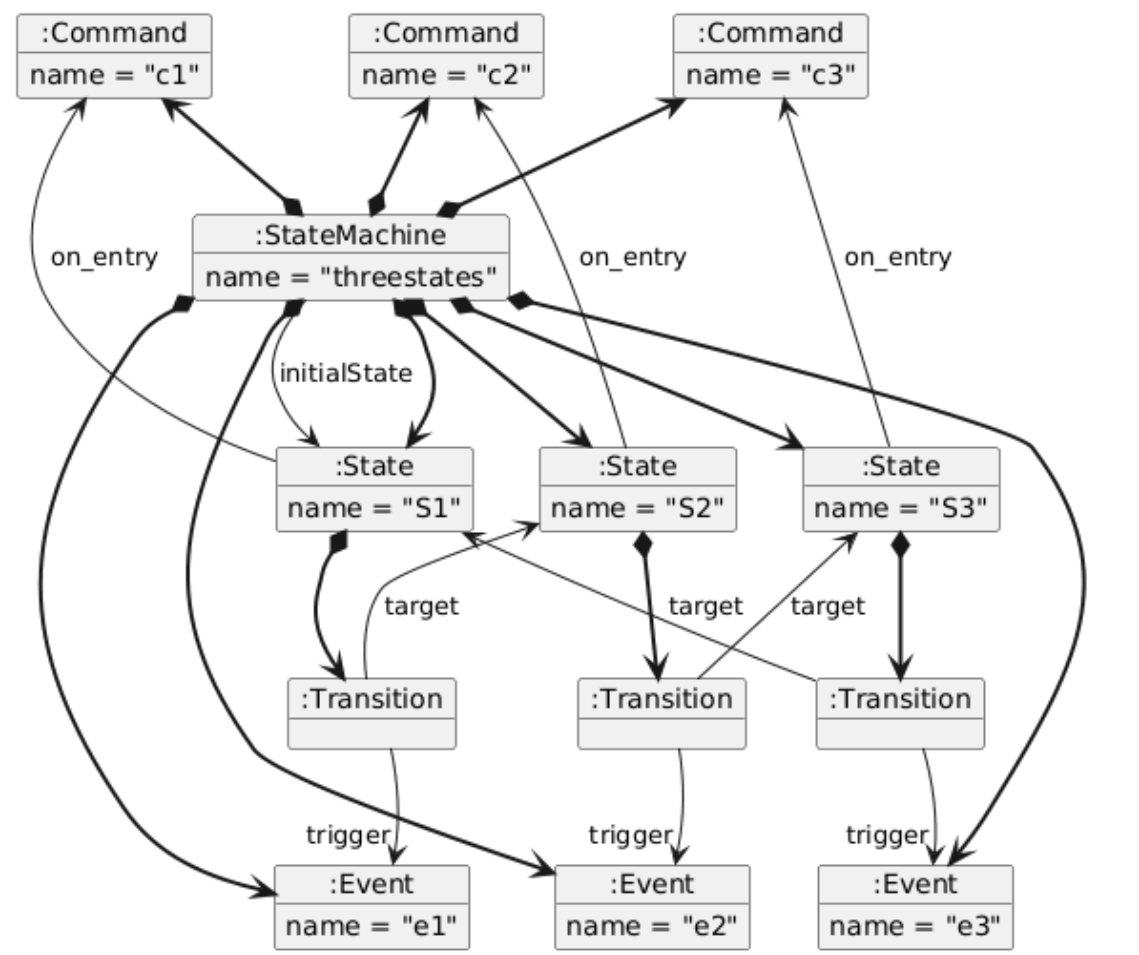
- A state must know all commands it should execute on entry, hence we add an association between `State` and `Command`.
- A state machine should have an initial state, hence we add an association between `StateMachine` and `State`.
- A transition should have an output state, hence we add an association between `Transition` and `State`.



Using this final metamodel as our abstract syntax for our State Machines DSL, we are able to use it to create an AST for the following example model:



which results in the following AST:





A complex AST is rapidly difficult to organize and read. But this is not a problem: it is not actually made to be shown! The goal of an AST is only to be an *in-memory* representation of a program.

4.2. Implementating an abstract syntax in Langium

In this section we will see how to implement an abstract syntax design into usable **data types** using Langium.

4.2.1. The Langium Grammar Language

A Langium grammar is written in the `packages/language/src` folder in the form of one or multiple `.langium` files written using the *Langium Grammar Language*. This language can be divided in two parts:

- A subset of the Langium Grammar Language called "Declared Types" [https://langium.org/docs/reference/semantic-model/#declared-types] can be used to implement the *abstract syntax* of the developed language, in the form of simple object-oriented data types.
- The rest of the Langium Grammar Language [https://langium.org/docs/reference/grammar-language/] can be used to implement the *concrete syntax* of the developed language, in the form of grammar rules.



Quite confusingly, the Langium documentation [https://langium.org/docs/] uses the term *Semantic Model* to refer to the *abstract syntax* of the developed language. Be

careful: while the word "semantic" is used, this has nothing to do with what we called earlier the *semantics of the language*!



The abstract and concrete syntaxes can either be defined in separate `.langium` files, or in the same `.langium` file.



In this course we do not cover all the many possibilities offered by the Langium Grammar Language. For more information, refer to the corresponding page in the official documentation [https://langium.org/docs/reference/grammar-language/].

4.2.2. Implementing the abstract syntax using *Declared Types*

Defining the data type for a **concept** of the abstract syntax is achieved using the `interface` keyword:

```
interface MyConcept {
    ...
}
```



This is similar to a *class* in an object-oriented model or program, but without operations/methods.

A concept can contain **properties**, each with a name and a mandatory type using the `:` separator:

```
interface MyConcept {
    aPrimitiveProperty: string
    aContainmentProperty: OtherType
    aCrossReferenceProperty: @AnotherType
}
```

There are three main types of properties:

- A *primitive property* is typed with one of the Langium primitive types: `boolean`, `number`, `string`, `bigint` or `Date`.
- A *containment* property is typed with another existing concept of the abstract syntax. In an AST, when a node A is linked to a node B using a containment, the node B is said to be *contained* (or *owned*) by the node A. In an AST, a node can only be contained by a single container node at a time.
- A *cross-reference* property is typed with another existing concept of the abstract syntax, using the character `@` as prefix. In an AST, when a node A is linked to a node B using a cross-reference, the node B is said to be *referenced* (or *known*) by the node A.



A cross-reference from A to B is only possible if:

- The concept B contains a **primitive property called `name`** and with type `string`. This is because the concrete syntax requires a way to encode how to identify another existing element.
- An instance of B is always (transitively) contained in a container node of the instance A that possess this cross-reference. This means if B is contained

deeper in a separate sub-tree of the AST, it won't be accessible. This limitation can be solved by implementing a *scope provider*, see the Langium documentation on scoping [https://langium.org/docs/recipes/scoping/].



Apart from the root node, **each node of an AST must be contained in another node**.

A property can also define a **cardinality** using either the optional suffix `?` on the property identifier, or the array suffix `[]` on the type name.

```
interface MyConcept {
    anOptionalProperty?: string
    aArrayProperty: number[]
}
```

- An *optional* property means that there can be zero or one value.
- A *array* property means that there can be zero or multiple values.



It is of course possible to combine the different types of properties with the cardinality suffixes to define "optional containments", "arrays of primitives", "arrays of cross-references", etc.

Finally, a concept can be defined as an **subtype** of another existing concept using the `extends` keyword:

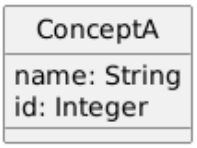
```
interface MyConcept extends OtherConcept {
    ...
}
```

This gives the possibility to *generalize* different concepts by introducing a supertype, which is both useful to define a reference to multiple types, and to avoid redundant properties.

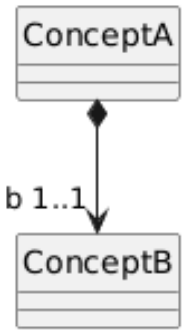
4.2.3. From MOF to Langium Declared Types

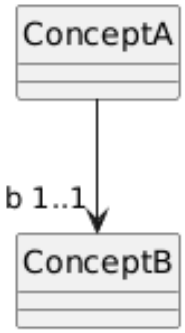
As mentioned above, Langium Declared Types are mostly similar to object-oriented *classes*. It is therefore possible to directly map a MOF metamodel into Langium as shown in this table:

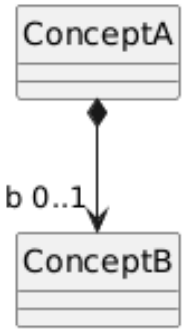
	MOF	Langium

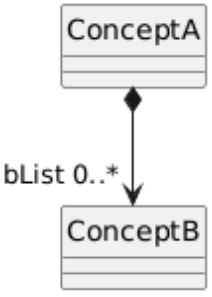
Properties		`interface ConceptA {`
		`  name: string`
		`  id: number`
		`}`

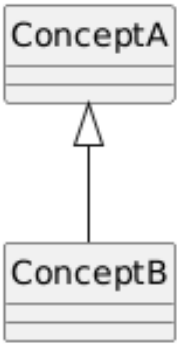
	MOF	Langium

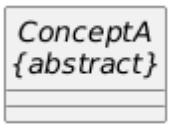
Composition		`interface ConceptA {`
		`  b: ConceptB`
		`}`
		
		`interface ConceptB {}`

Associations		`interface ConceptA {`
		`  b: @ConceptB`
		`}`
		
		`interface ConceptB {}`

Optional composition		`interface ConceptA {`
		`  b?: ConceptB`
		`}`
		
		`interface ConceptB {}`

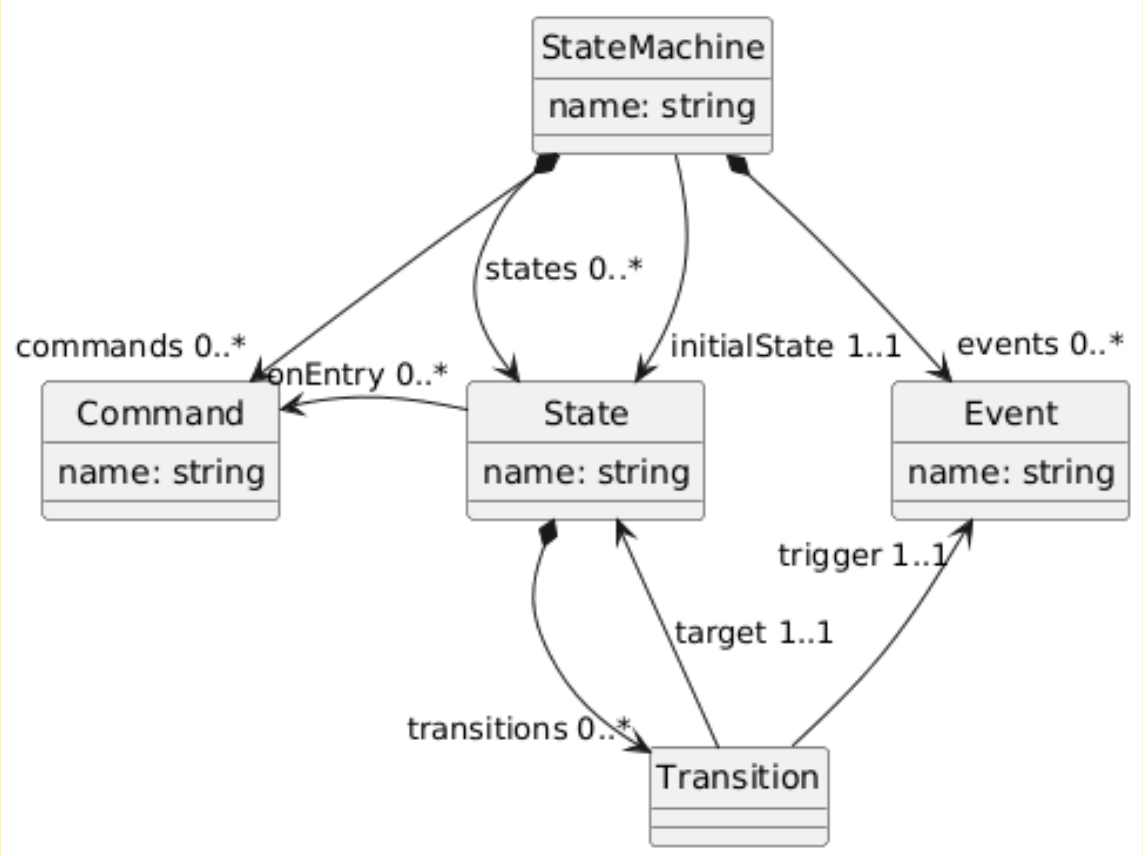
Array property		`interface ConceptA {`
		`  b: ConceptB[]`
		`}`
		
		`interface ConceptB {}`

Generalization		`interface ConceptA {}`
		
		`interface ConceptB extends ConceptA {}`

	MOF	Langium
Abstract classes		Unfortunately there are no abstract types in Langium: `interface ConceptA {}`
Enumerations		There are no enums in Langium, but we can use a union type: `type Color = 'Red' | 'Blue'`

Example 13. Langium abstract syntax of a State Machines DSL

We consider the abstract syntax we designed earlier for the State Machines DSL:



We implement this abstract syntax using **Langium Declared Types** :

```
interface StateMachine {
    name: string
    commands: Command[]
    states: State[]
    events: Event[]
    initialState: @State
}

interface Command {
```

```
    name: string
}

interface State {
    name: string
    transitions: Transition[]
    on_entry: @Command[]
}

interface Event {
    name: string
}

interface Transition {
    trigger: @Event
    target: @State
}
```

In our Langium project, we put this file in the following location:

```
packages/language/src/state-machines-dsl.abstractsyntax.langium
```

And in the main grammar file located in `packages/language/src/state-machines-dsl.langium` (where we will ultimately implement our concrete syntax), we add at the top the following line to import our abstract syntax:

```
import './state-machines-dsl.abstractsyntax'
```

4.3. Manually constructing an AST in Langium



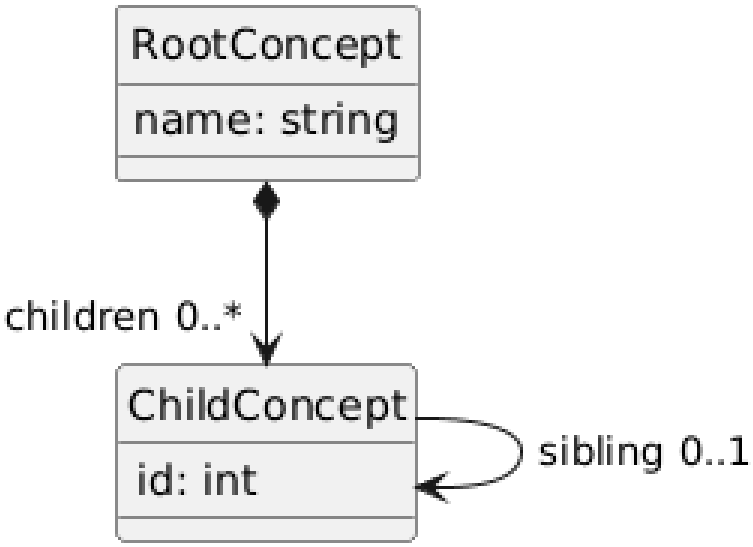
In practice it is never required to manually create an AST, because the parser always does it for us. This part is therefore presented mostly for learning purposes.

Once an abstract syntax has been implemented, we may want to manually test it by programatically constructing an AST using the TypeScript data types that have been generated.

Unfortunately, the Typescript data types that have been generated only include *interfaces* but no *classes* with constructors that would allow easily creating proper Typescript objects. In fact, the only officially supported way to create an AST in Langium is by writing a program using the DSL concrete syntax, and then by asking the *parser* to parse said program into an AST. **But at this stage, we have no parser since we have not yet designed nor implemented our concrete syntax!**

There is however a way to *circumvent this limitation* of Langium using the possibiblity in TypeScript to create an object *without a class*.

In this section, we will use the following example abstract syntax to illustrate an AST construction:



We consider this abstract syntax is implemented in Langium as follows:

```
interface RootConcept {
    name: string
    children: ChildConcept[]
}

interface ChildConcept {
    id: number
    sibling?: @ChildConcept
}
```

And we consider we have already called the Langium generator (`npm run langium:generate`) which generated the TypeScript data types we can import.

4.3.1. Importing the required TypeScript types

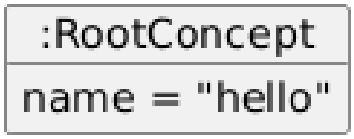
Considering we are writing code within the `packages/language/src` folder, we first have to import the following types:

```
import { AstUtils } from "langium"
import { RootConcept, ChildConcept } from "./generated/ast.js"
```

- The first line imports a utility class from Langium that we will need,
- The second line imports the two concepts of the example abstract syntax shown just before.

4.3.2. Creating a root object

Consider the following AST:



We can create this AST with the following TypeScript program:

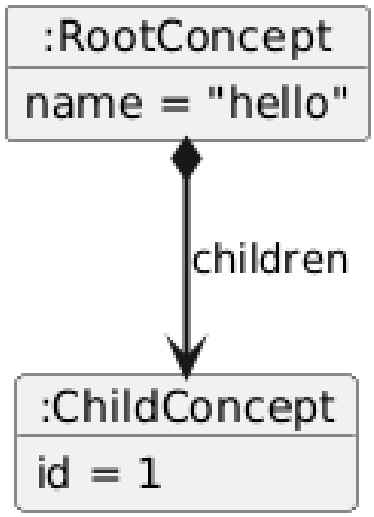
```
const root : RootConcept {
    $type: "RootConcept",
    name: "hello",
    children: []
}
```

Note that :

- For all AST nodes, Langium requires us to store the name of the concept in the `$type` attribute,
- Even if there are no `children`, we have to initialize an empty list.

4.3.3. Creating a composition with another object

Consider the following AST:



We can create this AST with the following TypeScript program:

```
const child1 : child1 = {
    $type: "ChildConcept",
    $container: undefined!,
    id: 1,
}

const root : RootConcept = {
    $type: "RootConcept",
    name: "hello",
    children: [ child1 ]
}
```

```
AstUtils.linkContentToContainer(root, {deep: true})
```

Note that:

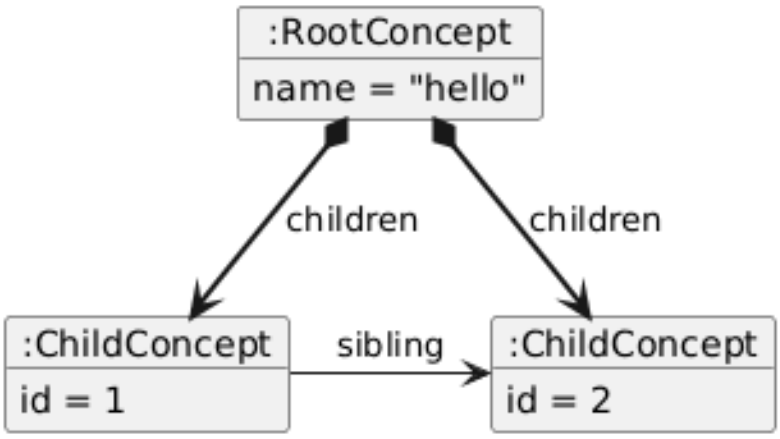
- When creating the `ChildConcept` object, we are required to set an attribute `$container` to `undefined!`, because at this stage the object is not yet contained in a parent node of the AST,
- The last line calls the Langium `linkContentToContainer` utility function to automatically set all the `$container` attributes of the whole AST, once objects have been connected together. Here, the `$container` attribute of the `ChildConcept` element will be automatically changed to point to the `RootConcept` element.



While type-safe, the API we are using to manually create the AST does *not* ensure that the tree is "correct by construction". This means it is possible to create invalid ASTs, such as trees where the same node is simultaneously contained in two different parent nodes.

4.3.4. Creating a cross-reference from an object to another

Consider the following AST:



We can create this AST with the following TypeScript program:

```
const child1 : ChildConcept = {
    $type: "ChildConcept",
    $container: undefined!,
    id: 1,
}

const child2 : ChildConcept = {
    $type: "ChildConcept",
    $container: undefined!,
    id: 2,
}

child1.sibling = { $refText: undefined!, ref: child2 }

const root : RootConcept = {
    $type: "RootConcept",
```

```
    name: "hello",
    children: [ child1, child2 ]
}

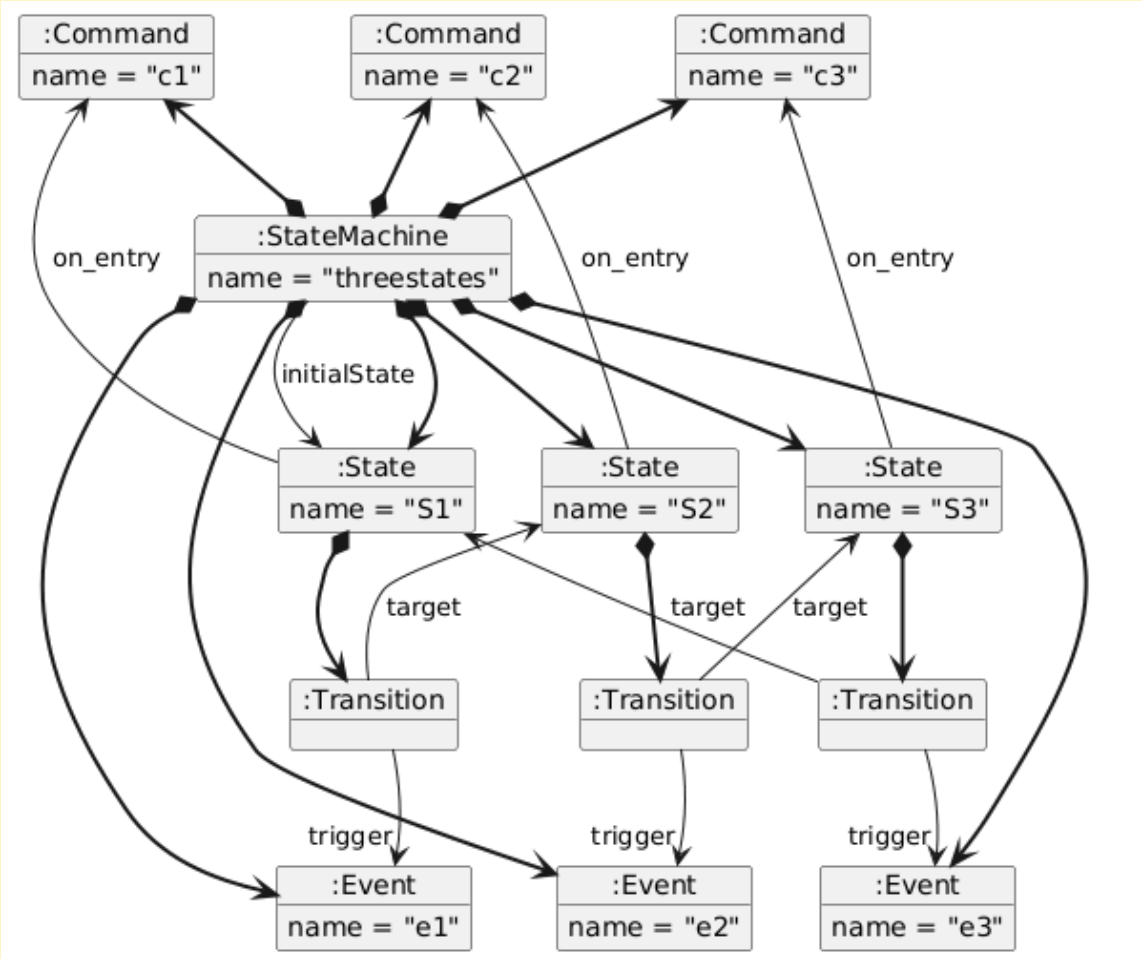
AstUtils.linkContentToContainer(root, {deep: true})
```

Note that:

- For the `sibling` link between the two `ChildConcept` objects, we have to set it through a dedicated object with a `$refText` attribute (that we don't need because it is only used by the parser), and a `ref` attribute that contains the actual pointer to the other object being referenced.

Example 14. Manually creating an AST for the State Machines DSL

Consider the following AST that we previously shown for the State Machines DSL:



We can create this AST with the following TypeScript program:

```
import { AstUtils } from "langium"
import { StateMachine, Event, State, Transition, Command } from "./generated/ast.js"


const c1 : Command = {
    $type: "Command",
    $container: undefined!,
```

```
    name: "c1"
}

const c2 : Command = {
    $type: "Command",
    $container: undefined!,
    name: "c2"
}


const c3 : Command = {
    $type: "Command",
    $container: undefined!,
    name: "c3"
}

const e1 : Event = {
    $type: "Event",
    $container: undefined!,
    name: "e1"
}

const e2 : Event = {
    $type: "Event",
    $container: undefined!,
    name: "e2"
}

const e3 : Event = {
    $type: "Event",
    $container: undefined!,
    name: "e3"
}


const s1 : State = {
    $type: "State",
    $container: undefined!,
    name: "S1",
    on_entry: [{ $refText: undefined!, ref: c1}],
    transitions: []
}

const s2 : State = {
    $type: "State",
    $container: undefined!,
    name: "S2",
    on_entry: [{ $refText: undefined!, ref: c2}],
    transitions: []
}

const s3 : State = {
    $type: "State",
    $container: undefined!,
    name: "S3",
    on_entry: [{ $refText: undefined!, ref: c3}],
    transitions: []
```

```
}

const s1ToS2 : Transition = {
    $type: "Transition",
    $container: undefined!,
    target: { $refText: undefined!, ref: s2},
    trigger: { $refText: undefined!, ref: e1},
}

s1.transitions.push(s1ToS2)

const s2ToS3 : Transition = {
    $type: "Transition",
    $container: undefined!,
    target: { $refText: undefined!, ref: s3},
    trigger: { $refText: undefined!, ref: e2},
}

s2.transitions.push(s2ToS3)

const s3ToS1 : Transition = {
    $type: "Transition",
    $container: undefined!,
    target: { $refText: undefined!, ref: s1},
    trigger: { $refText: undefined!, ref: e3},
}

s3.transitions.push(s3ToS1)

const root : StateMachine = {
    $type: "StateMachine",
    name: "threestates",
    commands: [c1,c2,c3],
    events: [e1,e2],
    states: [s1,s2],
    initialState:  { $refText: undefined!, ref: s1},
}

AstUtils.linkContentToContainer(root, {deep: true})
```



The `push` operation is used to add an element to the list of `transitions` of each state.

We put this code in a file `packages/language/src/ast-test.ts`, then we build the project:

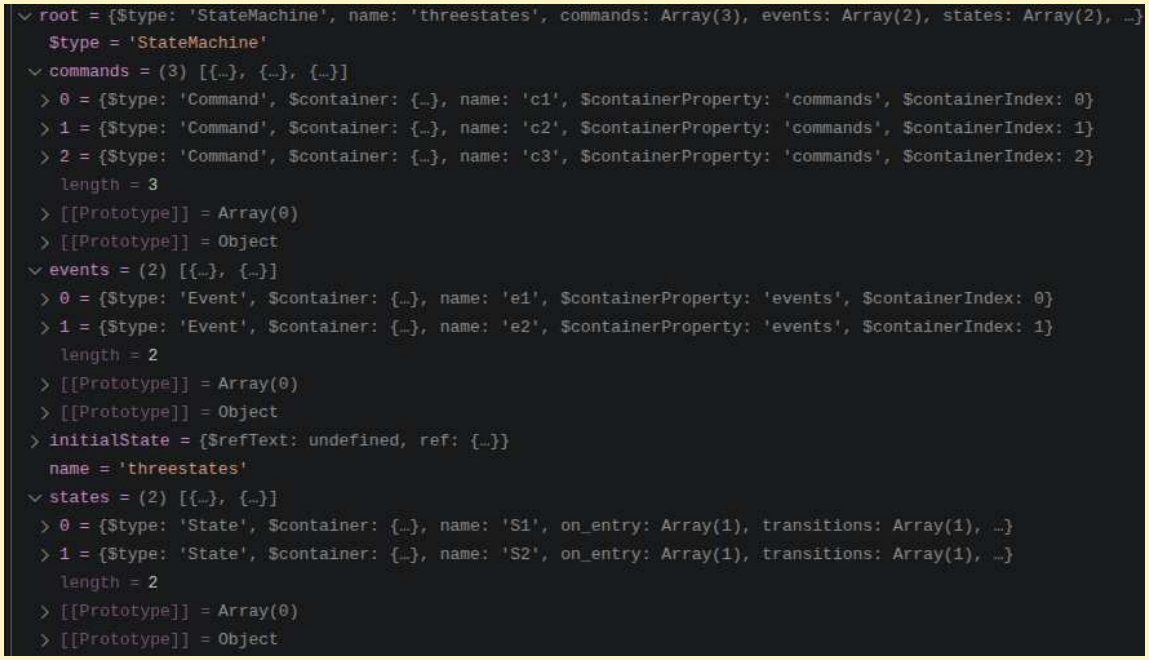
```
npm run build
```

And finally we can run the compiled file using `node`:

```
node packages/language/out/ast-test.js
```

But the code does nothing we can see! So we run it in a *JavaScript Debug Terminal*, we set a

breakpoint after the creation of the AST, and we can browse our AST loaded in memory in the variable view of the debugger:



Chapter 5. Ch5. Textual concrete syntax

The **concrete syntax** is a mandatory part of a software language that defines how a user of the language (a programmer or a domain expert) will concretely read and write programs in said language. The concrete syntax can be seen as the *user interface* (UI) of a software language.

In this chapter we will first learn how to *design* a concrete syntax using annotated mockups, then we will learn to *implement* an abstract syntax in a Langium DSL.



This course only deals with textual syntaxes designed "from the ground up". Another possibility, not shown in the course, is to rely on standardized **markup languages** (such as YAML, TOML, JSON) to provide a textual concrete syntax for a DSL.

In such a case, a concrete syntax takes the form of a *schema* describing what is allowed or not in a program, of an existing *parser* for the chosen markup language, and of a transformation able to map the parser output to a proper AST.

5.1. Designing a textual concrete syntax

5.1.1. Mockups

In user interface (UI) design, a **mockup** is a purely graphical representation of what a UI should look like once implemented. It is essentially a drawing showing the expected layout and the expected appearance of the different parts of a UI. It is a very efficient *example-driven* way to design a graphical UI, and a way to unleash creativity without being constrained by technical aspects.

In SLE, since a concrete syntax is a sort of UI for a software language, **mockups are also a great approach to design a concrete syntax**. Given a description of an example program, or given an AST of an example program, we can create a *mockup textual representation* showing the concrete syntax we envision for this example.

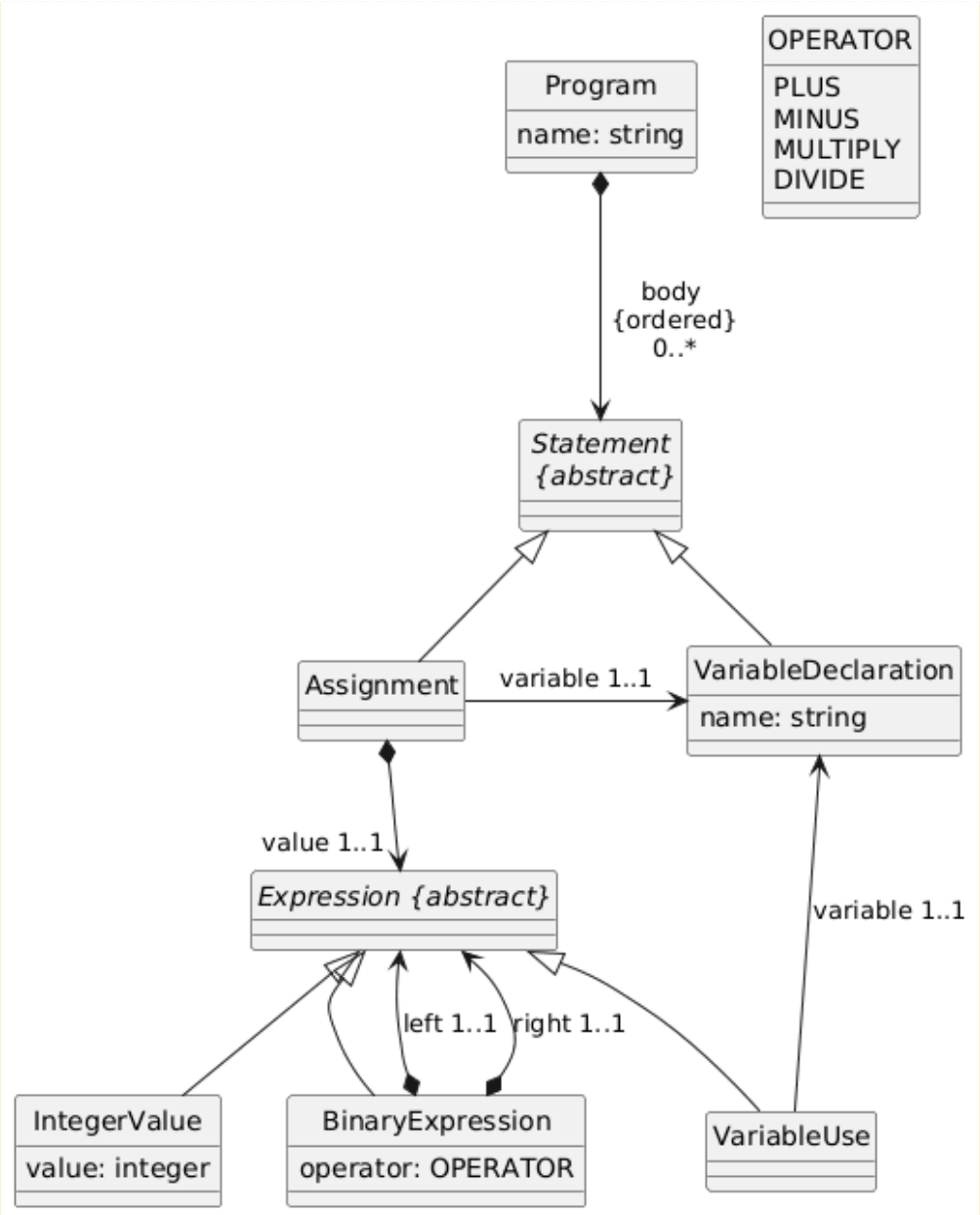


A **good mockup** must show examples all concepts, properties and associations of the abstract syntax of the DSL!

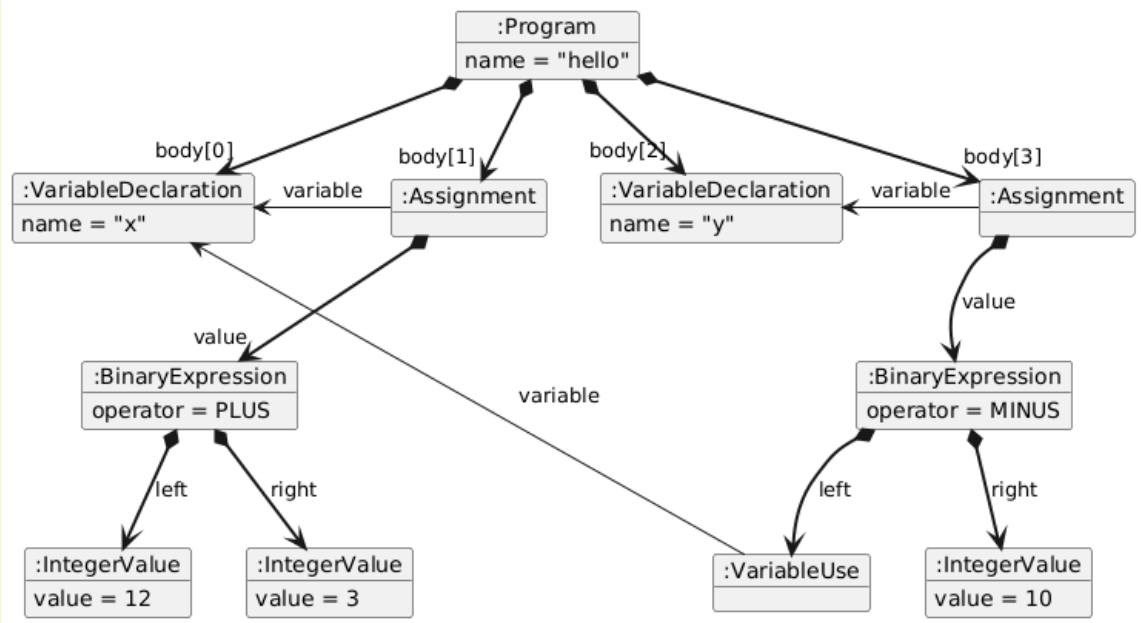
Example 15. Mockup of a Tiny programming DSL program

We continue the design of a tiny programming language DSL, to write simple programs with variables and basic arithmetic expressions.

We already designed the abstract syntax of this DSL with the following MOF metamodel:



And using the classes of this abstract syntax, we already instantiated the following example AST:



We now would like to design a textual concrete syntax for this DSL. In that goal, we prepare a **mockup** concrete syntax representation of this AST, which looks like this:

```
program hello

var x;
x = 12 + 3;
var y;
y = x - 10;
```

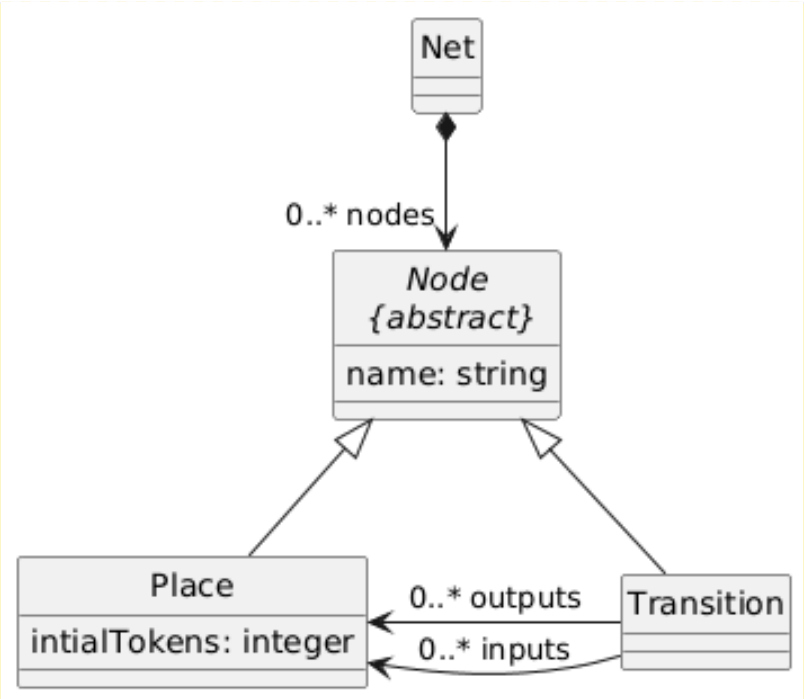


In the chapter on abstract syntaxes, we acually did the opposite: the mockup concrete syntax representation was an example part of the requirements, which we used to test the abstract syntax.

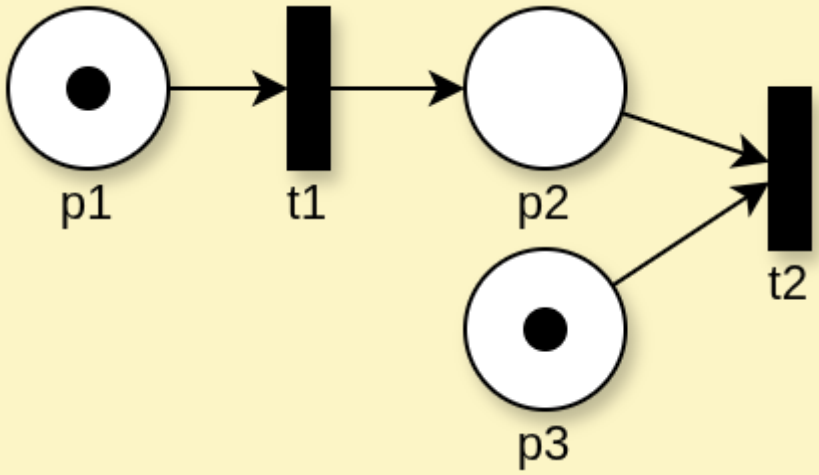
Example 16. Mockup of a Petri nets DSL program

We continue the design of a DSL that allows the modeling of Petri nets [https://en.wikipedia.org/wiki/Petri_net].

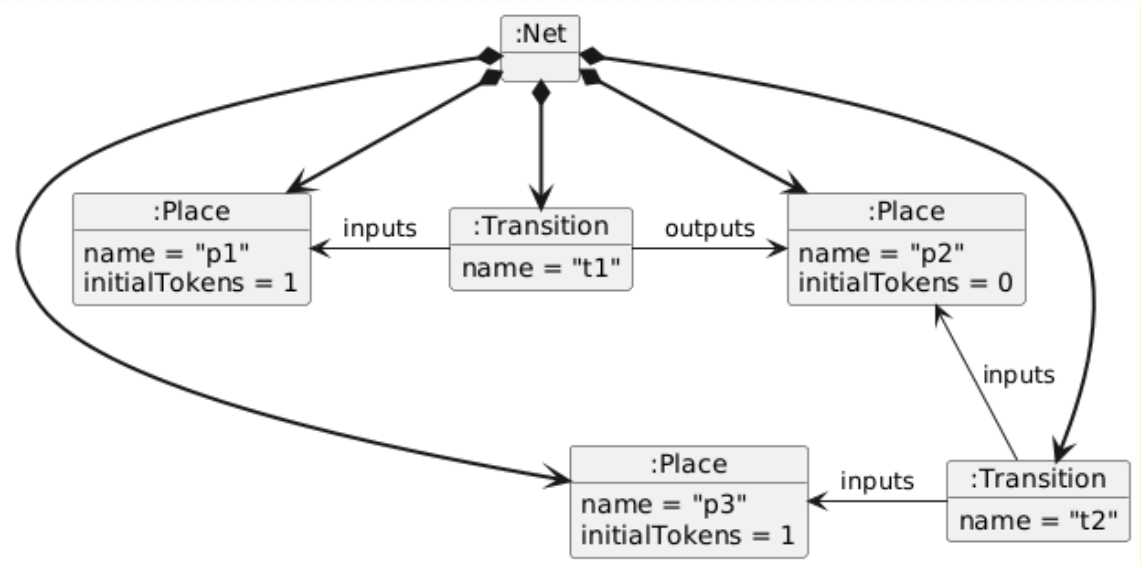
We already designed the abstract syntax of this DSL with the following MOF metamodel:



We already considered the following example of Petri net, represented with the traditional graphical syntax of the formalism:



And using the classes of this abstract syntax, we already instantiated the following example AST that corresponds to this model:



We now would like to design a textual concrete syntax for this DSL (even though Petri nets are commonly represented graphically). In that goal, we prepare a **mockup** concrete syntax representation of this AST, which looks like this:

```
PLACE p1 WITH 1 TOKEN
PLACE p2
PLACE p3 WITH 1 TOKEN

TRANSITION t1
    IN [p1]
    OUT [p2]
END

TRANSITION t2
    IN [p2,p3]
END
```

5.1.2. Tokens and token types

A textual representation of a program or model is essentially a sequence of **tokens** (usually obtained by the *lexer*, at the implementation level). A token is a sequence of characters.

There are five main types of tokens:

- **Keyword**: a reserved word of the language, usually in english, used to precisely declare specific concepts of the language (`if`, `state`, `while`, `place`, etc.), or to declare begining or end of a declaration (`begin`, `end`, `done`, etc.).
- **Punctuation**: a special character (or a combination of special characters) used to unambiguously structure and separate the different elements of a program. There are three main types of punctuation:
  - *operator*: a symbol that marks an operation applied to one or multiple elements, or that mark some element as special (`+`, `-`, `*`, `/`, `!`, `=`, etc.)
  - *delimiters*: a pair of characters that mark the beginning and end of an element or a set of elements (`(` and `)`, `{` and `}`, `[` and `]`, etc.)

    - *separator*: a single character used to separate elements in a list of elements (examples: `,`, `;`)
- **Identifier** : a word assigned by the programmer that *uniquely* designate a specific element of the AST, usually composed only of alphanumeric characters (`x`, `main`, `MyClass`, etc.)
- **Comments**: a whole sequence of characters that should be ignored by the parser (`/* a comment */`, `// another comment`, `# a third comment`, etc.)
- **Literal**: raw data value written by the programmer, such as strings, integers, booleans (`12`, `true`, `"word"`, `-8.5`, etc.)

When desining a concrete syntax using mockups, a useful step is to identify precisely *what token types are used in which parts of a given mockup*. This information will be helpful in later stages when implementing a grammar.

Example 17. Tokens of a Tiny programming DSL program

We already prepared the following mockup for a concrete syntax design of the Tiny programming DSL:

```
program hello

var x;
x = 12 + 3;
var y;
y = x - 10;
```

In this mockup, we can distinguish the following types of tokens:

- **Keywords**: `program`, `var`
- **Punctuation**:
    - *operators*: `=`, `+`, `-`
    - *delimiters*: N/A
    - *separators*: `;`
- **Identifiers**: `hello`, `x`, `y`
- **Comments**: N/A
- **Literals**: `3`, `10`

Example 18. Tokens of a Petri nets DSL program

We already prepared the following mockup for a concrete syntax design of the Petri nets DSL:

```
PLACE p1 WITH 1 TOKEN
PLACE p2
PLACE p3 WITH 1 TOKEN

TRANSITION t1
    IN [p1]
    OUT [p2]
END
```

```
TRANSITION t2
    IN [p2,p3]
END
```

In this mockup, we can distinguish the following types of tokens:

- **Keywords**: `PLACE`, `WITH`, `TOKEN`
- **Punctuation**:
  - *operators*: `=`, `+`, `-`
  - *delimiters*: `[]`
  - *separators*: N/A
- **Identifiers**: `p1`, `p2`, `p3`, `t1`, `t2`
- **Comments**: N/A
- **Literals**: `1`

5.1.3. Mapping a concrete syntax with an abstract syntax

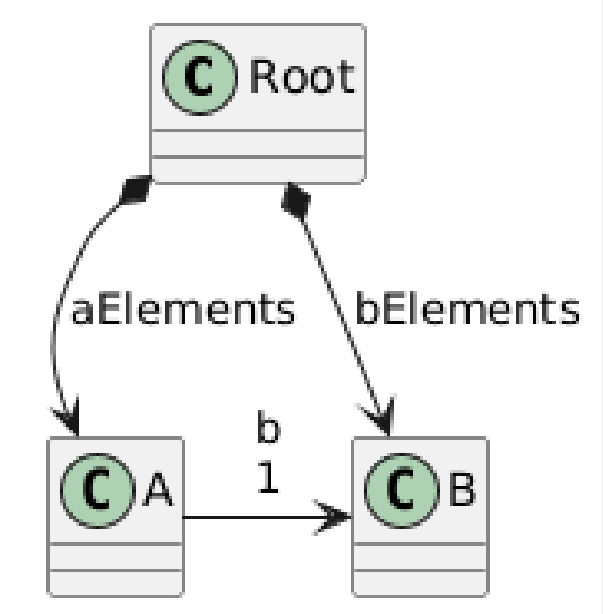
Now that we are able to create mockups and to understand the tokens it contains, the next step is to **map the concrete syntax with the abstract syntax** of the language being made. This is important to associate precisely what *concept* of the language is being represented by a given piece of concrete syntax, and to prepare a blueprint of how our future parser should build an AST given a textual representation.

When mapping a concrete syntax with an abstract syntax, tokens are usually in the following way:

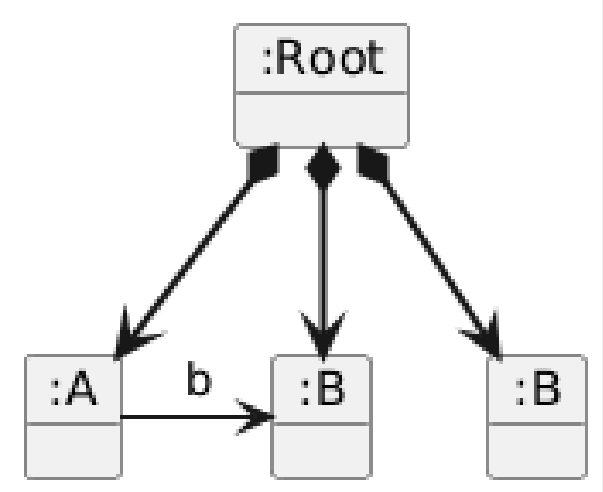
- *Keywords* and *operators* explictly mark what concepts or properties of the abstract syntax are being instantiated (example: the keyword `PLACE` declares an instance of the concept `Place`)
- *Identifiers* are used in two ways:
  - for *string properties* of the abstract syntax, when such a property is used to *name* the element (example: in `PLACE pl`, `p1` is an identifier for the `name` property of the concept `Node`)
  - for *navigable associations* of the abstract syntax, to identify an existing element that is being pointed to (example: in `in [p1]`, `p1` is an identifier for the `inputs` association of the `Transition` concept)
- *Literals* are used to represent *properties* of the abstract syntax (example: in `WITH 1 TOKEN`, `1` is a literal for the `initialTokens` property of the concept `Place`)

Representing navigable associations in a textual syntax

Consider the following abstract syntax:

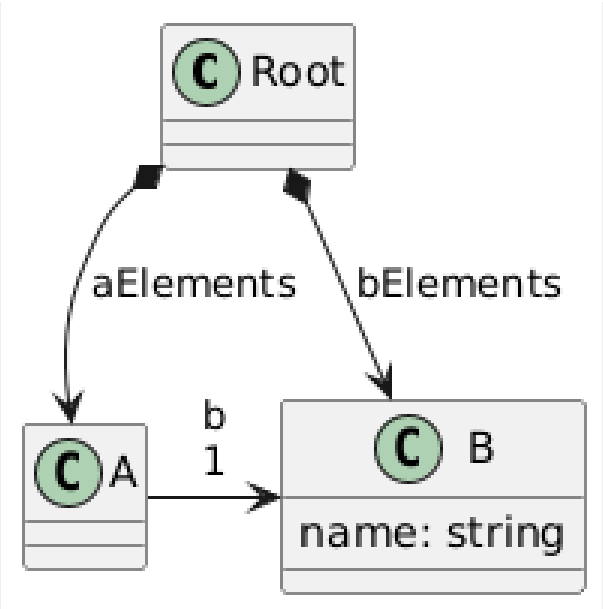


With this perfectly valid abstract syntax, we can create the following AST:

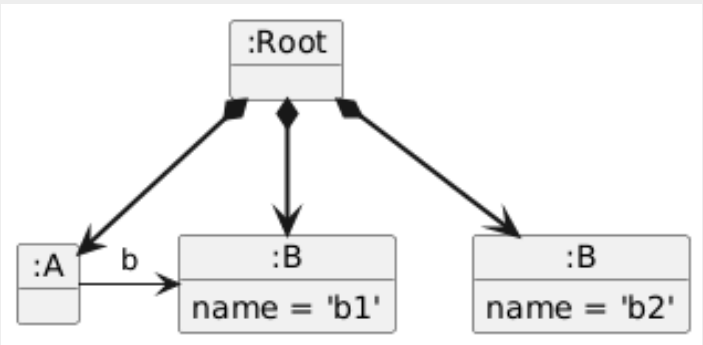


From there we would like to create a *mockup concrete syntax representation* of this AST. **Problem: how to *textually* represent that the `A` element points to the first `B` element, and not the second?** Because both `B` elements are identical, there is no way to textually disambiguate which one is being referenced by the `A` element.

The only way to solve this problem is to make sure each `B` instance can be uniquely identified through a dedicated property. The most common way to achieve this is to modify the abstract syntax and simply add a `name` property to the concept that needs to be referenced, here `B`:



Then in the AST we can give a different name to each `B` instance:



And finally we can design a mockup concrete syntax representation, for instance:

```
B #b1
B #b2
A @b1
```

Here we invent the `#` operator to specify the name of a `B` element, and we invent the `@` operator to specify the `B` element that the `A` element points to through the `b` navigable association of the abstract syntax.



While this is one example of situation where the abstract syntax must be changed to be compatible with a textual concrete syntax, **we usually prefer to avoid changing too much the abstract syntax when designing the concrete syntax**. The abstract syntax should stay close to the studied domain concepts, and should ideally not be influenced by the constraints of the concrete representation.

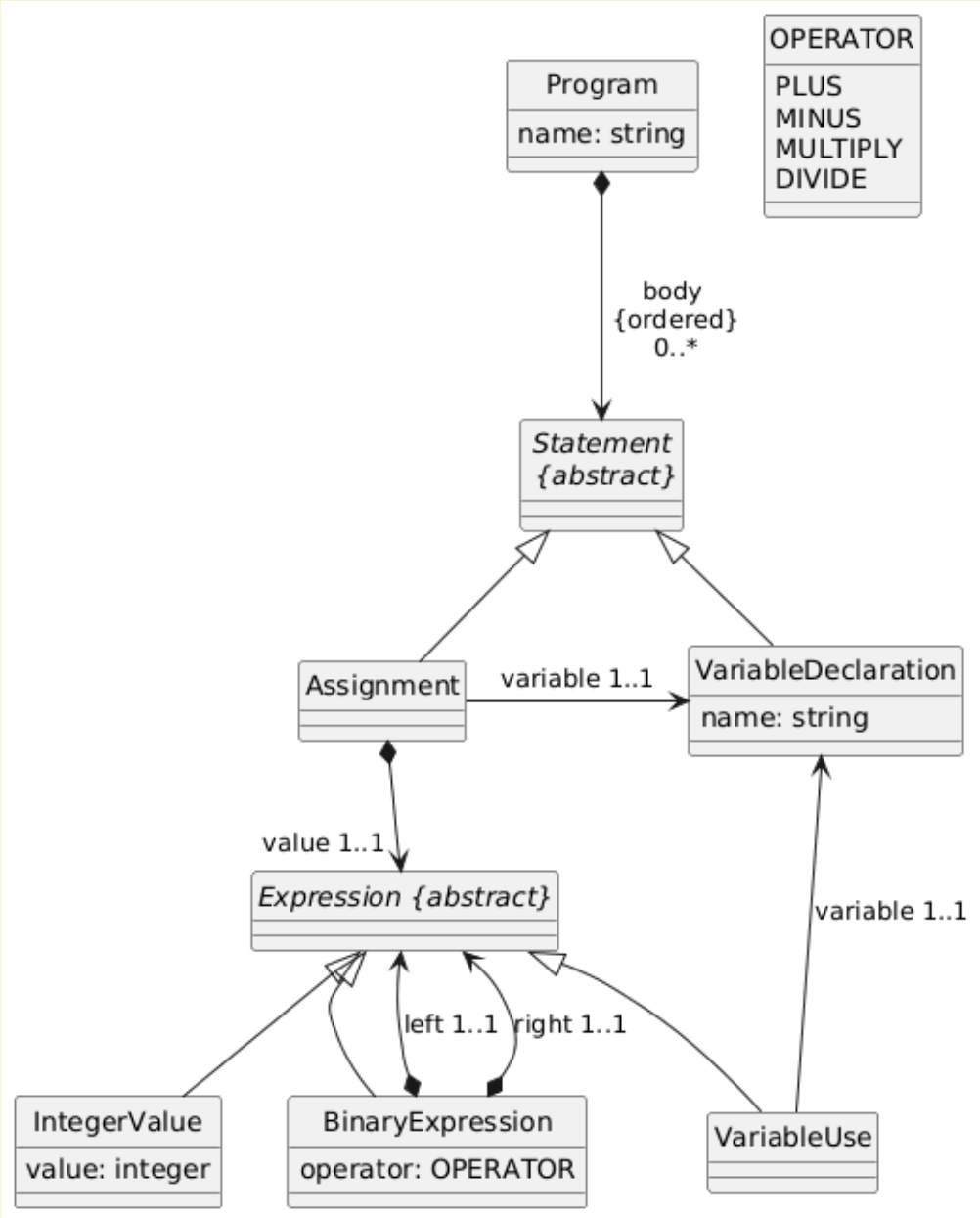
To capture the mapping from concrete syntax to abstract syntax in a graphical and intelligle way, one simple approach is to create **annotated mockups** where each mockup gives an example of concrete representation of a specific concrete *concept*, and where said mockup is *annotated* to highlight where the *properties* and *navigable associations* are managed.



It is not required to prepare an annotated mockup for an *abstract concept* of the abstract syntax: only concrete concepts can be instantiated in an AST, and hence only concrete concepts can have a textual representation.

Example 19. Annotated mockups of a Tiny programming DSL program

We designed in the previous chapter the abstract syntax of a Tiny programming DSL with the following MOF metamodel:



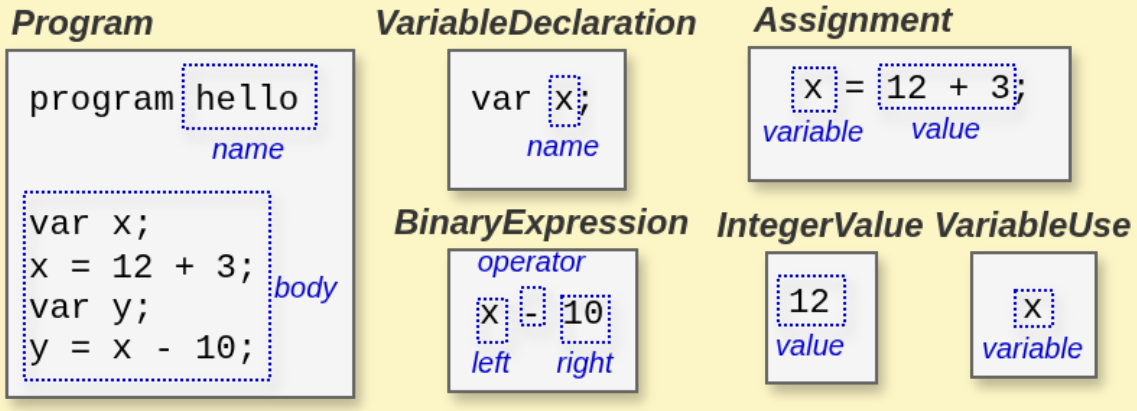
And we already prepared the following mockup for a concrete syntax:

```
program hello

var x;
x = 12 + 3;
var y;
```

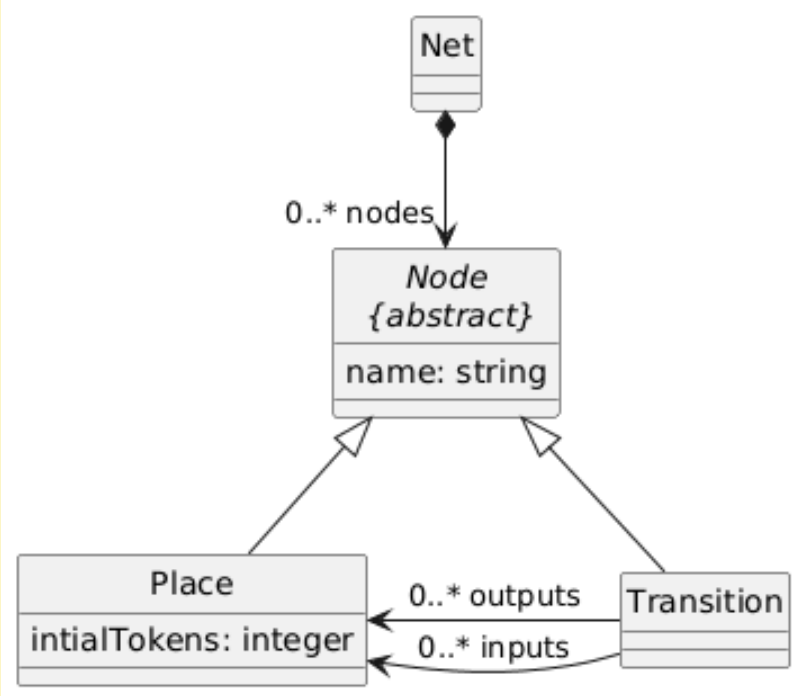
```
y = x - 10;
```

To map the proposed concrete syntax and the abstract syntax, we prepare one **annotated mockup** per concept of the abstract syntax:



Example 20. Annotated mockups of a Petri nets DSL program

We designed in the previous chapter the abstract syntax of a Petri nets DSL with the following MOF metamodel:



And we already prepared the following mockup for a concrete syntax:

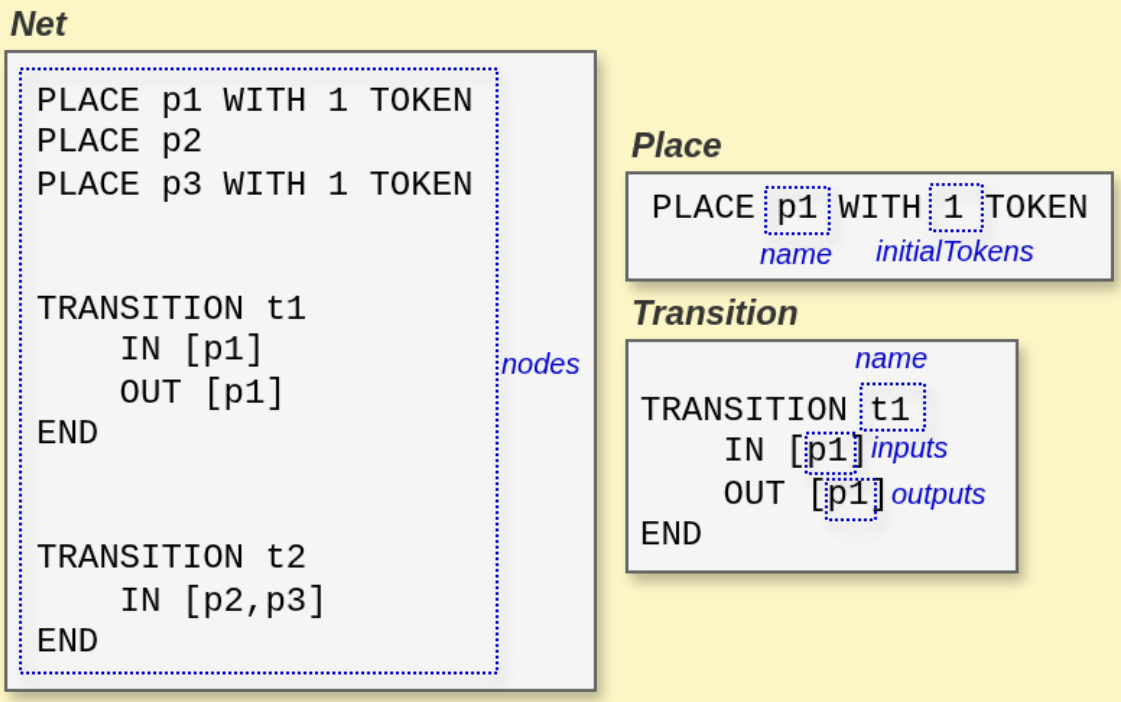
```
PLACE p1 WITH 1 TOKEN
PLACE p2
PLACE p3 WITH 1 TOKEN

TRANSITION t1
    IN [p1]
    OUT [p2]
```

```
END

TRANSITION t2
    IN [p2,p3]
END
```

To map the proposed concrete syntax and the abstract syntax, we prepare one **annotated mockup** per concept of the abstract syntax:





If some concepts, properties or associations of the abstract syntax are not shown in the annotated mockups, it means the mockup was not complete enough and should be improved.

5.1.4. Good practices

Where to start the design of a concrete syntax of a DSL? How to decide what keywords to use, what punctuation, and what structure should be preferable? In this section we review a selection of advices and common good practices.

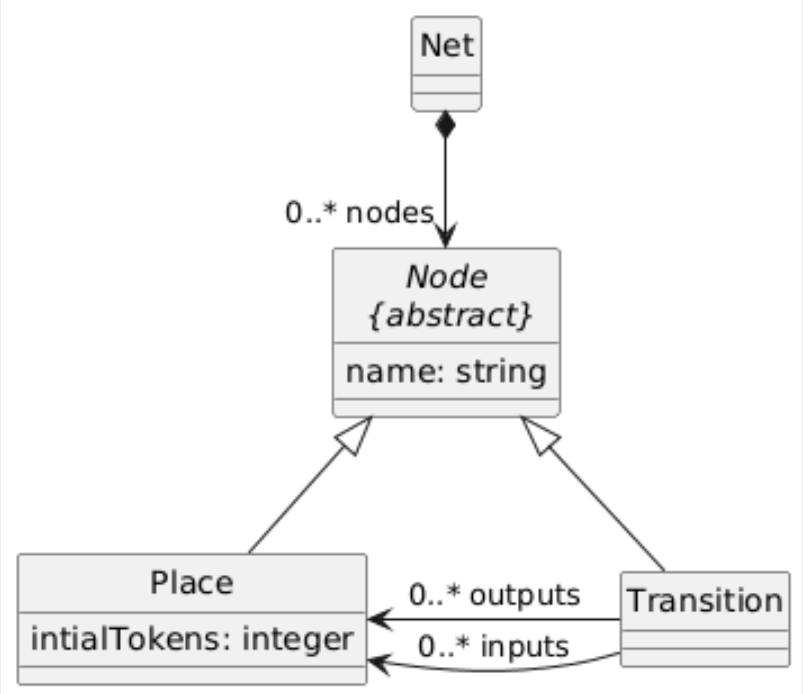
- **Consider carefully both *context* and *stakeholders*.** Stakeholders includes both people who will *write* program/model using the DSL, and (possibly different) people who will *read* resulting programs/models. All this people come with a specific technical background, a specific culture, and an existing set of habits. Ideally, a concrete syntax should take all this into account, and should provide notations that are as intuitive and easy to understand as possible for the stakeholders.
  - If they are already using formal/informal notations used by the stakeholders, use them as inspiration. For example if they are used to certain abbreviations or are used to write mathematics, the DSL concrete syntax can resue some of them.
  - If they have a specific *jargon* for their domain, don't hesitate to use them as keywords.
  - If they already use software languages that they like, have a look at the concrete syntax of

these languages and use them as inspiration. For example if they are used to C or Java programming, use lowcase keywords and use braces `{}` as delimiters.

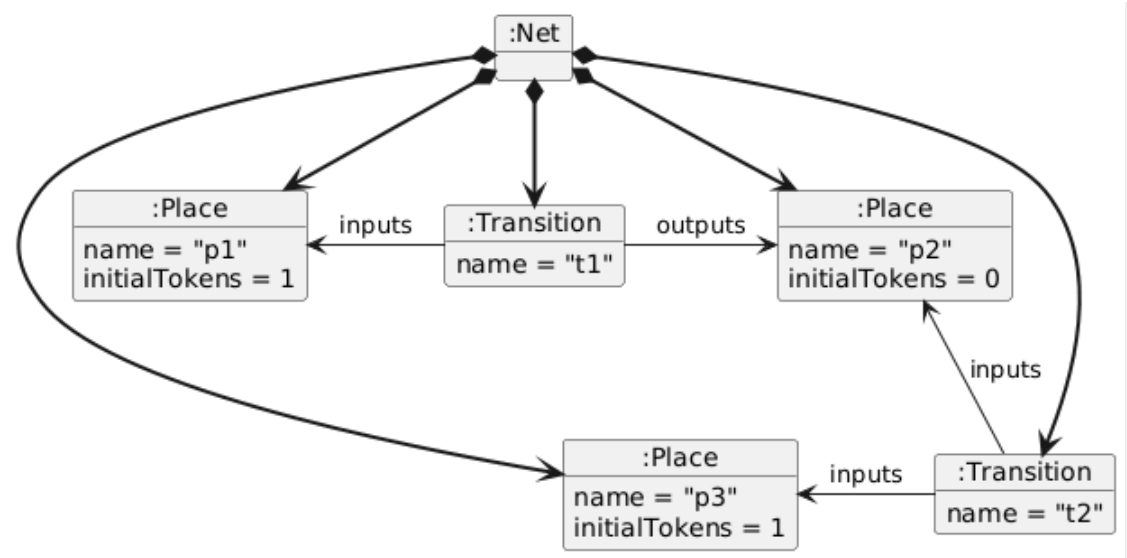
- **Avoid making a DSL just for you! (except if you are the main target user)** You are a software engineer, hence you have all the preferences of a software engineer, which may differ greatly from those of a specific domain expert.
- **Have a consistent syntax.** A program or model is made to be *read*, which is easier if everything looks balanced and homogenous. Try using a limited set of punctuation that you always use in the same way, and try to always declare things in a similar fashion. Make sure your keywords are styled similarly (lower *vs* upper case, abbreviations *vs* full words, etc.)
- **Use existing concrete syntaxes as inspirations.** Many software languages have been made already, each with different stylistic choices regarding their concrete syntax. Don't hesitate to draw inspiration or to mimic what already exists – *if it does not contradict previous points*!

One DSL, many possible concrete syntaxes

For a given abstract syntax and a given DSL, **many possible concrete syntaxes can be invented**. Consider for example the Petri nets DSL shown previously, where we designed the following abstract syntax:



And using the classes of this abstract syntax, we already instantiated the following example AST that corresponds to this model:



We previously proposed a first design using the following mockup, where we avoid braces {} because they look too programming oriented, and where we use uppercase keywords to very clearly distinguish identifiers from keywords:

```
PLACE p1 WITH 1 TOKEN
PLACE p2
PLACE p3 WITH 1 TOKEN

TRANSITION t1
    IN [p1]
    OUT [p2]
END

TRANSITION t2
    IN [p2,p3]
END
```

But another possible concrete syntax could be closer to programming languages and how variables and their attributes are assigned:

```
place p1, p2, p3;
p1.tokens = 1;
p3.tokens = 1;
tran t1, t2;
t1.in = {p1};
t1.out = {p2};
t2.in = {p2,p3};
```

Or another could use action keywords to explain how tokens move around:

```
place p1(1)
place p2
place p3(1)

transition t1 {
```

```
    take from p1
    give to p2
}

transition t2 {
    take from p2
    take from p3
}
```

Or another could provide an arrow notation to bind places and transitions together:

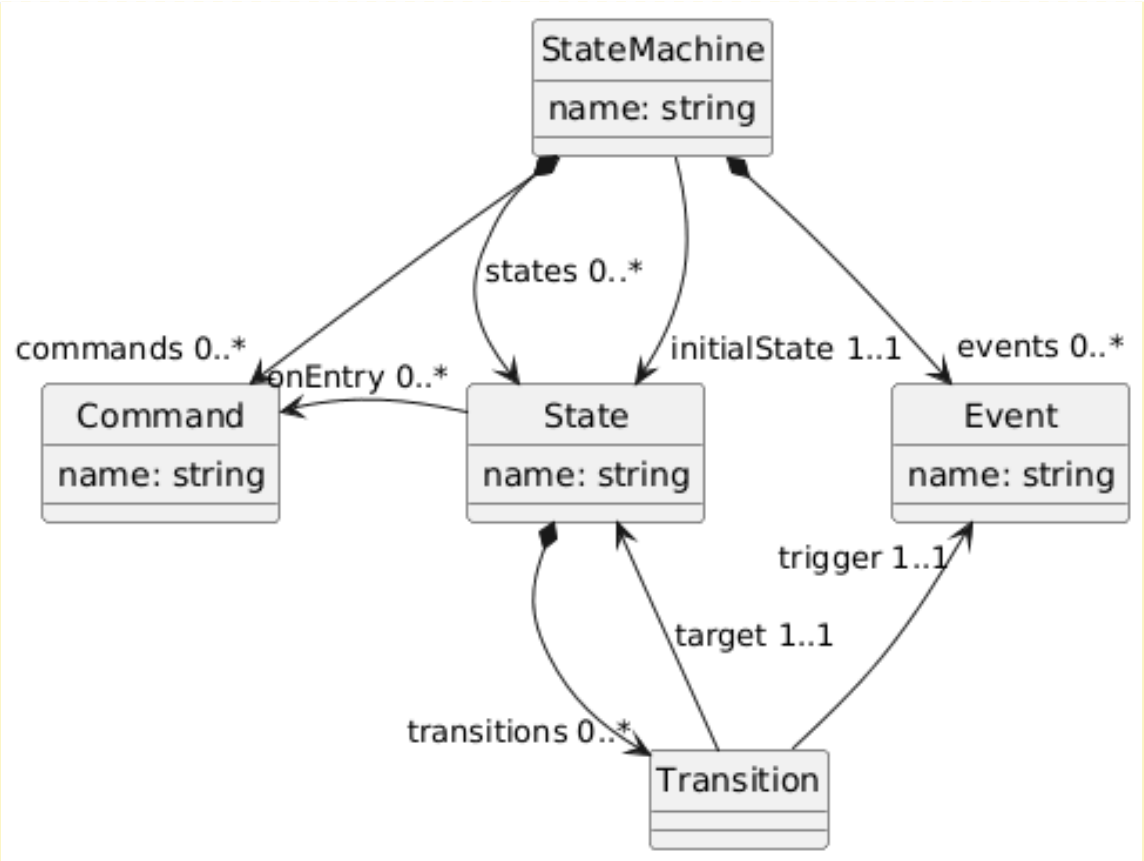
```
places [
    p1: 1
    p2: 0
    p3: 1
]
p1 -> t1 -> p2
p2,p3 -> t2
```

All these solutions can work, there is no bad answer *per se*! The "good choice" will depends on requirements, context and and user feedback.

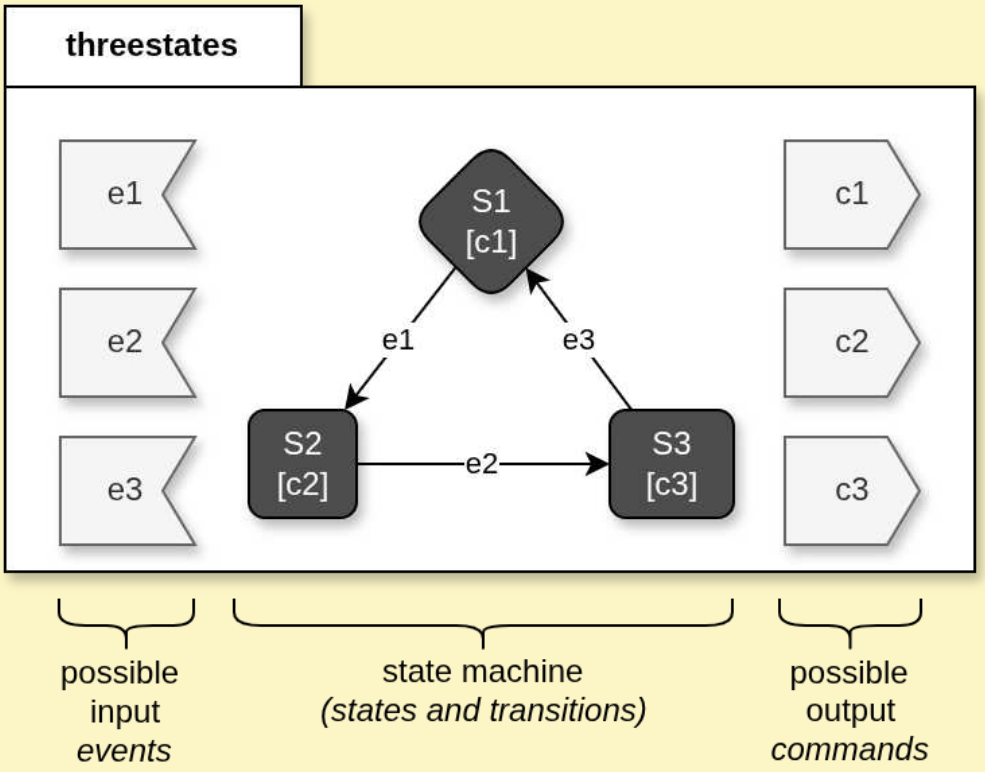
Example 21. Designing a concrete syntax for a State Machines DSL using annotated mockups

We would like to design the textual concrete syntax of the States Machines DSL that we began in the previous chapter.

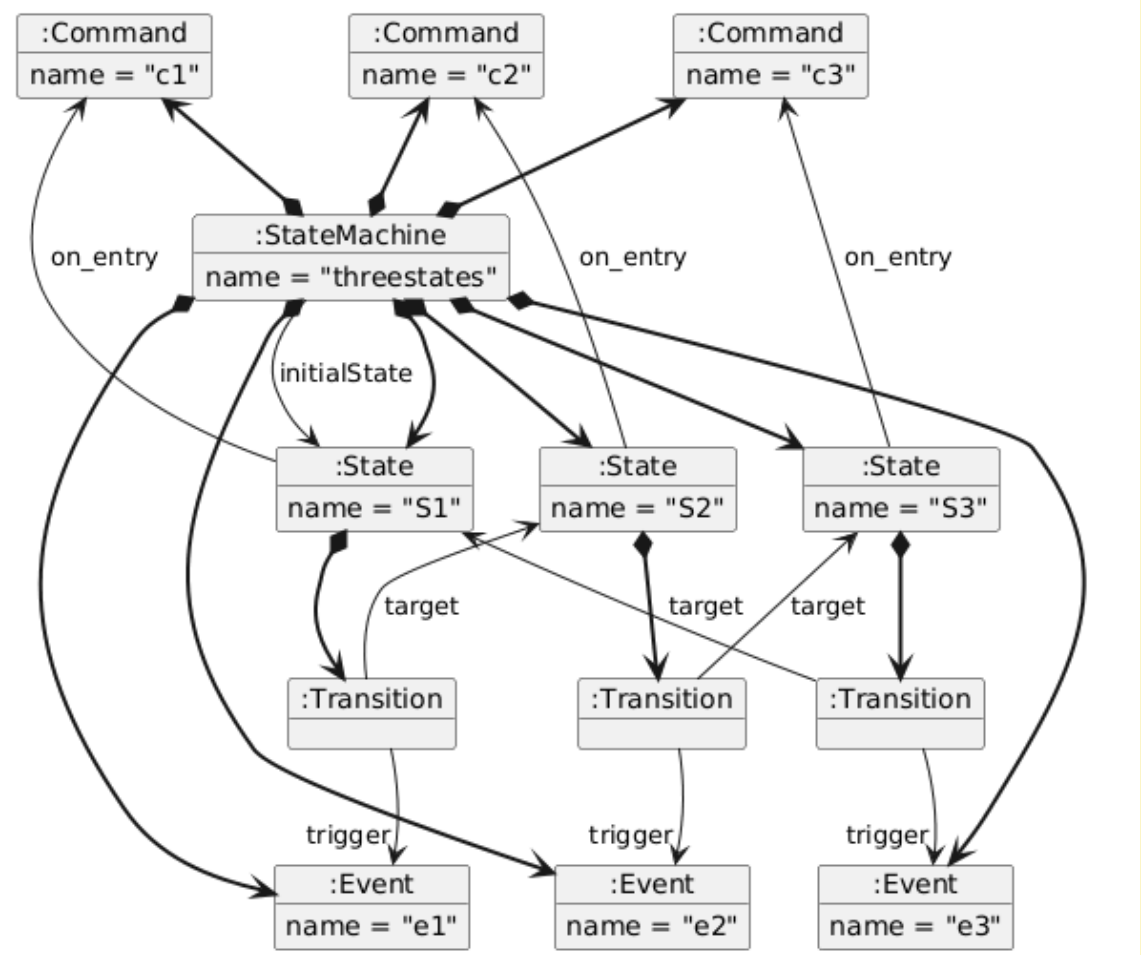
As a reminder, we designed the following abstract syntax:



Using this metamodel, we are able to use it to create an AST for the following example model:



which results in the following AST:



What textual concrete syntax do we want for this DSL? After discussions with (imaginary) stakeholders, we gather the following useful information:

- they prefer lowcase keywords,
- they are not programmers and thus are not used to using braces {},
- they are used to graphical notations such as symbols and arrows, but do not like abbreviations in words,
- they would like too be able to write comments in the code.

We make the following choices: - use very clear, unabbreviated lowcase keywords to declare each part of a program: `statemachine`, `events`, `state`, `transitions`, etc. - use the keyword `end` as a delimiter when appropriate, to avoid braces, - use a rather visual arrow notation ⇒ for declaring transitions, - use the traditional `//` notation to declare single-line comments.

We end-up with the following **mockup** textual representation for the AST shown above:

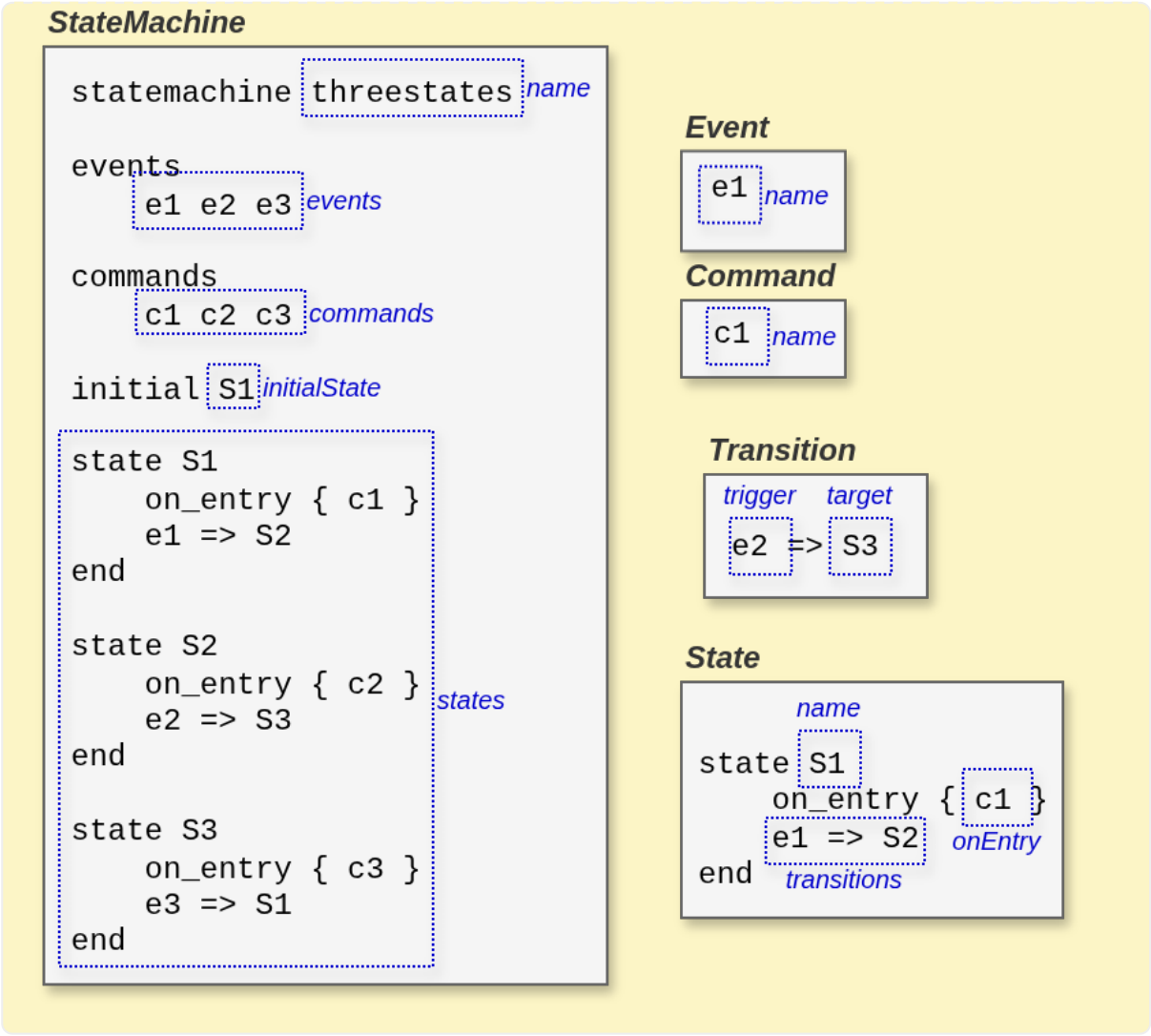
```
1 statemachine threestates
2
3 events:
4     e1, e2, e3
5
6 commands:
7     c1, c2, c3
8
9 initial: S1
```

```
10
11 state S1:
12     on_entry: c1
13     e1 => S2
14 end
15
16 state S2:
17     on_entry: c2
18     e2 => S3
19 end
20
21 state S3:
22     on_entry: c3
23     e3 => S1
24 end
```

In this mockup, we distinguish the following tokens and types of tokens:

- **Keywords**: `statemachine`, `events`, `commands`, `initial`, `state`, `on_entry`, `end`
- **Punctuation**:
  - *operators*: `=>`
  - *delimiters*: N/A
  - *separators*: `:`
- **Identifiers**: `threestates`, `e1`, `e2`, `e3`, `c1`, `c2`, `c3`, `S1`, `S2`, `S3`
- **Comments**: `//`
- **Literals**: N/A

Finally, we map this proposed concrete syntax with the abstract syntax by preparing the following **annotated mockups**:



5.2. Implementing a textual concrete syntax in Langium

In this section we will see how to implement a concrete syntax design into a usable **parser** using Langium.

5.2.1. Implementing the concrete syntax using grammar rules

A Langium concrete syntax is implemented in the form of a *grammar* and *grammar rules*. When a grammar is "executed", it transforms the text of an input program (provided as a file) into a structured AST (in memory) of the same program .

First, a **grammar** is declared with the keyword `grammar`:

```
grammar MyDSL
```

Another `.langium` file can be **imported** using the `import` keyword, without writing the `.langium` extension of said file:

```
import './otherfile' // this imports the file 'otherfile.langium'
```



This can for instance be used to import the abstract syntax defined in a separate file.

A **grammar rule** can be understood as a function that transforms a *specific piece of text of the input program* into *a node of the output* AST. This node is an instance of a concept of the abstract syntax (specified using the `returns` keyword).

Abstract syntax

```
interface MyConcept { ⋯ }
```

Concrete syntax

```
// Parser rule that produces an instance of 'MyConcept'
MyConceptRule returns MyConcept:
    ⋯
;

// Terminal rule that recognizes a valid identifier, and outputs it as a string
terminal ID returns string:
    /[_a-zA-Z][\w_]*/
;
```

There are three sorts of grammar rules:

- A **parser rule** defines a *sequence of tokens* that is possible at this location of the input program. From an abstract syntax point of view, executing a parser rule generally produces an instance of a *concept*.
- A **terminal rule** is declared with the `terminal` keyword, and contains a regular expression. It defines a possible kind of *lexical token* that can be parsed using the grammar, such as an *identifier*, an *arbitrary string*, an *integer*, etc. From an abstract syntax point of view, executing a terminal rule produces the value of a *primitive property*.
- A **hidden terminal rule** is declared with the `hidden terminal` keywords, and also contains a regular expression. However, instead of defining lexical tokens, it defines which sequences of characters that must be be *ignored* during parsing. This is typically used to ignore whitespaces and comments.

Langium default terminal rules

By default, a Langium project comes with the following set of pre-defined terminal rules:

```
hidden terminal WS: /\s+/;
terminal ID: /[_a-zA-Z][\w_]*/;
terminal INT returns number: /[0-9]+/;
terminal STRING: /"(\\.|[^"\\])*"|'(\\.|[^'\\])*'/;

hidden terminal ML_COMMENT: /\/\*[\s\S]*?\*\//;
hidden terminal SL_COMMENT: /\/\/[^\n\r]*/;
```

Each of these terminal rules can be used to either match or ignore different sequences of

characters:

- `WS` is a *hidden* terminal rule that matches (and therefore ignore) **whitespaces**, including tabulations and newline characters. This means the grammar will ignore all whitespaces, which means any amounts of whitespaces are allowed between two tokens.
- `ID` is a terminal rule that matches an **identifier** (examples: `List`, `myVariable`, `_building5`), and implicitly produces a _string_ primitive value. This rule is almost always used to parse the `name` attribute of a concept, and to define a *cross-reference* to an existing element.
- `INT` is a terminal rule that matches an **integer** (examples: `15`, `896`, `0`) and produces a *number* primitive value.
- `STRING` is a terminal rule that matches a **string** enclosed either between quotations marks `"` or single quotations marks `'` (examples: `"Good morning"`, `"hello"`, `'the car is red'`), and implicitly produces a string primitive value. It allows whitespaces between characters.



Be careful not to mix `ID` and `STRING`! `ID` does not allow whitespaces and cannot contain quotation marks, while `STRING` can contain whitespaces and must be enclosed in quotation marks.

- `ML_COMMENT` is a *hidden* terminal rule that matches (and therefore ignore) multi-line comments enclosed between `/*` and `*/`.
- `SL_COMMENT` is a *hidden* terminal rule that matches (and therefore ignore) single-line comments enclosed between `//` and a newline character.



You can customize the comments delimiters of your language by changing the `ML_COMMENT` and/or the `SL_COMMENT` rules. For example for Python-style comments (using `#`), you can rewrite `SL_COMMENT` this way:

```
hidden terminal SL_COMMENT: /#[^\n\r]*/;
```



Most of the time, we can simply reuse the *default terminal rules* generated in a new Langium project.

A **token** declared in a *parser rule* can be defined in two ways:

Abstract syntax

```
interface MyConcept {
    name: string
}
```

Concrete syntax

```
// Parser rule that produces an instance of 'MyConcept'
MyConceptRule returns MyConcept:
    'somekeyword' name=ID ';'
;
```

- A token in single quotes means that this exact string should be found at this specific location in the program. This is mostly used for definin *keywords* and *punctuation*. For example, here `somekeyword` is a keyword and `;` is punctuation, and mean that `MyConceptRule` always starts with `somekeyword` and ends with `;`.
- A call to an existing grammar rule, which means that this other grammar rule should apply at this location of the program. In addition, calling another grammar rule requires *assigning* (`=`) the output of the called rule into a containment property of the output instance. For example here the rule `ID` is expected after `somekeyword`, and the output of the rule `ID` is stored in the property `name` of the output `MyConcept` instance.



By default Langium considers that there can be an arbitrary amount of whites spaces between two tokens.

It is possible to define **repeated tokens** in a parser rule:

Abstract syntax

```
interface MyConcept {
    name: string
    children: OtherConcept[]
}

interface OtherConcept {}
```

Concrete syntax

```
MyConceptRule returns MyConcept:
    'somekeyword' name=ID (children+=OtherConceptRule)* ';'
;

OtherConceptRule returns OtherConcept: ··· ;
```

- A *repetition* of a group of tokens is declared with parentheses `()` followed by either a star `\*` or a plus `` `. This means that, at this location of the program, the enclosed tokens must be found _zero, one or multiple times_ for a `*`, and _one or multiple times_ for a ` ``.
- An assignment in a repetition must be written as `+=`, and must mandatorily target an *containment array property* of the returned concept. For instance here `children` is a containment array property, therefore we can have a repetition that adds multiple `OtherConcept` instances in the property using the `OtherConceptRule`.

It is possible to define **optional tokens** in a parser rule:

Abstract syntax

```
interface MyConcept {
    name: string
    isActive: boolean
}
```

Concrete syntax

```
MyConceptRule returns MyConcept:
    'somekeyword' (isActive?='active')? name=ID ';'
;
```

- A optional group of tokens is declared with parentheses `()` followed by a question mark `?`. This means that the enclosed tokens can be found *either zero or one* time at this location.
- The presence of an optional token can be stored in a boolean property of the concept using the `=?` operator. For instance here `isActive` is a boolean property which is `true` if the keyword `active` is found after the keyword `somekeyword`, and false otherwise.

It is possible to define **alternatives** in a group of tokens:

Abstract syntax

```
interface MyConcept {
    name: string
    isActive: boolean
    others: OtherConcept[]
    anothers: AnotherConcept[]
}

interface OtherConcept {}

interface AnotherConcept {}
```

Concrete syntax

```
MyConceptRule returns MyConcept:
    'somekeyword' ( others+=OtherConceptRule | anothers+=AnotherConceptRule )* ';'
;

OtherConceptRule returns OtherConcept: ··· ;

AnotherConceptRule returns AnotherConcept: ··· ;
```

- Inside any group declared with parenthese `()`, the pipe operator `|` can be used to declare mutually exclusive sequences of tokens. This can be combined with a repetition, meaning that each iteration of the repetition can have multiple possibilities. For example here the repetition will use either `OtherConceptRule` or `AnotherConceptRule` at each iteration, storing the result in `others` or `anothers` respectively.

It is possible to define a specific kind of token corresponding to a **cross-reference** of the abstract syntax:

Abstract syntax

```
interface MyConcept {
    name: string
    existingOther: @OtherConcept
}
```

```
interface OtherConcept {
    name: string
}
```

Concrete syntax

```
MyConceptRule returns MyConcept:
    'somekeyword' name=ID existingOther=[OtherConcept:ID] ';'
;

OtherConceptRule returns OtherConcept: ⋯
    'otherkeyword' name=ID
;
```

- A special cross-reference token can be declared with brackets with the following syntax `<cross-reference>=[<concept>:<terminal rule>]`:
  - `<cross-reference>` is the name of a cross-reference property
  - `<concept>` is the name of the concept (and **not the parser rule!**) referenced by the cross-reference property
  - `<terminal rule>` is the name of the terminal rule that is used to obtain the `name` attribute of the referenced concept.

For instance here the `name` of an `OtherConcept` is obtained using the `ID` terminal rule. Accordingly, the value of the `existingOther` cross-reference is an instance of `OtherConcept` whose name is written as an `ID`.



Writing *parser rules* can be a difficult task when writing a Langium grammar – and we only cover here a small subset of possibilities! If you want to learn more, the corresponding part in the Langium documentation [https://langium.org/docs/reference/grammar-language/#parser-rules] is very complete.

Finally, each Langium grammar *must* have a unique **entry parser rule**, which is a grammar rule declared with the keyword `entry`:

```
entry MyConceptRule returns MyConcept: ⋯  ;
```

An *entry rule* is the first rule executed when parsing a file with the grammar. It is comparable to the `main` function of a program.

Example 22. Langium concrete syntax of a State Machines DSL

We continue the State Machines DSL started previously, and for which we stored the abstract syntax in a file named `statemachines-dsl.abstractsyntax.langium`, with the following contents:

```
1 interface StateMachine {
2     name: string
3     commands: Command[]
4     states: State[]
```

```
    events: Event[]
    initialState: @State
}

interface Command {
    name: string
}

interface State {
    name: string
    transitions: Transition[]
    on_entry: @Command[]
}

interface Event {
    name: string
}

interface Transition {
    trigger: @Event
    target: @State
}
```

We would like to be able to write a state machine program in the following way:

```
statemachine threestates

events:
    e1, e2, e3

commands:
    c1, c2, c3

initial: S1

state S1:
    on_entry: c1
    e1 => S2
end

state S2:
    on_entry: c2
    e2 => S3
end

state S3:
    on_entry: c3
    e3 => S1
end
```

We therefore write the following grammar (which, as a reminder, relies on the concepts of the previously defined abstract syntax):

```
grammar StateMachinesDsl

import './state-machines-dsl.abstractsyntax'

entry StateMachineRule returns StateMachine:
    'statemachine' name=ID
    ('events' ':' events+=EventRule (',' events+=EventRule)*)?
    ('commands' ':'  commands+=CommandRule  (',' commands+=CommandRule)*)?
    'initial' ':' initialState=[State:ID]
    states+=StateRule*;

EventRule returns Event:
    name=ID;

CommandRule returns Command:
    name=ID;

StateRule returns State:
    'state' name=ID ':'
        ('on_entry' ':' on_entry+=[Command:ID] (',' on_entry+=[Command:ID])* )?
        transitions+=TransitionRule*
    'end';

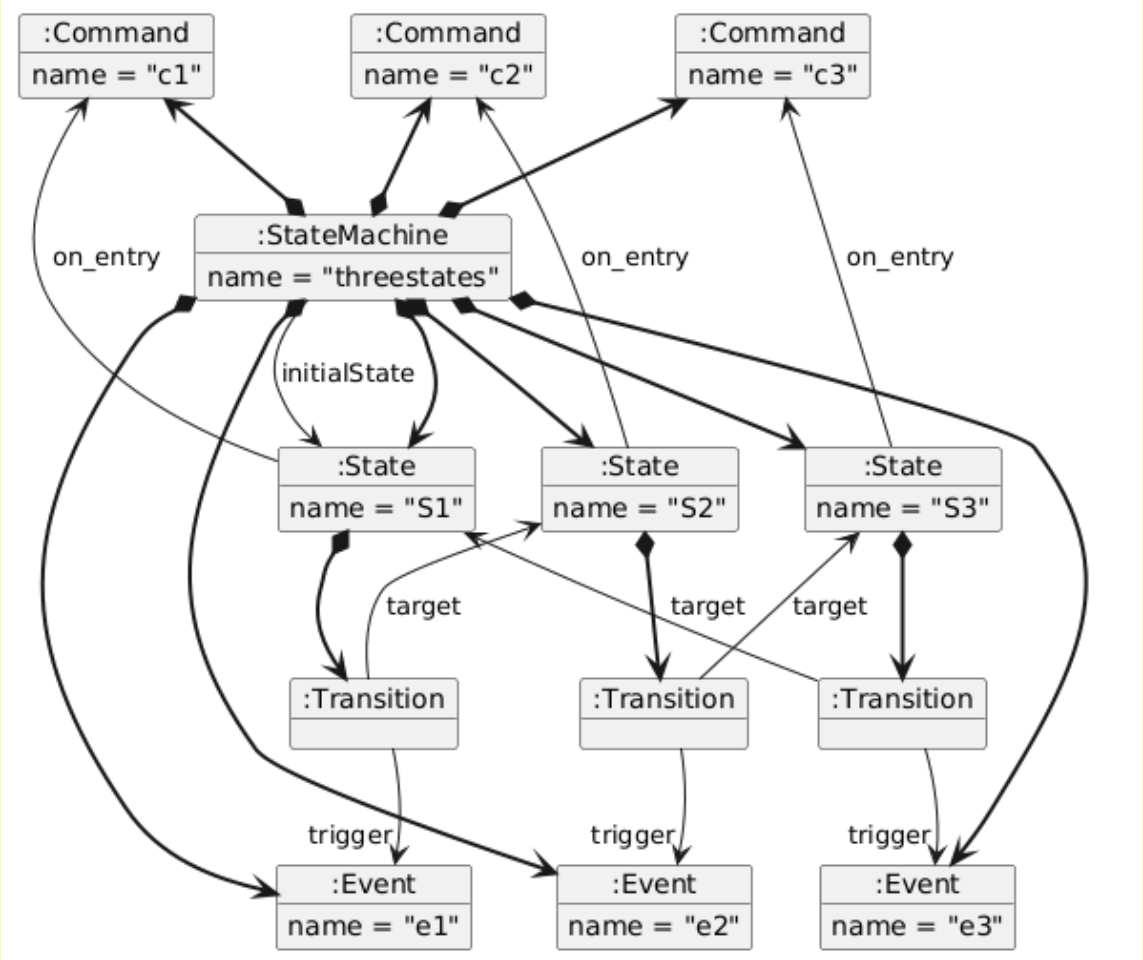
TransitionRule returns Transition:
    trigger=[Event:ID] '=>' target=[State:ID];


///////// Primitive rules //////////

hidden terminal WS: /\s+/;
terminal ID returns string: /[_a-zA-Z][\w_]*/;
terminal INT returns number: /[0-9]+/;
terminal STRING: /"(\\.|[^"\\])*"|'(\\.|[^'\\])*'/;

hidden terminal ML_COMMENT: /\/\*[\s\S]*?\*\//;
hidden terminal SL_COMMENT: /\/\/[^\n\r]*/;
```

Using the parser generated from this grammar, we can parse the example State Machines DSL program shown just before, which results in the following AST:



Automatic inference of the abstract syntax

Langium offers the possibility to **automatically infer the abstract syntax from the concrete syntax**, which can reduce the amount of lines of code in a Langium grammar. For example, this Langium grammar file:

```
// Abtract syntax
interface ConceptA {
    name: string
}

// Concrete syntax
ConceptARule returns ConceptA:
    name=ID;
;
```

can be rewritten like this:

```
// Concrete syntax + automatic inference of the abstract syntax
ConceptA:
    name=ID;
;
```

However, because this method "hides" the underlying abstract syntax definition, **we prefer in this chapter to *always separate* the abstract and concret syntaxes implementations**.



Most Langium examples online rely on automatic inference.

5.2.2. Fixing ambiguity and left-recursion problems

A common difficulty when writing a grammar is to avoid having **ambiguous** rules. Ambiguity arises when, for the same input program, multiple *different* ASTs can be produced by the parser, depending on the order grammar rules are called.

In most simple cases, a common way to fix an ambiguity problem is through the clever use of *keywords* and *punctuation* in the grammar, to make sure each case in a program can only correspond to a single possible grammar rule.

Example 23. Fixing a simple ambiguity in a grammar

Consider the following abstract syntax implemented in Langium:

```
interface Staff {
    persons: Person[]
}

interface Person {
    name: string
}

interface Student extends Person {}

interface Teacher extends Person {}
```

We prepare the following concrete syntax as a Langium grammar:

```
entry StaffRule returns Staff:
    (persons+=StudentRule | persons+=TeacherRule)*;

StudentRule returns Student:
    name=ID;

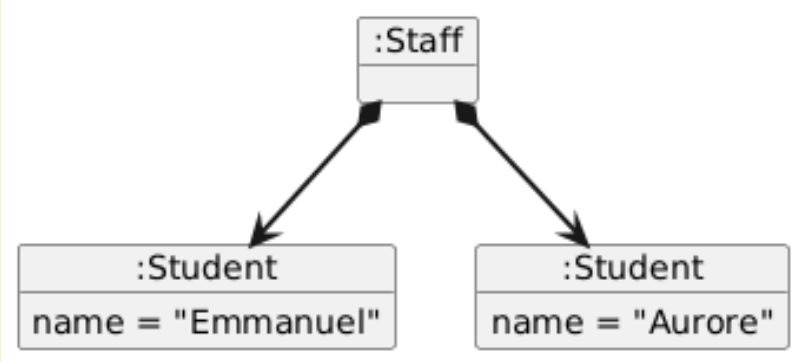
TeacherRule returns Teacher:
    name=ID;
```

We build the Langium project, and we open an editor where we write this program:

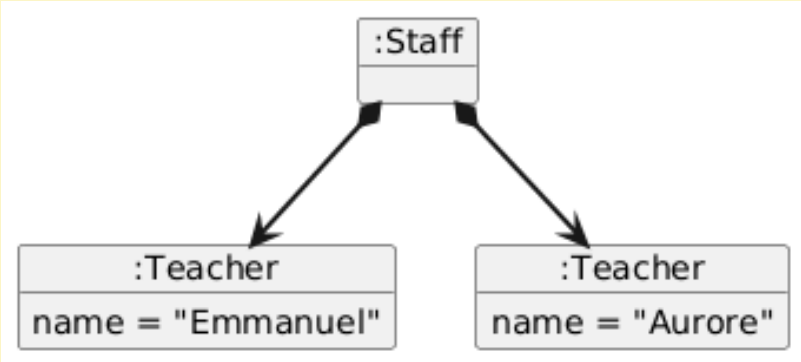
```
Emmanuel
Aurore
```

What will the AST produced by the parser look like? For each line of the program, there is no way for the parser to know whether it should apply the `StudentRule` or the `TeacherRule`. This

means the following AST is a possible valid result:



But *also* the following AST:



This means the grammar is **ambiguous**.

A simple solution to this problem is to disambiguate using dedicated keywords, like so:

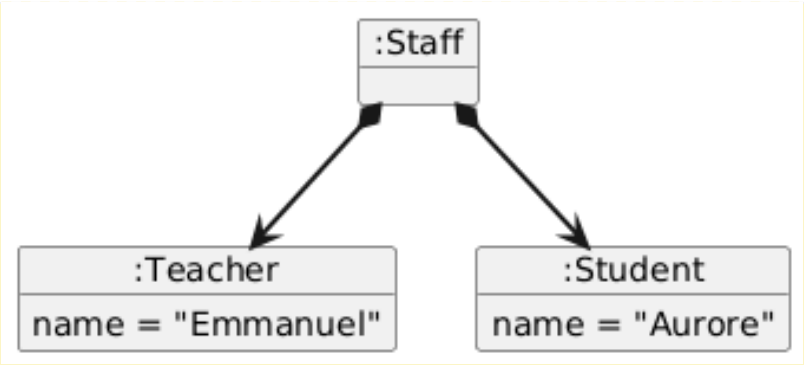
```
StudentRule returns Student:
    'student' name=ID;

TeacherRule returns Teacher:
    'teacher' name=ID;
```

Then we can write this program:

```
teacher Emmanuel
student Aurore
```

Which can only be parsed into the following AST:



While most ambiguity problems can be solved by adding keywords and/or punctuations in the rules, one particular type of ambiguity problem is called the **left-recursion problem**. We say that a grammar is *left-recursive* when a grammar rule will call itself (directly or indirectly) in the leftmost part of the rule. Not only this leads to many different ways to parse the same program (hence *ambiguity*), but it can also lead to *infinite recursion* in the parsing algorithm.



Langium will automatically detect a left-recursion problem in a grammar and trigger a **build error**.

The most common cases of left-recursion happen when a language provides ***infix operators*** (such as `+` and `*` in an expression such as `2+7*3`). An *infix operator* is an notation where the operands of an operator are placed on the left and on the right of the operator. Different infix operators usually have different *priorities* (such as multiplication has higher priority than addition).

Example 24. A left-recursive grammar for typical arithmetic expressions

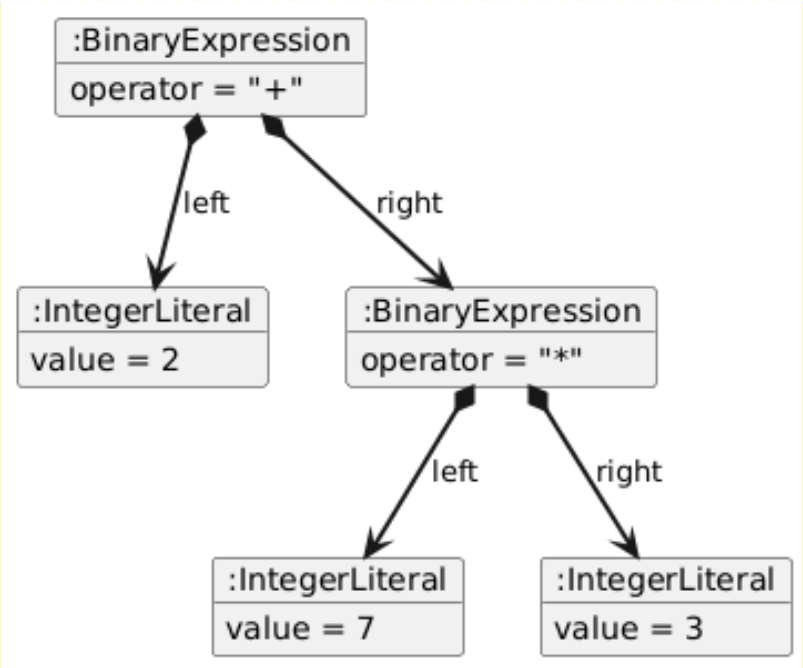
Consider the following abstract syntax implemented in Langium, which allows representing ASTs with binary expressions which are themselves composed of expressions:

```
interface Expression {}

interface BinaryExpression extends Expression {
    left: Expression
    operator: string
    right: Expression
}

interface IntegerLiteral extends Expression {
    value: number
}
```

We would like a grammar that, given the textual representation `2+7*3`, gives us the following AST (where the higher priority of `*` is taken into account):



To do so, we try writing the following (*wrong*) Langium grammar:

```
entry ExpressionRule returns Expression:
    BinaryExpressionRule | IntegerLiteralRule
;

BinaryExpressionRule returns BinaryExpression:
    left=ExpressionRule operator=('+' | '*') right=ExpressionRule
;

IntegerLiteralRule returns IntegerLiteral:
    value=INT
;
```

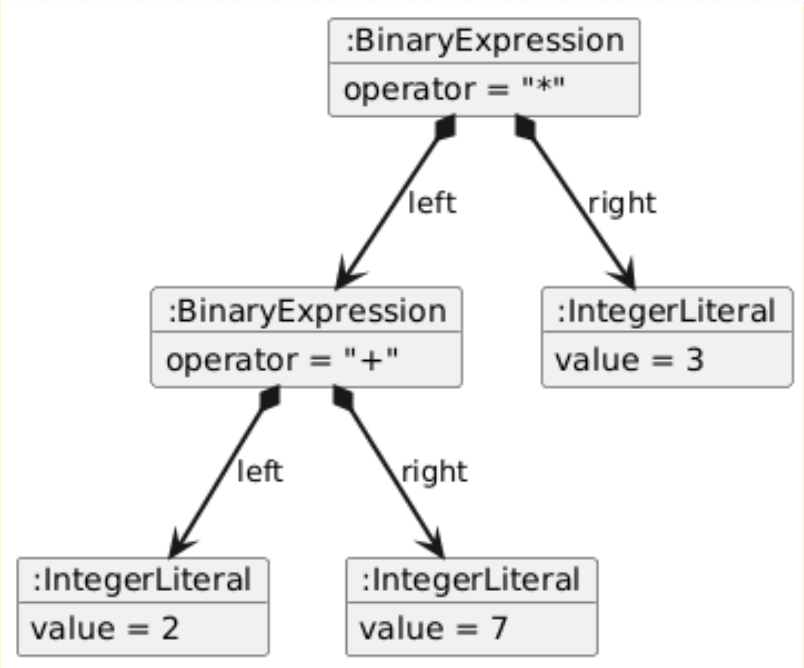
Unfortunately this grammar is left-recursive: when we apply `ExpressionRule`, then `BinaryExpressionRule`, we end-up with applying again `ExpressionRule` at the left-most part of our computation:

ExpressionRule

→ *BinaryExpressionRule*

→ **ExpressionRule** + *ExpressionRule*

Moreover, the grammar does not provide any way to ensure the multiplication (*) has a higher priority than the addition (+). This means if this grammar was applicable in Langium, we *could* obtain a wrong AST for the textual representation `2+7*3` such as:



Fixing a left-recursion problem is called **left-factoring**, and consists in avoiding having a single grammar rule for all operators, and instead to have a "hierarchy" of grammar rules that each depend on the next one in a strict sequence. To do so, each grammar rule is dedicated at parsing all occurrences of a single type of operator (for example all additions), and then explicitely calls the (similar) grammar rule of the operator that has higher priority, and so on.

Unfortunately in Langium this is quite verbose, and this requires an advanced feature of Langium called *tree-rewriting actions*, written using braces `{}`, which define explicit action that must occur when building the AST during the parsing.



It is not required to know this method, we only show it to give an idea of the general principle before showing the better way to solve the problem using an `infix` rule.

Example 25. Left-factoring a left-recursive grammar (the old-fashioned way)

Consider the (wrong) left-recursive grammar prepared in the last example.

```
entry ExpressionRule returns Expression:
    BinaryExpressionRule | IntegerLiteralRule
;

BinaryExpressionRule returns BinaryExpression:
    left=ExpressionRule operator=('+' | '*') right=ExpressionRule
;

IntegerLiteralRule returns IntegerLiteral:
    value=INT
;
```

Left-factoring this grammar would give us the following (good) grammar, which would not be left-recursive anymore:

```
entry ExpressionRule returns Expression:
    AdditionRule
;

AdditionRule returns Expression:
    MultiplicationRule
    ({BinaryExpression.left=current} operator="+" right=MultiplicationRule)*
;

MultiplicationRule returns Expression:
    IntegerLiteralRule
    ({BinaryExpression.left=current} operator="*" right=IntegerLiteralRule)*
;

IntegerLiteralRule returns IntegerLiteral:
    value=INT
;
```

With this fixed grammar, given a textual representation such as `2+7*3`, we would apply without any ambiguity the grammar rules in the following way:

ExpressionRule

→ *AdditionRule*

→ *MultiplicationRule* + *MultiplicationRule*

→ (*IntegerLiteralRule*) + (*IntegerLiteralRule* * *IntegerLiteralRule*)

Twice, we use the *tree-rewriting action* `{BinaryExpression.left=current}` to make sure we create a `BinaryExpression` node in the AST when an operator is encountered, and we assign to the `left` attribute the node built before encountering the operator.

Fortunately, Langium provides a much easier solution for left-factoring with a dedicated type of grammar rule called the `infix` grammar rule. An `infix` grammar rule is a special kind of grammar rule that only requires an ordered list of operators (from the highest priority to the lowest), and a complementary normal grammar rule that defines that is the simplest type of element we can find in an expression (for example, an integer).



An `infix` grammar rule must return an AST node that *must* have the attributes `left`, `operator` and `right`.

Example 26. Left-factoring a left-recursive grammar (using a Langium `infix` grammar rule)

Consider again the (wrong) left-recursive grammar prepared in the last example.

```
entry ExpressionRule returns Expression:
    BinaryExpressionRule | IntegerLiteralRule
;

BinaryExpressionRule returns BinaryExpression:
    left=ExpressionRule operator=('+' | '*') right=ExpressionRule
```

```
;

IntegerLiteralRule returns IntegerLiteral:
    value=INT
;
```

We can use an `infix` grammar rule to left-factor this grammar:

```
entry ExpressionRule returns Expression:
    BinaryExpressionRule
;

infix BinaryExpressionRule on IntegerLiteralRule returns BinaryExpression:
    '*' > '+'
;

IntegerLiteralRule returns IntegerLiteral:
    value=INT
;
```

The `BinaryExpressionRule` is an `infix` grammar rule, which uses the rule `IntegerLiteralRule` as the simplest type of elements that can be found in an expression, and which has two operators `and +, with` the one with the highest priority.

Moreover, `BinaryExpressionRule` returns a `BinaryExpression`, which is correct because `BinaryExpression` was implemented earlier with the required three attributes: `left`, `operator` and `right`.

5.2.3. The Langium generator

Once a complete Langium grammar has been defined in the `packages/language/src` folder in the form of one or multiple `.langium` files, it is time to call the Langium generator to automatically produce data types, an editor and a parser for our software language.

As already explained in the chapter on Langium, to run the Langium generator, and to compile the generated Typescript code:

```
npm run langium:generate && npm run build
```

This mainly automatically produces the following artifacts:

- `packages/language/src/generated/ast.ts`: Typescript data types for all concepts of your abstract syntax, along with many utility functions and boilerplate code for an easy manipulation of your AST.
- `packages/language/src/generated/grammar.ts`: a JSON representation of your grammar, which is then automatically used to configure the Langium parser.
- `packages/language/src/generated/module.ts`: a Typescript module with metadata (such as the file extension) and pointers to the other language components.



You will never have to manually edit these files, as they are *entirely* derived from your Langium code.

5.3. Testing a generated editor of a Langium language

See the dedicated section in the chapter on Langium.

Chapter 6. 📚 Ch5. Semantics with code generation

The **semantics** of a software language defines the *meaning* and/or the *effect* of a program/model of said language. Without a semantics, a language is purely syntactic and can neither be used to document, structure, or perform anything tangible.

In this chapter we mostly focus on *code generation semantics*. We first review some generalities about the semantics of a software language, then we learn how to *design* a code generation semantics using reference example implementations, and finally we learn to *implement* the code generation semantics of a Langium DSL.



🚧 This chapter is still under construction.

6.1. Types of semantics

6.1.1. Executable vs. non-executable semantics

There are two broad categories of software languages: languages whose programs or models can be *executed*, and languages whose programs or models cannot (alone) be executed. We call *executable semantics* the semantics of an executable language, and *non-executable* semantics the semantics of a language that cannot be executed.

Executable semantics

An executable semantics of a software language describes what *happens* when a program or model of the language is *executed*. This means the execution of saif program has a *visible effect* on its environment.

The most common purpose of an executable semantics is to perform **computations**. A computation is defined by some *runtime state* in memory that changes over time (for example the *current statement* of a program, or the *current values* assigned to variables), and optionnaly by *side effects* that occur during the execution (for example creating/modifying a file, or sending messages over the network).

Example 27. Some executable languages for computations

- **Any *General-Purpose Programming Language* (GPL)** such as Java, C, C++ or Python, is an executable language made for computing. It provides the means to write algorithms, statements, conditionnals that will modify the current state of the program over time.
- **Logo [https://en.wikipedia.org/wiki/Logo_(programming_language)]** is an educational executable programming language where a program describes how a virtual "turtle" will move and draw a picture. The state of a Logo program is defined equally by the position of the Turtle, by the progress of the drawing, and by the current values of variables of the Logo program being executed.
- **UML state machines** [https://en.wikipedia.org/wiki/UML_state_machine] is an executable modeling language describing explicitly the different states that a system (or subsystem) can reach, under which conditions each state is reached, and what happens when the current state changes (possibly with side effects). Each computation step is the firing of one or multiple transitions when the required transitions are met, which changes the current state.
- **Petri nets** [https://en.wikipedia.org/wiki/Petri_net] is an executable modeling language describing concurrent systems in the form of places that exchange *tokens* at each execution step.

- **UML activity diagrams** [https://en.wikipedia.org/wiki/Activity_diagram] is also an executable modeling language that can be used to describe possible sequences of *actions* that a system or individual must perform. Its execution semantics is also based on tokens whose computations decide what will the next action be, and whether actions should be executed in parallel.

Another common purpose of an executable semantics is to create some form of **visualization**. In that case, executing the program or model will *display* some graphical or textual information, typically a document or an interactive media. An execution semantics for visualization describes how should some information visually appear and how it should be distributed in space.

Example 28. Some executable languages for visualizations

- The **Hypertext Markup Language (HTML)** **[https://en.wikipedia.org/wiki/HTML]** is by far the most famous and used software language for visualization. It provides concepts (such as *titles*, *divisions*, *tables*, *paragraphs*, etc) to structure and organize the visual contents of a page meant to be browsed. When an HTML document is executed by a web browser, a visual representation of the page is produced and displayed.
- The **Portable Document Format (PDF)** **[https://en.wikipedia.org/wiki/Portable_Document_Format]** is the second most famous executable software language for visualization. A PDF file is essentially composed of a sequence of instructions describing how to draw and write the contents of each page when the PDF is executed by a PDF viewer. Contrary to HTML, PDF is never meant to be directly written by humans, but is used instead as an export format or code generation target.
- **LaTeX** **[https://en.wikipedia.org/wiki/LaTeX]** is software language to write and compose complex documents, with a focus on scientific documents. LaTeX provides concepts to start/end a document, to declare chapters or sections, to emphasize text, or to declare a bibliography. When executed, a LaTeX program should produce a (mostly textual) read-only visualization of the encoded document, often in the form of a generated PDF.



An executable language can be *simultanously* made for computing and for visualization. Examples include Logo (cited above) or Processing [https://en.wikipedia.org/wiki/Processing].

In fact, many languages for visualization provide concepts and operators for *computing* parts to be displayed, such as PDF and LaTex.



An executable semantics can be implemented either as an AST *interpreter*, or as a **code generator**. We discuss both cases in a later section.

Non-executable semantics

It is also possible for a software language to have a *non-executable* semantics, and instead to be used only for *describing* an aspect of a system, or for capturing knowledge in a systematic way.

A typical purpose for such languages is to define a **structure** or **organisation** of a system. While models of such languages cannot be "executed", they can instead be used as a structural dependency for another model (for example a UML activity diagram can use the data types of a UML class diagram), or can be purely be used as a blueprint for the engineering of a system (for example a UML component diagam can be used as a blueprint for manually implementing a software system).

A non-executable semantics should not describe what *happens*, but should instead only describe precisely what the program or model *means* structurally and in the context of the considered domain.

Example 29. Some non-executable languages and their semantics

- **Cascading Style Sheets (CSS)** **[https://en.wikipedia.org/wiki/CSS]** is a software language to specify the appearance of a document written in HTML (or other compatible markup language). The core concept of CSS is the *rule*, itself composed of a *selector* declaring what elements of the document are concerned by the rule, and of a *block* describing what should the visual style of said elements. Alone, a CSS program cannot be executed; however an HTML document can import a CSS program as a dependency, which will influence how the web browser executes the HTML document.
- **UML class diagrams** **[https://en.wikipedia.org/wiki/Class_diagram]** is a modeling language that can be used to precisely define the object-oriented data structures of a software program. The concepts allow the declaration of classes, methods, attributes, and relations between classes, and rather precise semantics are provided for each concept in the UML standard. While one cannot *execute* a UML class diagram, it is possible to implement a code generator that would translate a UML class diagram into *class skeletons* that can be later completed by developpers.
- **Protobuff** **[https://en.wikipedia.org/wiki/Protocol_Buffers]** is a software language to describe serializable data types for network messaging. The core concept of the language is the *message* which is a data type composed of a set of *properties*. A Protobuff program cannot be executed *per se*, however the language comes with a set of code generators targeting a wide range of GPLs such as Java, Rust or Python. The code generated from a Protobuff program takes the form of one data type (for example a Java class) for each *message*, along with helper functions to build, serialize and deserialize messages.
- **Feature models** **[https://en.wikipedia.org/wiki/Feature_model]** is a modeling language to represent *product lines*, and particular Software Product Lines (SPLs) [https://en.wikipedia.org/wiki/Software_Product_Line]. The core concept of a feature model is the *feature*, which can then be decomposed in children features, and so on. *Constraints* define that features exclude or require other features, or that some features are optional or mandatory. While a feature model cannot be executed, it efficiently captures the set of all possible valid combinations of features of a given product line.



A non-executable semantics cannot be implemented as an AST *interpreter*, but can in some cases be implemented as a **code generator**. For example, a Protobuff program is not executable *per se*, but can be compiled into a Java library providing all the Java types and methods allowing to create the objects described in the Protobuff program.

6.1.2. AST interpreter vs. Code generator

We have seen in Chapter 2 that there are two main approaches to implement a semantics: either in the form of an AST *interpreter* (the language implementation is then called an *interpreter*), or in the form of of a *code generator* (the language implementation is then called a *compiler*).

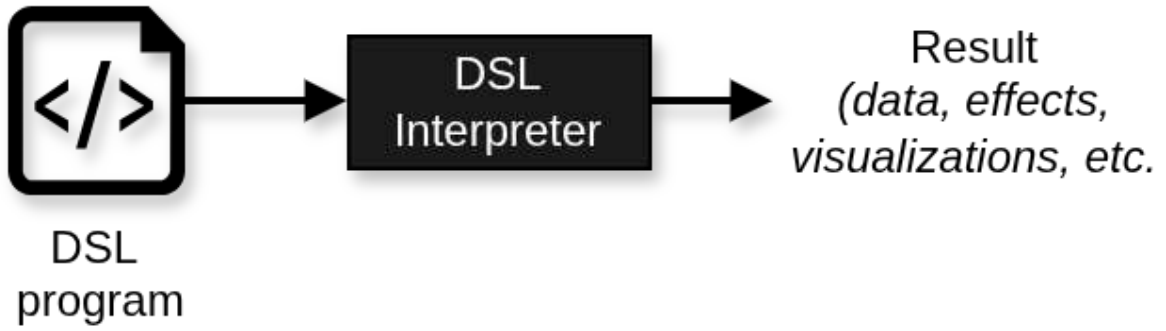


Figure 17. A DSL implemented as an interpreter, which can interpret a DSL program and directly produce a result

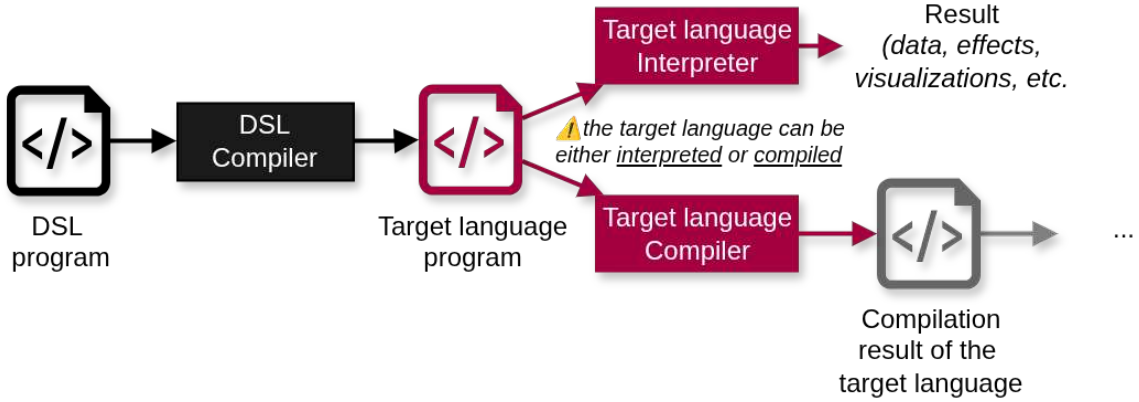


Figure 18. A DSL implemented as a compiler, which can translate a DSL program into code written in another target software language. This target language can itself be either interpreted or compiled.

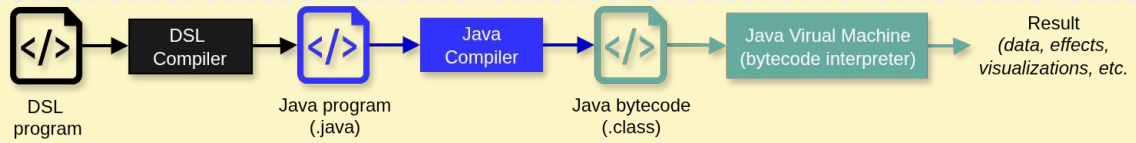
While **building an AST *interpreter* is usually considered the simplest and cheapest option** to implement the semantics of a DSL, it very often fails to address many important concerns, in particular:

- It is technically difficult to obtain good *performance* such as speed and low memory consumption.
- The programming language chosen to implement an AST interpreter will *constrain what software librairies and tools can be reused* in the interpreter's code.
- If the DSL is used in a context where it must be used jointly with more traditionnal GPL code, an *integration problem* can appear to allow a GPL program to call a DSL program (or the opposite).

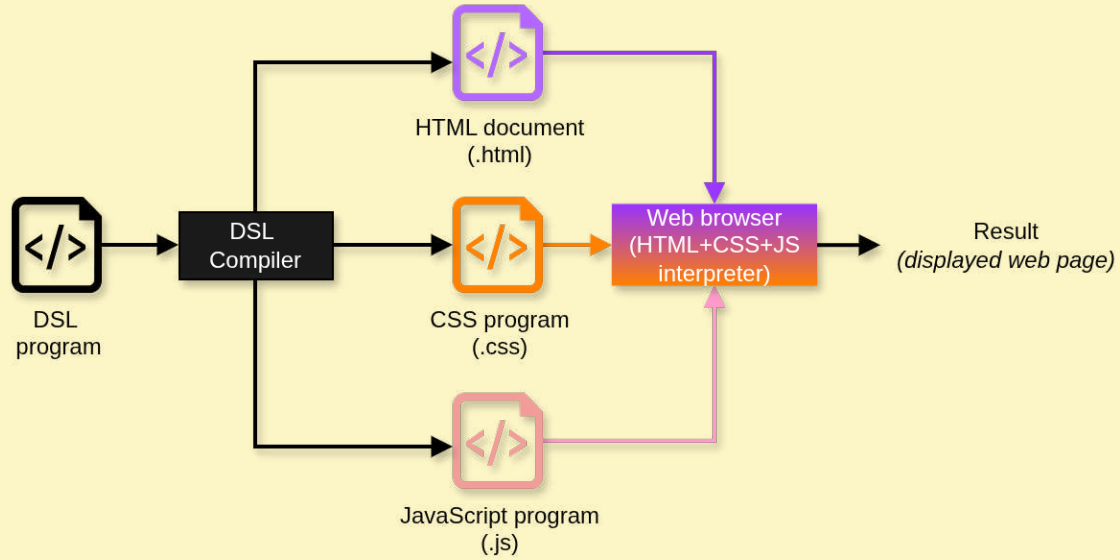
For this reason **we focus solely in this chapter on how to design and engineer a semantics in the form of a *code generator***, which allows the execution to be performed by lower-level code, and thus to rely on the performance of the target software language. This also allows generating code in a convenient target language, for example to be more easily integrated with manually written code.

Example 30. DSLs compiling to Java or to the web

We decide to implement the execution semantics of a DSL in the form of a *code generator* that generates **Java** code. Because Java is a compiled language, this Java code must then be compiled into Java bytecode, which can then be interpreted by a Java Virtual Machine (JVM). The resulting compilation process can be schematized as follows:



Alternatively, we decide to implement the execution semantics of a DSL in the form of a *code generator* that simultanously generates **HTML**, **CSS** and **Javascript** code. These software languages must then be *interpreted* within a web browser. The resulting compilation process can be schematized as follows:



6.2. Designing a code generation semantics

Writing code that writes code is not an easy task for the mind. Hence before diving into how to implement a code generator, a good first step is to *design* the envisionned code generation process. We will see how to do this with the help of reference *example implementations*, which can then be mapped to concepts of the abstract syntax.

6.2.1. Reference output implementation

Before implementing the code generator of a DSL, it is important to have a very clear idea of *what code* should be produced from a given input DSL program, and how this code should be organized. As everything else, what this code should look like depends heavily on the context and on the purpose of the DSL.

In SLE, we call **reference output implementation** a concrete example of what the code generator should produce given an example of DSL program. A reference output implementation must be written *manually* and its contents should only be based on what the information available in the input DSL program.

Example 31. Reference output implementation of a tiny programming DSL program

C generator TODO

Example 32. Reference output implementation of a Petri nets DSL program

TODO

6.2.2. Mapping a reference example with an abtract syntax

TODO

TODO parts of the code are static and parts are dynamic

Example 33. Abstract syntax mapping of a reference example implementation of a tiny programming DSL program

TODO

Example 34. Abstract syntax mapping of a reference example implementation of a Petri nets DSL program

TODO

6.2.3. Good practices

TODO

6.3. Implementation of a code generation semantics

TODO

Using the Langium Grammar Language and the Langium generator, we can efficiently obtain software components that implement both the *abstract syntax* and the *concrete syntax* of a software language. But how can we implement the **semantics** of a Langium-based software language?

6.3.1. Visiting an AST

Visiting an AST using Langium is achieved using the TypeScript types generated from the abstract syntax. These types are found in the generated `ast.ts` file, an can be used in an object oriented fashion.

For example, given the following abstract syntax in a Langium grammar:

```
interface Formular {
    name: string
    fields: MultipleChoiceField[]
}

interface MultipleChoiceField {
    name: string
    choices: Choice[]
    default: @Choice
}
```

```
interface Choice {
    name: string[]
}
```

Then we call the Langium generator to produce an `ast.ts` file containing all the abstract syntax data types.

The `ast.ts` file is generated in the `language` package of the Langium project, and can therefore be imported like so:

```
import type { Formular, MultipleChoiceField, Choice }
                            from 'formular-language';
```

where:

- `Formular`, `MultipleChoiceField`, `Choice` are concepts of the abtract syntax that we need to import,
- `formular-language` is the name of the package for the DSL called `formular`.

Going back to our visitor, this `ast.ts` file can therefore be used in the following way:

```
// Import the abstract syntax data types
import type { Formular, MultipleChoiceField, Choice }
                            from 'formular-language';

// Define a function to visit a 'Formular' node
function visitFormular(formular: Formular) {
    console.log("Visiting a Formular with name: " + formularASTRoot.name)

    for (let multipleChoiceField of formular.fields) {
        visitMultipleChoiceField(multipleChoiceField)
    }
}

// Define a function to visit a 'MultipleChoiceField' node
function visitMultipleChoiceField(multipleChoiceField: MultipleChoiceField) {
    console.log("Visiting a MultipleChoiceField with name: "
                                            + multipleChoiceField.name)
    console.log("and with default choice: "
                                + multipleChoiceField.default.ref!.name)

    for (let choice of multipleChoiceField.choices) {
        visitChoice(choice)
    }
}

// Define a function to visit a 'Choice' node
function visitChoice(choice : Choice) {
    console.log("Visiting a Choice with name: " + choice.name)
    console.log("and contained in MultipleChoiceField: " + choice.$container.name)
}

// Obtain the root of the AST produced by the parser, a 'Formular' node
const formularASTRoot : Formular = ···
```

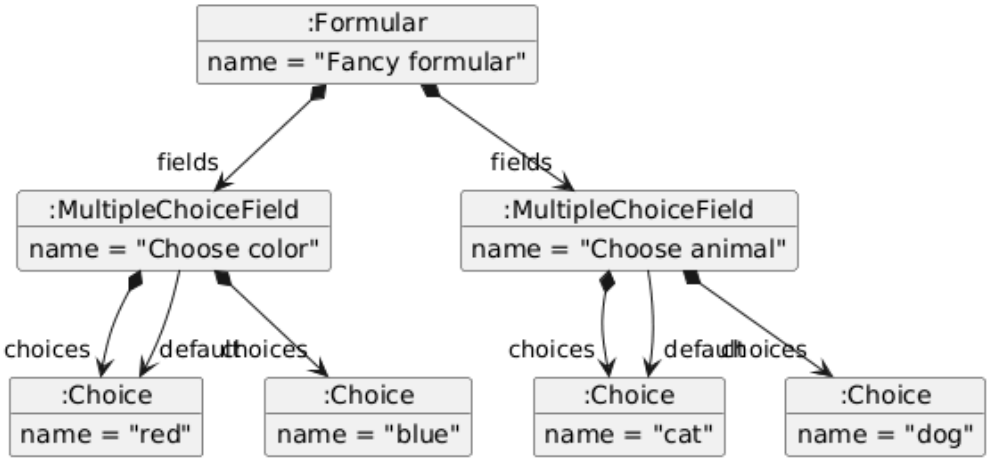
```
// Call visit function
visitFormular(formularASTRoot)
```

- `visitFormular` is a function that visits a `Formular` node, and that then trigger the visit of all enclosed `TextField` nodes using the `visitTextField` function,
- `visitMultipleChoiceField` is a function that visits a `MultipleChoiceField` node, that then trigger the visit of all enclosed `Choice` nodes using the `visitChoice` function,
  - in the expression `multipleChoiceField.defaultChoice.ref!.name`, the `ref` attribute is required to resolve a *cross-reference property* (here the `defaultChoice` cross-reference),
- `visitChoice` is a function that visits a `Choice` node,
  - in the expression `choice.$container.name` the `$container` attribute is used to access the parent node (here the enclosing `MultipleChoiceField`)



While we achieve a similar goal, we are *not* strictly speaking following the visitor design pattern [https://refactoring.guru/design-patterns/visitor], because the code here is not object-oriented, and because it mixes the algorithm that perform the navigation between node, and the code that performs actions when visiting a node.

Then if we call this piece of code on the following AST:



We obtain the following output:

```
Visiting a Formular with name: Fancy formular
Visiting a MultipleChoiceField with name: Choose color
and with default choice: red
Visiting a Choice with name: red
and contained in MultipleChoiceField: Choose color
Visiting a Choice with name: blue
and contained in MultipleChoiceField: Choose color
Visiting a MultipleChoiceField with name: Choose animal
and with default choice: cat
Visiting a Choice with name: cat
and contained in MultipleChoiceField: Choose animal
Visiting a Choice with name: dog
and contained in MultipleChoiceField: Choose animal
```

Example 35. AST visit for the State Machines DSL

We write the following sample visit code for the State Machines DSL:

```
import type { StateMachine, Command, Event, State, Transition }
                                from 'state-machines-language';

function visitStateMachine(stateMachine: StateMachine) {
    console.log("Visiting state machine: " + stateMachine.name)
    console.log("with initial state: " + stateMachine.initialState.ref!.name)

    for (let command of stateMachine.commands) {
        visitCommand(command)
    }

    for (let event of stateMachine.events) {
        visitEvent(event)
    }

    for (let state of stateMachine.states) {
        visitState(state)
    }

}
function visitCommand(command: Command) {
    console.log("Visiting command: " + command.name)
}

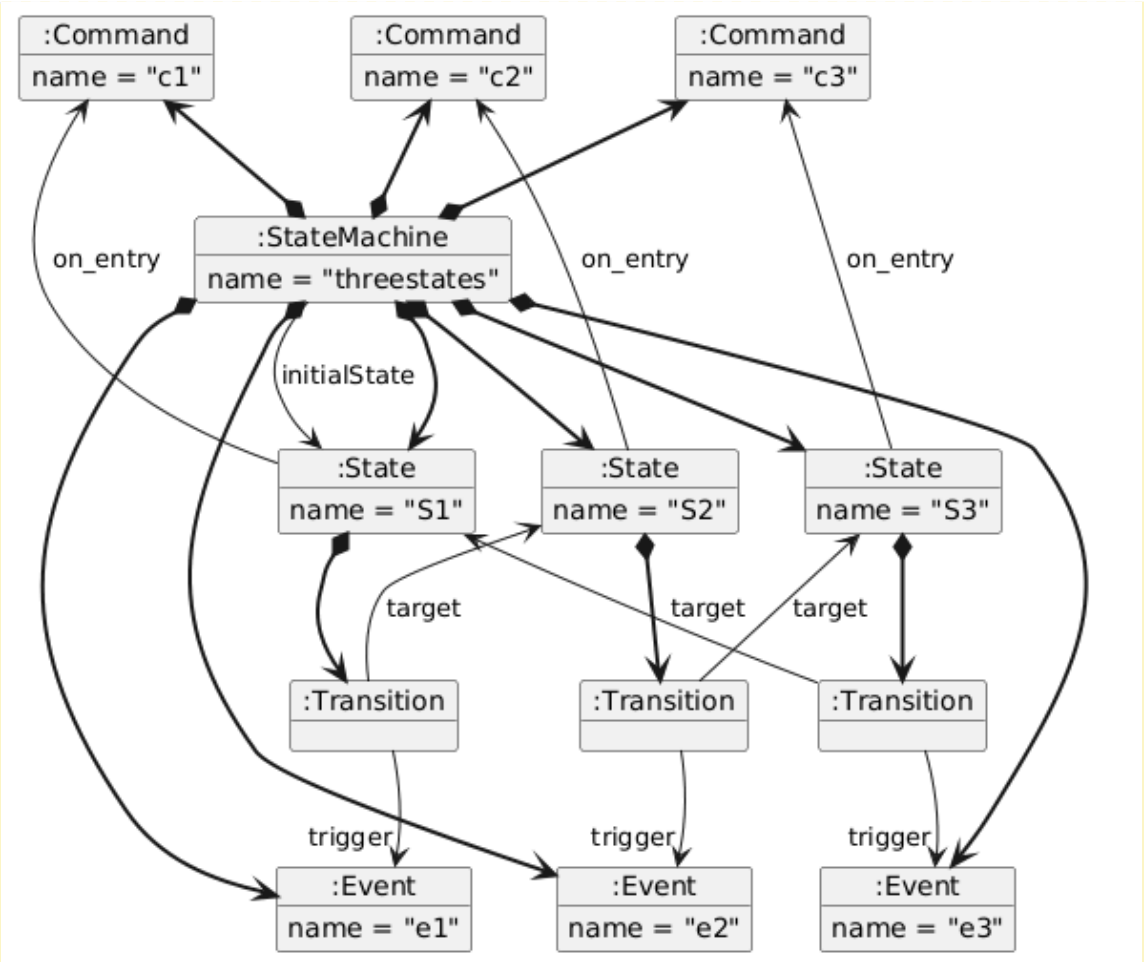
function visitEvent(event: Event) {
    console.log("Visiting event: " + event.name)
}

function visitState(state: State) {
    console.log("Visiting state: " + state.name)

    for (let transition of state.transitions) {
        visitTransition(transition)
    }
}

function visitTransition(transition: Transition) {
    console.log("Visiting transition contained in state: "
                                + transition.$container.name)
}
```

If we call `visitStateMachine` on the following AST (shown before):



Then we obtain the following output:

```
Visiting state machine: threestates
with initial state: S1
Visiting command: c1
Visiting command: c2
Visiting command: c3
Visiting event: e1
Visiting event: e2
Visiting event: e3
Visiting state: s1
Visiting transition contained in state: s1
Visiting state: s2
Visiting transition contained in state: s2
Visiting state: s3
Visiting transition contained in state: s3
```

6.3.2. Implementing a *code generator*

We have seen earlier that the semantics of a software language can be implemented either as a *code generator* or as an AST *interpreter*. In this section we focus on the case of **code generators**. More specifically, we consider the case of a code generator whose target language is a GPL such as Java or C, or a markup language such as HTML or Markdown.

A code generator must accomplish two main types of tasks:

- **Visiting the AST** of the program being compiled, which means going over all nodes of the AST. Programming this visit is achieved by the *abstract syntax data types* that are part of the language implementation (the `ast.ts` file in Langium).
- **Producing output code** by constructing a string containing the code to be generated. The contents of this output string depends on the contents discovered during the visit of the AST. Constructing such string is commonly achieved using *string templates*.

6.3.3. Implementing an AST interpreter

TODO

Producing output code using string templates

The goal when visiting an AST is to generate some output code. This requires producing a very large string using some form of string concatenation. Because string concatenation can be quite verbose and annoying to write (with lots of expressions like `"first" + "second"`), code generation is better achieved using *string templates*.

A **string template** is a string expression where it is possible to directly embed variables or function calls within a string. For example in TypeScript:

```
let name: string = "Georges"
let output: string = `Hello ${name}!`

console.log(output)
```

produces:

```
Hello Georges!
```

Here:

- `name` is a variable,
- `output` is a string we construct using a string template, which is declared using *backticks*. In the template we use the placeholder syntax `${⋯}` to write a valid string expression – here we simply use the available `name` variable.

Another benefit of string templates is that they **can be written on multiple lines**:

```
let name: string = "Georges"
let city: string = "Nantes"

let output: string = `
    Hello ${name}
    from ${city}!
`

console.log(output)
```

produces:

```
Hello Georges
from Nantes!
```

We can put any sort of expression in a placeholder `${⋯}`, including **function calls**, which enables code like this:

```
function generateHello(name: string): string {
    return `Hello ${name}`
}

function generateFrom(city: string): string {
    return `from ${city}`
}

let name: string = "Georges"
let city: string = "Nantes"

let output: string = `
    ${generateHello(name)}
    ${generateFrom(city)}!
`

console.log(output)
```

which produces:

```
Hello Georges
from Nantes!
```

Unfortunately, it is not possible to declare a `for` loop within a template in order to display a collection of values. However a good alternative is to combine on **the `map` and `join` operators** available in TypeScript. For example:

```
type Person = {
    name: string
    city: string
}

function generateHelloFromPerson(person: Person): string {
    return `
        Hello ${person.name}
        from ${person.city}!
    `
}

let manyPersons: Person[] = [
    { name: "Georges", city: "Nantes" },
    { name: "Julia", city: "Bordeaux" }
]

let output: string = `
```

```
    Let's welcome everyone:

    ${manyPersons.map(person => generateHelloFromPerson(person)).join("\n\n")}
    `

console.log(output)
```

which produces:

```
    Let's welcome everyone:

        Hello Georges
        from Nantes!

        Hello Julia
        from Bordeaux!
```

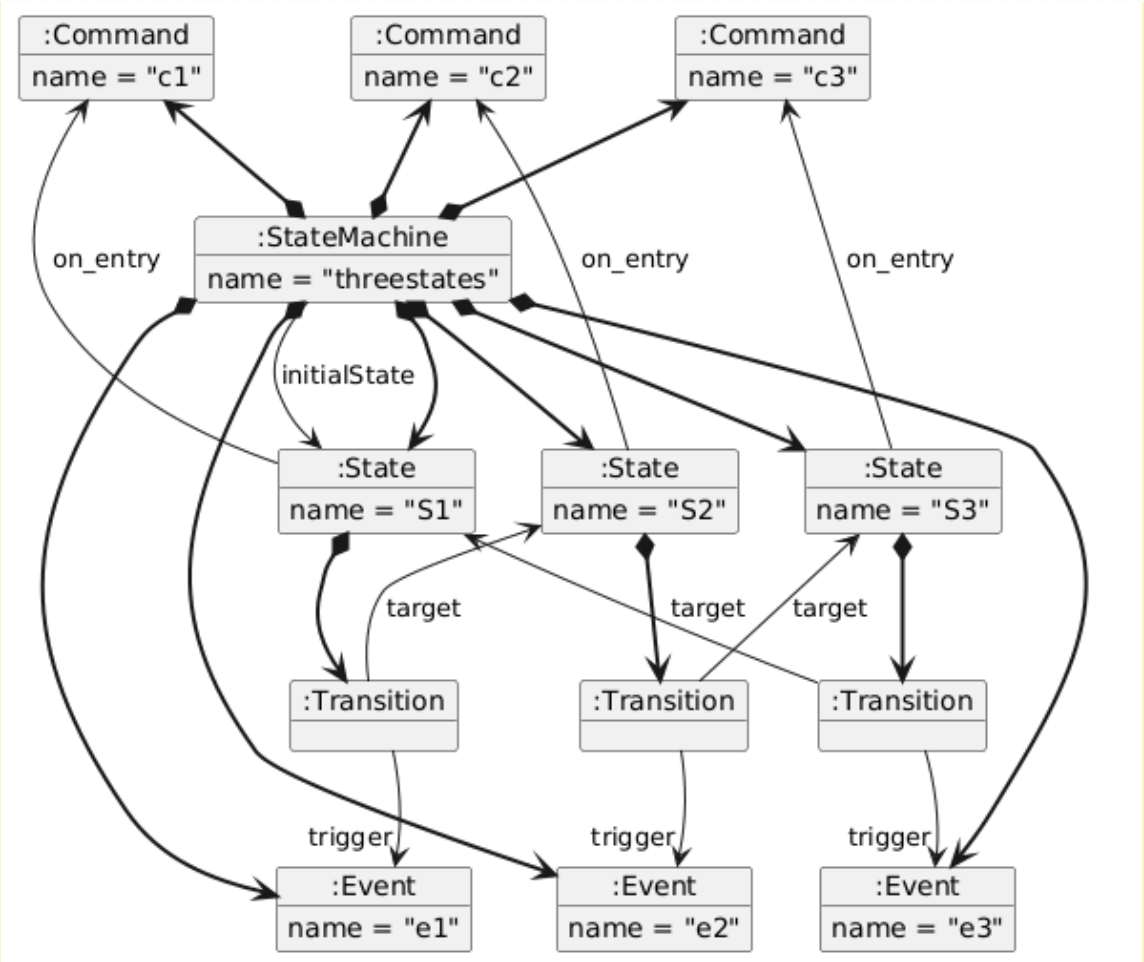
Example 36. Code generator using string templates and visit functions for the State Machines DSL

Using string templates and visit functions, we would like to implement a code generator for the State Machines DSL. The idea is to be able to generate, from a valid state machine, a Java class which roughly follows the state design pattern [https://refactoring.guru/design-patterns/state].

For example the following state machine:

```
1  statemachine threestates
2
3  events:
4      e1, e2, e3
5
6  commands:
7      c1, c2, c3
8
9  initial: S1
10
11 state S1:
12     on_entry: c1
13     e1 => S2
14 end
15
16 state S2:
17     on_entry: c2
18     e2 => S3
19 end
20
21 state S3:
22     on_entry: c3
23     e3 => S1
24 end
```

Which can be parsed in the following AST:



Should result in the following Java code:

```
1  package threestates;
2
3  public class threestates {
4
5      State currentState = new S1();
6
7      public static void c1() {
8          System.out.println("Executing command c1");
9          // TODO command provide implementation
10     }
11
12     public static void c2() {
13         System.out.println("Executing command c2");
14         // TODO command provide implementation
15     }
16
17     public static void c3() {
18         System.out.println("Executing command c3");
19         // TODO command provide implementation
20     }
21
22     public void processEvent(Event event) {
23         System.out.println("Processing event " + event);
24         switch (event) {
25
```

```
26              case e1:
27                  this.currentState = this.currentState.e1();
28                  break;
29              case e2:
30                  this.currentState = this.currentState.e2();
31                  break;
32              case e3:
33                  this.currentState = this.currentState.e3();
34                  break;
35          }
36          System.out.println("Current state: " +
37                          this.currentState.getClass().getSimpleName());
38
39          if (currentState != null) {
40              currentState.entryCommands();
41          }
42      }
43
44      public enum Event {
45          e1, e2, e3
46      }
47
48      interface State {
49          State e1();
50          State e2();
51          State e3();
52          void entryCommands();
53      }
54
55      class S1 implements State {
56
57          @Override
58          public State e1() {
59              return new S2();
60          }
61
62          @Override
63          public State e2() {
64              return this;
65          }
66
67          @Override
68          public State e3() {
69              return this;
70          }
71
72          @Override
73          public void entryCommands() {
74              threestates.c1();
75          }
76
77      }
78
79      class S2 implements State {
80
81          @Override
82          public State e1() {
```

```
            return this;
        }

        @Override
        public State e2() {
            return new S3();
        }

        @Override
        public State e3() {
            return this;
        }

        @Override
        public void entryCommands() {
            threestates.c2();
        }

    }

    class S3 implements State {

        @Override
        public State e1() {
            return this;
        }

        @Override
        public State e2() {
            return this;
        }

        @Override
        public State e3() {
            return new S1();
        }

        @Override
        public void entryCommands() {
            threestates.c3();
        }
    }
}
```

We can achieve this result with the following code generator:

```
import { StateMachine, Event, Command, State, Transition } from
"./generated/ast.js"

export function generateStateMachineCode(stateMachine: StateMachine): string {
    return `
    package ${stateMachine.name};

    public class ${stateMachine.name} {

```

```
        State currentState = new ${stateMachine.initialState.ref?.name}();

        ${stateMachine.commands.map(command => generateCommandDeclarationCode(command)).join("\n")}

        public void processEvent(Event event) {
            System.out.println("Processing event " + event);
            switch (event) {
                ${stateMachine.events.map(event => generateStateMachineEventHandler(event)).join("\n")}
            }
            System.out.println("Current state: " + this.currentState.getClass().getSimpleName());

            if (currentState != null) {
                currentState.entryCommands();
            }
        }

        public enum Event {
            ${stateMachine.events.map(event => event.name).join(",")}
        }


        interface State {
            ${stateMachine.events.map(event => generateEventDeclarationCode(event)).join("\n")}
            void entryCommands();
        }

        ${stateMachine.states.map(state => generateStateCode(state)).join("\n")}

    }
    `

}

function generateStateCode(state: State): string {
    return `

        class ${state.name} implements State {

            ${state.$container.events.map(event => generateStateEventHandlerCode(state, event)).join("\n")}

            @Override
            public void entryCommands() {
                ${state.on_entry.map(commandRef => generateCommandCall(commandRef.ref!)).join("\n")}
            }

        }
    `
}

function generateCommandDeclarationCode(command: Command): string {
    return `
```

```
        public static void ${command.name}() {
            System.out.println("Executing command ${command.name}");
            //TODO command provide implementation
        }
    `
}

function generateEventDeclarationCode(event: Event): string {
    return `
        State ${event.name}();
    `
}

function generateStateEventHandlerCode(state: State, event: Event): string {
    return `
        @Override
        public State ${event.name}() {
            return ${generateEventHandlerReturn(state, event)};
        }
    `
}

function generateEventHandlerReturn(state: State, event: Event) {
    let targetState : State = findTargetState(state, event);
    if (targetState === state){
        return "this"
    } else {
        return `new ${targetState.name}()`
    }
}

function findTargetState(state: State, event: Event): State {
    const candidateTransitions: Transition[] = state.transitions.filter
(transition => transition.trigger.ref! === event)
    if (candidateTransitions.length > 1) {
        throw "Generation error: cannot process a non deterministic state
machine"
    } else if (candidateTransitions.length == 0) {
        return state
    } else {
        return candidateTransitions.pop()?.target.ref!
    }
}


function generateStateMachineEventHandler(event: Event): string {
    return `
        case ${event.name}:
            this.currentState = this.currentState.${event.name}();
            break;
    `
}

function generateCommandCall(command: Command): string {
    return `
        ${command.$container.name}.${command.name}();
    `
```

```
115 }
```

Integrating the code generator in the CLI



In this section we assume that, when your Langium project was generated using Yeoman, you chose to *not* generate an example project.

Once the code generator code has been implemented for a given Langium language, the last step is to provide a user interface to make the code generator available to language users. In Langium by default this is achieved by implementing a *generate* command in the Command Line Interface (CLI) of the language.

A Langium project freshly created using Yeoman should have the following files:

- `packages/cli/src/main.ts` contains the entry point of the CLI, and should include a `generateAction` function. This function is called when the `generate` keyword is given to the CLI, and it accomplishes two important tasks:
  - `extractAstNode` is the function that will call your parser in order to produce an AST, that is then stored in the `model` varible,
  - `generateOutput` is the main function of your code generator provided in the `packages/cli/src/generator.ts` file.
- `packages/cli/src/generator.ts` contains the code generator itself, and in particular a main function called `generateOutput`.

In order to integrate your code generator in this architecture, put your code generator in the `packages/cli/src/generator.ts` file, and call your code generation functions from the `generateOutput` function. Change the `generateOutput` as much as required to create folders as needed by your code generation logic.

Example 37. Integration of the State Machines DSL code generator in the CLI

For the State Machines DSL, we start in the `packages/cli/src/generator.ts`, and we look at the `generateOutput` function. Then we modify this function to properly create the Java package folder required for the generated Java class, and to prepare the path to the generated Java file, before calling the code generator:

```
export function generateOutput(model: StateMachine, source: string, destination:
string): string {
    // Prepare generation paths
    const data = extractDestinationAndName(destination);
    const generatedJavaPackageName = model.name
    const generatedJavaFileName = `${model.name}.java`
    const generatedFolderPath = path.join(data.destination,
                                          generatedJavaPackageName);
    const generatedFilePath = path.join(generatedFolderPath,
                                          generatedJavaFileName)

    // Call the code generator
    const fileContents = generateStateMachineCode(model)

    // Write the generated code in a file
```

```
    fs.mkdirSync(generatedFolderPath, { recursive: true });
    fs.writeFileSync(generatedFilePath, fileContents);
    return generatedFilePath;
}
```

We then run `npm run build` and run our code generator with the CLI :

```
node ./packages/cli/bin/cli.js generate examples/threestates.sm
```

which produces the file `generated/threestates/threestates.java`.

6.4. References

- 🔗 Langium official documentation [https://langium.org/docs], by TypeFox (accessed on 11/2025)